University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Jacek Karwowski
Student no. 359713

Formal semantics of a reversible language: symmetric groups in Homotopy Type Theory

Master's thesis
in MATHEMATICS

Supervisors:

prof. dr hab. Marek Zawadowski
Institute of Mathematics, University of Warsaw

prof. dr hab. Bartosz Klin
Department of Computer Science, University of Oxford

Warsaw, June 2022

Abstract

We construct categorical semantics for a reversible programming language Π, using Homotopy Type Theory, and formalise it in Agda proof assistant. The thesis first outlines the full proof characterising the equivalence between the weak free symmetric rig groupoid on zero generators and the univalent universe of finite sets and bijections, and then describes in detail the part of the proof that formalises a certain presentation of the symmetric group $\mathcal{S}_n$.

Keywords

Category theory, denotational semantics, Homotopy Type Theory, Curry-Howard-Lambek correspondence, group theory, rewriting theory

Thesis domain (Socrates-Erasmus subject area codes)

11.1 Matematyka

Subject classification

18 Category theory
18C Categories and theories
18C50 Categorical semantics of formal languages

Tytuł pracy w języku polskim

Formalna semantyka języka obliczeń odwracalnych: grupy symetrii w Homotopijnej Teorii Typów

Contents

1. Introduction . . . 5
- 1.1. Technical content . . . 6
- 1.2. Outline . . . 6

2. Category theory . . . 9
- 2.1. Basic categorical notions . . . 9
  - 2.1.1. Categories . . . 9
  - 2.1.2. Functors . . . 10
  - 2.1.3. Natural transformations . . . 10
- 2.2. Cartesian closed categories . . . 11
- 2.3. Monoidal categories . . . 13
- 2.4. Interlude: coherence conditions . . . 14
- 2.5. Symmetric monoidal categories . . . 15
- 2.6. 2-groupoids . . . 16
- 2.7. Weak symmetric rig groupoids . . . 17

3. Trinitarianism . . . 19
- 3.1. Lambda calculus . . . 19
- 3.2. Simply-typed λ-calculus . . . 22
- 3.3. Extending STLC . . . 23
- 3.4. Minimal logic . . . 24
- 3.5. (Negative fragment of) intuitionistic propositional logic . . . 24
- 3.6. STLC and CCC . . . 25
- 3.7. Curry-Howard-Lambek correspondence . . . 25

4. Homotopy Type Theory . . . 27
- 4.1. Martin-Lof type theory . . . 27
  - 4.1.1. Type universes . . . 28
  - 4.1.2. Context rules . . . 28
  - 4.1.3. Structural rules . . . 28
  - 4.1.4. Primitive types . . . 29
- 4.2. Identity type . . . 31
  - 4.2.1. Identity type operations . . . 32
- 4.3. Univalence axiom . . . 34
- 4.4. Homotopy levels . . . 35
- 4.5. Inductive types . . . 36
- 4.6. Higher inductive types . . . 38
- 4.7. Finite types . . . 39

4.8. Univalent universes . . . 39

5. Reversible programming language . . . 41
5.1. Types . . . 41
5.2. Programs . . . 41
5.3. Program equivalences . . . 42
5.4. Equivalences of equivalences . . . 43
5.5. Intended semantics . . . 43

6. Equivalence of $\mathbf{\Pi}_{\mathrm{cat}}$ and $\mathcal{U}_{\mathsf{Fin}}$ . . . 45
6.1. Π^{+} language . . . 45
6.2. $\Pi^{\wedge}$ language . . . 46
6.3. Symmetric group . . . 46
6.4. Syntactic categories . . . 46
6.5. Proof outline . . . 47

7. The Group of Permutations . . . 49
7.1. Presenting the permutation group . . . 49
7.2. Groups . . . 50
7.3. Free groups . . . 50
7.4. Group presentations . . . 52
7.5. Rewriting via Coxeter . . . 53
7.6. Lehmer Codes . . . 57
7.7. Running Lehmer Codes . . . 58

8. Complexity questions . . . 61
8.1. Π program equivalence problem . . . 61
8.2. Synthesising short Π programs . . . 63

9. Proof assistant . . . 65
9.1. Normalisation-by-evaluation . . . 65
9.2. Transfer of theorems . . . 66
9.3. Word of caution . . . 67

10. Discussion . . . 69
10.1. Curry-Howard-Lambek correspondence revisited . . . 69
10.2. $\mathcal{U}_{\mathsf{Fin}}$ and **FinBij** . . . 69
10.3. FSMG . . . 70
10.4. Future work . . . 70
10.5. Note on tools . . . 71

A. Full syntax of Π . . . 77

B. Full syntax of Π^{+} . . . 83

C. Full syntax of $\Pi^{\wedge}$ . . . 87

Chapter 1

Introduction

Landauer principle places a lower limit on how much energy has to be expended to perform one *unit of computation* - how much energy is needed to erase one bit of information. The limit is, as of today, mostly of theoretical significance - but being optimistic about the future, it will not always be. One way of avoiding it, and thus unlocking much denser hardware implementations of computing machines, is to never erase information - i.e, to do it in a *reversible* way. This is how we arrive at reversible computing - a programming paradigm, in which all computations can be run in both forward- and backward- directions. To study properties of reversible computing, we use a toy programming language named Π, from *permutation*.

This is the first part of the story.

The second part is about writing proofs formally, in a proof assistant. At first sight, using a PA makes proving things more difficult and tedious - formalisation works on a very low-level, and all those abstraction layers usually accessible to a mathematician have to be developed from scratch. However, besides having the proof formally verified, working in a proof assistant can also make for better ideas. The prohibitive amount of implementation effort required to re-use classical techniques can be a source of motivation to simplify, and invent alternative techniques. Overall, formal setting makes proving things very different.

The third part of the story is about a particular type theory we use - namely, Homotopy Type Theory/Univalent Foundations, as developed in the book [Univalent Foundations Program, 2013]. Many currently used proof assistants, including the one that we employ here, are based on some kind of type theory; the choice of the type theory influencing how easy (or difficult) certain constructions are.

The grand story arch that binds these is the connection between programming, logic and category theory, referenced under the name of trinitarianism, or Curry-Howard-Lambek correspondence. It reveals itself, and appears in this thesis, on many levels and in different contexts.

- It enables us to use HoTT as the underlying (meta-)logic, with its dependent version of Brouwer-Heyting-Kolomogorov interpretation of propositions-as-types
- It guided the design of Π, and inspired us to phrase our proof in terms of denotational semantics for a programming language
- It enables extracting our proofs as executable computer procedures, which can then be

used to manipulate Π programs and permutations

- It provides an interpretation for a univalence axiom - previously understood in terms of homotopy equivalence - in terms of the semantics of programming languages.

I find C-H-L correspondence to be one of the most fascinating and inspiring ideas in the whole of mathematics and computer science - and hope to do it justice in chapters that follow.

1.1. Technical content

The focus of this work is to formally prove that the groupoid of finite types and bijections is equivalent to the free symmetric rig groupoid on zero generators[1]. The proof is developed as a construction of denotational semantics for a reversible language Π, which codifies the properties and coherence laws of weak symmetric rig groupoids.

The proof is done in several stages. First, the full language is rewritten in its additive fragment Π^+, corresponding, categorically, to a free symmetric monoidal groupoid on one generator. Then, it is converted to an intermediate representation $\Pi^\wedge$ (categorically - a PROP), which simplifies the types to natural numbers, and adjusts the combinators correspondingly.

Programs are then transformed to an abstract representation of lists of adjacent transpositions, while the programs' equalities are written as rules of a rewriting system inspired by the Coxeter presentation of the symmetric group. Writing it in this form allows us to use the rewriting system for the program normalisation, which is crucial to map the "syntactic" representation of the program to its "extensional" form. The normal forms of lists are described using Lehmer codes. These are then finally evaluated to concrete bijections, which form the loopspace of the groupoid of finite types and bijections $\mathcal{U}_{\mathsf{Fin}}$. This groupoid can be proven to form a univalent subuniverse in Homotopy Type Theory.

This thesis concerns itself mostly with the treatment of $\mathcal{S}_n$ – which was done mostly by myself – while the rest of the proof is outlined in general terms, as it was done in collaboration with my co-authors.

1.2. Outline

Chapter 2 introduces the basic notions of category theory that are needed to understand the contents of this thesis. It is not meant to be comprehensive, and consists mostly of definitions, with few examples or theorems, and no proofs. The final goal is to have a definition of the weak symmetric rig groupoid, which is the primary algebraic structure used to state the main theorem in Chapter 6.

Chapter 3 presents the connection between logic, computation and category theory, using the prototypical example of λ-calculus, intuitionistic logic, and Cartesian-closed categories. It is meant mostly as a reference and the starting point for the later chapters on Homotopy type theory, and reversible C-H-L.

[1] With certain subtleties regarding the definitions of both objects - more about this issue in the Chapter 10.

Chapter 4 introduces the technical background of Homotopy Type Theory/Univalent Foundations, as described in [Univalent Foundations Program, 2013]. Writing a self-contained introduction would require essentially copying the first hundred or so pages of the HoTT-book, which would certainly be outside of the scope of this thesis. However, we still try to provide at least the definitions of all the technical devices used afterwards, along with some motivating examples and short, relevant proofs. While our main theorems in Chapter 6 and Chapter 7, as written here, do not use much of the HoTT proper, they are a de-formalisation of HoTT-Agda code, so probably cannot be understand without this background.

Chapter 5 presents the reversible language Π, which is the focus on this thesis. The full syntax of the language is relegated to the Appendix A.

Chapter 6 outlines the full proof of the correspondence between Π and the groupoid of finite types and bijections, formalised as the symmetric rig equivalence of weak 1-groupoids. The description does not go into technical details, which are nevertheless present in the code attached to this thesis.

Chapter 7 is the most technical and detailed part of the thesis, describing the part of the proof that shows an equivalence between the presentation of $\mathcal{S}_n$ and $\mathsf{Aut}(\mathsf{Fin}_n)$ (it being the loopspace of the univalent universe $\mathcal{U}_{\mathsf{Fin}_n}$). We use notions of the Coxeter presentation of $\mathcal{S}_n$ and Lehmer codes, which are introduced gradually, as they are needed. This part of the proof is also fully formalised in a proof assistant, and included as a code appendix.

Chapter 8 discusses some complexity-related questions that arise from the proof. In particular, it shows how does the question of whether the procedures extracted from the proof in Chapter 7 can be made more efficient is related to the $\mathsf{co\text{-}NP} \subset \mathsf{NP}$ problem.

Chapter 9 describes the advantages and disadvantages of working in a constructive setting, with a proof assistant, showcasing an example of theorem transfer between different representations of permutations.

Chapter 10 provides an overview of the different areas the proof relates, including the reversible version of Curry-Howard-Lambek isomorphism. It also explains some issues regarding the relationship between the universe of finite types and bijections $\mathcal{U}_{\mathsf{Fin}}$, and the category of sets and bijections **FinBij**, the algebraic and syntactic *freeness* of the free symmetric rig groupoid, considers some potential avenues for future work, and lists the tools that were used in the development of this thesis.

Appendix A provides the full syntax of the Π language introduced in Chapter 5, and Appendix B together with Appendix C provide the full syntax of the Π^+ and $\Pi^\wedge$ languages discussed in Chapter 6.

The code attached to this thesis formalises all of the proofs Chapter 7 (modulo the termination property of the rewriting system), as well as all the statements and majority of proofs from Chapter 6.

Acknowledgements

The majority of the technical work presented here has been done under the guidance of prof. Amr Sabry, and in collaboration with Vikraman Choudhury. Almost all of it was published as [Choudhury et al., 2022], with the formalisation included in [Choudhury and Karwowski, 2021]. Part of the work has been funded by the Indiana University summer fellowship "Global Talent Attraction Program".

The thesis has been written with the support of my advisors: prof. Bartek Klin (previously at the University of Warsaw, now at the University of Oxford) and prof. Marek Zawadowski (at the University of Warsaw). I am grateful for all of the time they graciously spent working with me on this project, the feedback they provided, my numerous errors and mistakes they have spotted, and their infinite patience and support in getting it done.

Chapter 2

Category theory

Category theory is the main tool we will use to organise the proof on the high-level. This chapter presents the fundamentals required to define the objects used in Chapter 3 (categorical semantics of λ-calculus) and Chapter 6 (categorical semantics of the reversible language Π).

2.1. Basic categorical notions

2.1.1. Categories

Definition 2.1.1 (Category)**.** *A category $\mathbb{C}$ consists of a collection of objects* $\mathrm{Ob}(\mathbb{C})$*, and, for every pair of objects A, B in the category, a set of morphisms* $\mathrm{Hom}(A, B)$*, such that the following conditions are true:*

- *for objects A, B, C and morphisms $f : \mathrm{Hom}(A, B)$ and $g : \mathrm{Hom}(B, C)$ there exists a morphism $g \circ f : \mathrm{Hom}(A, C)$, which we call composition of f and g.*
- *for each object A, there is an identity morphism $\mathrm{id}_A : \mathrm{Hom}(A, A)$, such that for any other morphism $f : \mathrm{Hom}(A, B)$ we have $f \circ \mathrm{id}_A = f$, and similarly, for any $g : \mathrm{Hom}(B, A)$ we have $\mathrm{id}_A \circ g = g$.*
- *the composition operation is associative, that is, for any objects A, B, C, D and any morphisms $f : \mathrm{Hom}(A, B)$, $g : \mathrm{Hom}(B, C)$ and $h : \mathrm{Hom}(C, D)$ we have $h \circ (g \circ f) = (h \circ g) \circ f$.*

We will sometimes use a notation of $f : A \to B$ to describe $f : \mathrm{Hom}(A, B)$. We will also write $A : \mathbb{C}$ instead of $A : \mathrm{Ob}(\mathbb{C})$.

Categories would not be useful on their own, if we didn't have a notion of a map between them.

Definition 2.1.2 (Functor)**.** *A functor between categories $\mathbb{C}$ and $\mathbb{D}$ is given by a mapping of objects* $\mathrm{Ob}(\mathbb{C}) \to \mathrm{Ob}(\mathbb{D})$*, and, for each pair of objects $A, B : \mathbb{D}$, a function between* $\mathrm{Hom}(A, B)$ *and* $\mathrm{Hom}(F(A), F(B))$*, satisfying the following conditions:*

- *Identity is preserved, that is, for any object $A : \mathbb{C}$ we have $F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$.*
- *Composition is preserved, that is, for any objects $A, B, C : \mathbb{C}$ and any morphisms $f :$ $\mathrm{Hom}(A, B)$ and $g : \mathrm{Hom}(B, C)$ we have $F(g \circ f) = F(g) \circ F(f)$.*

Again, a small notation remark is that we will write $F : \mathbb{C} \to \mathbb{D}$ for the functor between $\mathbb{C}$ and $\mathbb{D}$. We will say that a category is a *small category* if the collection of its objects forms a set.

Examples of categories include:

- **Set**, where objects are sets and morphisms are functions,
- **Top**, where objects are topological spaces and morphisms are continuous transformations,
- **Grp**, where objects are groups and morphisms are group homomorphisms,
- A Boolean algebra, that is, a (complemented distributive) lattice, where objects are sets, and for any objects A, B, there is a unique morphism $A \to B$ if and only if $A \leq B$.

Definition 2.1.3 (Poset)**.** *We say that a category $\mathbb{C}$ is a poset, if for any objects $A, B : \mathbb{C}$, we have at most one morphisms $f : \mathrm{Hom}(A, B)$.*

2.1.2. Functors

Functors are mapping between categories - relevant lemmas show that they can be composed, similar to functions.

Lemma 2.1.4. *For any two functors $F : \mathbb{C} \to \mathbb{D}$ and $G : \mathbb{D} \to \mathbb{E}$, there exists a functor $G \circ F$ given by the composition of the maps on objects and on morphisms.*

Lemma 2.1.5. *For any category $\mathbb{C}$, there exists an identity functor $\mathbf{1}_{\mathbb{C}} : \mathbb{C} \to \mathbb{C}$, defined as acting as identity on objects and on morphisms. Identity functor is a neutral element wrt the functor composition, that is, $\mathbf{1}_{\mathbb{D}} \circ F = F$ and $G \circ \mathbf{1}_{\mathbb{C}} = G$ for any functors $F : \mathbb{C} \to \mathbb{D}$ and $G : \mathbb{D} \to \mathbb{C}$.*

Lemma 2.1.6. *Functor composition is associative, that is, for any three functors $F : \mathbb{C} \to \mathbb{D}$, $G : \mathbb{D} \to \mathbb{E}$ and $H : \mathbb{E} \to \mathbb{F}$, we have $H \circ (G \circ F) = (H \circ G) \circ F$ - equality holding on objects, as well as on morphisms.*

Proofs of those lemmas follow easily, and I omit them here. What is important is that taken together, they prove the following proposition.

Proposition 2.1.7. *There is a category **Cat**, its objects being small categories, and morphisms being functors between them.*

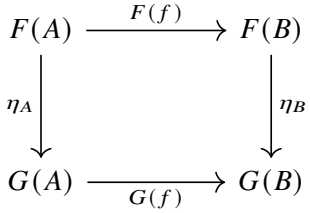
2.1.3. Natural transformations

> I didn't invent categories to study functors; I invented them to study natural transformations.
>
> Saunders Mac Lane

Finally, the last piece of the puzzle is getting one level higher and describing mapping between functors.

Definition 2.1.8. *Given a pair of functors $F, G : \mathbb{C} \to \mathbb{D}$, a natural transformation η between them is given by, for each object $A : \mathbb{C}$, a morphism $\eta_A : F(A) \to G(A)$, such that for any objects A, B in $\mathbb{C}$ and a morphism $f : A \to B$ between them, we have $\eta_B \circ F(f) = G(f) \circ \eta_A$.*

The last equation can be more easily presented by saying that the following diagram commutes:



As with the functors, we also can define a notion of an identity natural transformation, composition of natural transformations, and prove that the composition operation is associative. We can again state this succinctly by the following proposition.

Proposition 2.1.9. *For any categories $\mathbb{C}, \mathbb{D}$, there is a category $\mathbb{D}^{\mathbb{C}}$, whose objects are functors $\mathbb{C} \to \mathbb{D}$, and whose morphisms are natural transformations between functors.*

It is now clear that the original definition of a category might have been too restrictive. It placed the requirement on the $\mathrm{Hom}(A, B)$ to be a set - while we have already seen that, in the case of **Cat**, morphisms have actually a richer structure of a category themselves. This insight will be developed in the later parts of this chapter, for now, we will still stay with the normal (1-)-categories for a bit.

2.2. Cartesian closed categories

Having a notion of a category, a natural question is: what kinds of objects can we define within it? Since we are working in an arbitrary category, there is, at first sight, not much we can identify - having, as the environment, really only the structure of a multigraph. One idea is to look at a *terminal object* - that is, an object that we can travel to, in a unique way, from any other object.

Definition 2.2.1. *We say that $\top$ is a terminal object in a category $\mathbb{C}$, if for any object $A : \mathbb{C}$, there is a unique morphism $\top_\star : A \to \top$.*

Examples of terminal objects include:

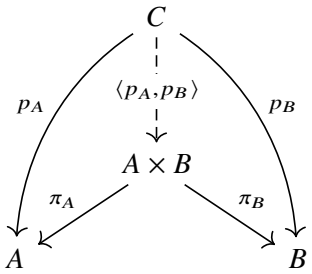
- singleton set in **Set**
- one-point topological space in **Top**
- trivial group in **Grp**
- Truth in a Boolean algebra
- Trivial category in **Cat** - that is, a category with one object $\star$ and one morphism $\mathrm{id}_\star : \mathrm{Hom}(\star, \star)$.

Another, slightly more complicated categorical notion is a product.

Definition 2.2.2. *For a given two objects $A, B : \mathbb{C}$, we say that an object $A \times B$ together with two morphisms $\pi_A : A \times B \to A$ and $\pi_B : A \times B \to B$ is their product, if:*

- *for any other object $C : \mathbb{C}$*
- *and any two morphisms $p_A : C \to A$ and $p_B : C \to B$*

there is a unique morphism $\langle p_A, p_B \rangle$ *such that the following diagram commutes:*



Again, we can point to various examples in the example categories: products of sets, groups and topological spaces, intersection (conjunction) in Boolean algebras. In **Cat**, objects in a product category $\mathbb{C} \times \mathbb{D}$ are pairs of objects (C, D) with components drawn from $\mathbb{C}$ and $\mathbb{D}$ respectively, and morphisms are simply pairs of morphisms.

It is important to note that, from the point of view of category theory, we are not necessarily able to distinguish between different objects of the underlying theory (as we would, if we were able to "look inside" them). For example, *any* singleton set in **Set** is, from our point of view, really the same - because, considering functions, it behaves in the same way with respect to other sets. Thus, terminal objects - as well as products, and other notions we introduce - are usually defined only *up to an isomorphism.*

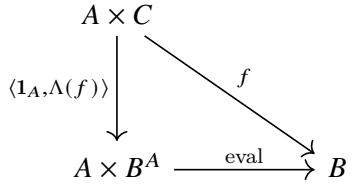
Definition 2.2.3. *For two objects* $A, B : \mathbb{C}$ *and a morphism* $f : A \to B$*, we say that* f *is an isomorphism if there exists* $g : B \to A$*, such that* $g \circ f = \mathrm{id}_A$ *and* $f \circ g = \mathrm{id}_B$*. We say that two objects are isomorphic if there exists an isomorphism between them.*

A quick definition which will be useful later is of a natural isomorphism:

Definition 2.2.4. *A natural transformation* $\eta : F \to G$ *is called a natural isomorphism, if all of its components (all of its constituent morphisms) are isomorphisms.*

The notion of a product is really about answering the question: can we talk about *pairs of objects* without leaving the category that we are working with? Or, in other words, is there something that can play a role of a product? In the same spirit, we can ask a question: for two objects in a category, is there something that can play the role of a function between them?

Definition 2.2.5. *For a category with products* $\mathbb{C}$*, for two objects* $A, B : \mathbb{C}$*, we say that an object* B^A *together with a morphism* $\mathrm{eval} : A \times B^A \to B$ *is an exponential object, if for any other object* C *and a morphism* $f : A \times C \to B$ *there is a unique morphism* $\Lambda(f) : C \to B^A$ *such that the following diagram commutes:*



Definition 2.2.6. *We say that a category* $\mathbb{C}$ *is Cartesian-closed (or* CCC*), if it has a terminal object, products and exponentials.*

Cartesian-closed categories will be crucial in the next chapter, to illustrate the prototypical example of computational trinitarianism – the idea that category theory, logic, and theory of computation are really, deep down, the same subject – three sides of the same coin, as one can put it!

2.3. Monoidal categories

Product is one particular binary operation on a category. However, we can try to understand the intuition of "putting objects in pairs" in a more general sense. This is how we arrive at the notion of a *monoidal category*. The two main sources I have used are: a short note on monoidal categories by [Baez, 2004] [1] and a paper by [Sabry et al., 2021].

Definition 2.3.1. *A monoidal category is given by a category $\mathbb{M}$, together with*

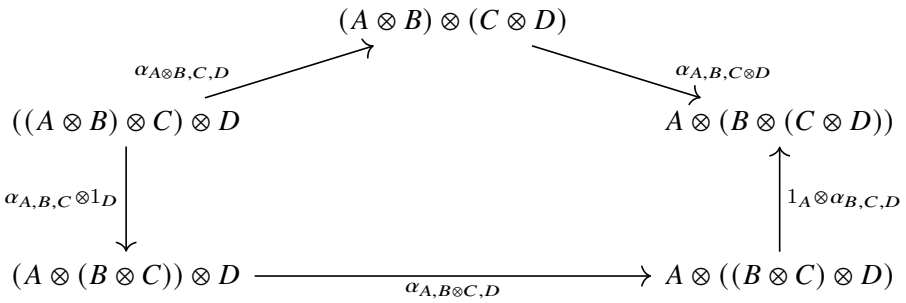
- *A functor $\otimes : \mathbb{M} \times \mathbb{M} \to \mathbb{M}$, called tensor product, written, for a pair of objects $A, B : \mathbb{M}$, in the infix notation $A \otimes B$,*
- *An object $\mathbf{1} : \mathbb{M}$, called identity object,*
- *A natural isomorphism α called associator*

$$\alpha_{A,B,C} : (A \otimes B) \otimes C \leftrightarrow A \otimes (B \otimes C)$$

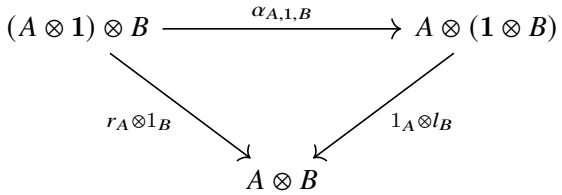
- *Natural isomorphisms l and r called left and right unit*

$$l_A : 1 \otimes A \leftrightarrow A$$
$$r_A : A \otimes 1 \leftrightarrow A$$

such that the pentagon diagram



and the triangle diagram



commute for all objects $A, B, C, D : \mathbb{M}$.

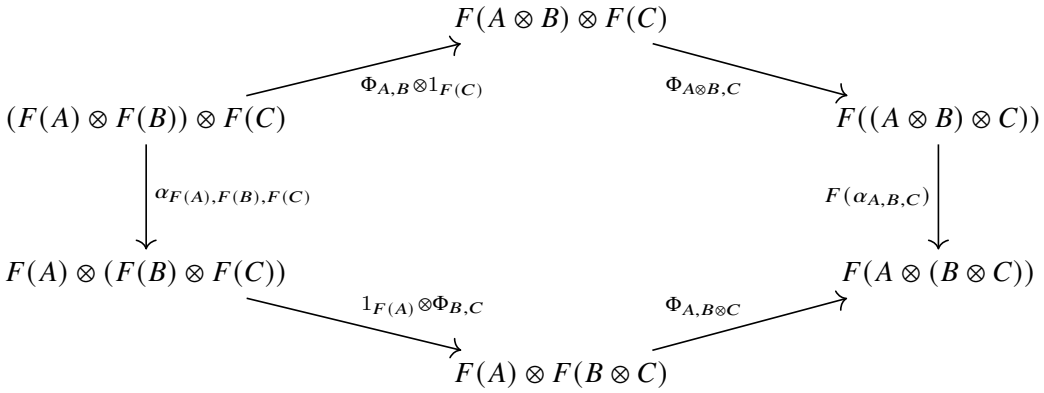
[1] For the intuition on the monoidal categories *actually* coming from the notion of graphical diagrams, I recommend his equally concise talk "Symmetric Monoidal Categories A Rosetta Stone"

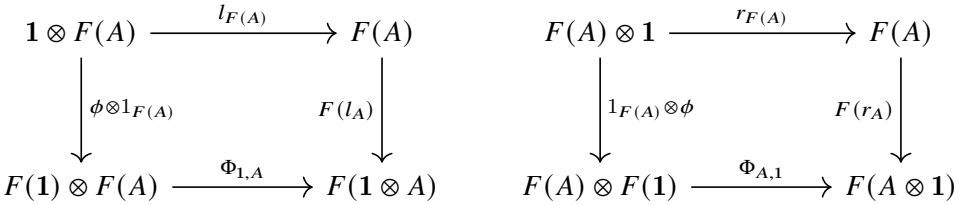
As usual, we also define the (appropriately extended) notions of monoidal functors and monoidal natural transformations.

Definition 2.3.2. *A (strong) monoidal functor between monoidal categories $\mathbb{M}$ and $\mathbb{N}$ is given by*

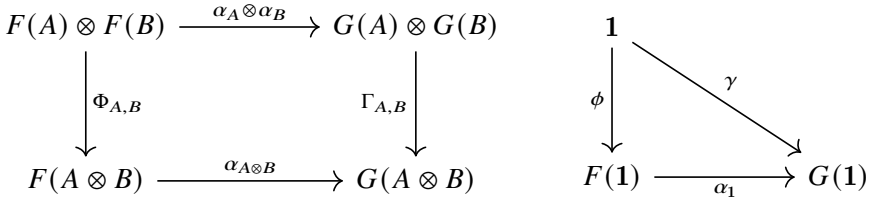
- *A functor $F : \mathbb{M} \to \mathbb{N}$*
- *A natural isomorphism $\Phi_{A,B} : F(A \otimes B) \leftrightarrow F(A) \otimes F(B)$, for objects $A, B : \mathbb{M}$*
- *An isomorphism $\phi : \mathbf{1}_{\mathbb{M}} \leftrightarrow F(\mathbf{1}_{\mathbb{N}})$*

such that the following diagrams commute





Definition 2.3.3. *Given two monoidal functors (F, Φ, ϕ) and (G, Γ, γ), a natural transformation α between F and G is a monoidal natural transformation, if the following diagrams commute*



2.4. Interlude: coherence conditions

The diagrams used in the definition of the monoidal category look quite strange, and one might wonder where they come from. One natural question is: do they fall out "automatically" from somewhere, as it often happens in category theory. The answer to this is, apparently: they do, but this requires a bit of a greater depth and understanding than lies in the scope of this

thesis. [2]

Even though we cannot answer the question how they came into being, we *can* answer a related, easier question: what is their purpose? And the keyword we are looking for here is: these are the first example of *coherence conditions* in action (which are themselves a big focus of this thesis, later on).

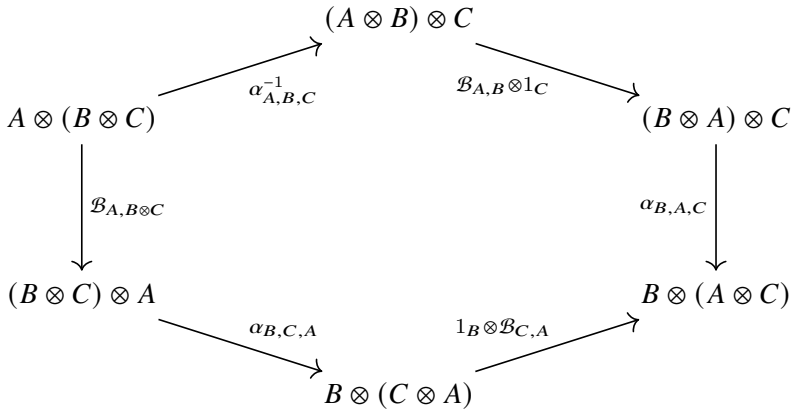
In short, these conditions are precisely what is required for the following property: all diagrams built in $\mathbb{M}$ using tensor product, associators and unitors will commute. This theorem was stated, and proven, by MacLane, as the Coherence Theorem for monoidal categories MacLane [1963]. Or, as Kelly Laplaza [1972] explains - a collection of isomorphisms is considered coherent *if any natural automorphism manufactured from them and their inverses alone (together with identity morphisms) is the identity automorphism.*

This is a property that we would really like to have - after all, associator and unitors don't really "do any work" - but as we are working in a formal setting, it is still necessary to formally add conditions that govern their interactions.

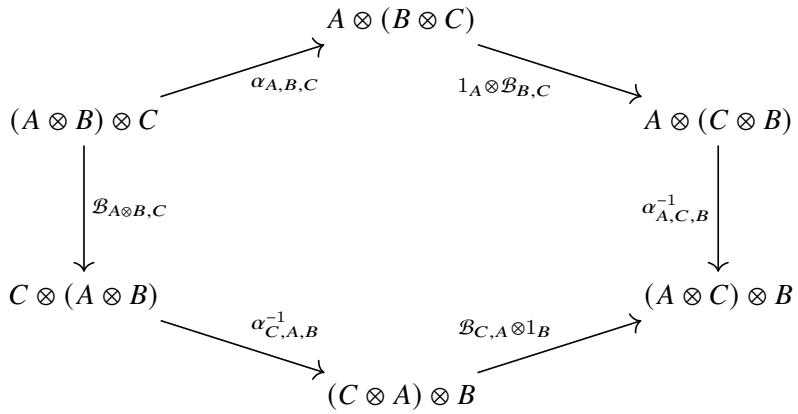
2.5. Symmetric monoidal categories

Until now, the tensor product itself did not have any externally accessible structure - it was not possible to manipulate objects A, B that were put in $A \otimes B$. This is going to change now, as we introduce a braiding operator.

Definition 2.5.1. *A braided monoidal category consists of a monoidal category, given by* $(\mathbb{M}, \otimes, \alpha, l, r)$*, together with a natural isomorphism* $\mathcal{B}_{A,B} : A \otimes B \leftrightarrow B \otimes A$*, such that the following diagrams (called hexagon identities) commute:*



[2] As nLab writes: *In fact a strict monoidal category is just a monoid internal to the category* ***Cat****. Unfortunately this definition is circular, since to define a monoid internal to* ***Cat****, we need to use the fact that* ***Cat*** *is a monoidal category! Furthermore, hardly any of the monoidal categories in nature are strict. So, we need to weaken the definition of monoidal category, and this is where the subtleties come in: we need the associator, left unitor, and right unitor to satisfy some 'coherence laws' — e.g. the pentagon identity.*



Definition 2.5.2. *A symmetric monoidal category consists of a braided monoidal category given by* $(\mathbb{M}, \otimes, \alpha, l, r, \mathcal{B})$*, where the braiding satisfies, for all objects* $A, B : \mathbb{M}$*, the equation* $\beta_{A,B} = \beta^{-1}_{B,A}$.

Definition 2.5.3. *A monoidal functor* F *between braided monoidal(/symmetric) categories* $\mathbb{M}$ *and* $\mathbb{M}'$ *is a braided monoidal(/monoidal symmetric) functor, if the following diagram commutes*

$$\begin{array}{ccc} F(A) \otimes F(B) & \xrightarrow{\mathcal{B}_{F(A),F(B)}} & F(B) \otimes F(A) \\ \downarrow{\scriptstyle \Phi_{A,B}} & & \downarrow{\scriptstyle \Phi_{B,A}} \\ F(A \otimes B) & \xrightarrow{F(\mathcal{B}_{A,B})} & F(B \otimes A) \end{array}$$

There are no additional conditions required for the braided or symmetric monoidal natural transformation - it is enough for them to be monoidal, and go between braided or symmetric monoidal functors.

Definition 2.5.4. *A monoidal(/braided /symmetric monoidal) functor* $F : \mathbb{M} \to \mathbb{N}$ *is a monoidal(/braided /symmetric monoidal) equivalence, if there exists a monoidal(/braided /symmetric monoidal) functor* $G : \mathbb{N} \to \mathbb{M}$ *and monoidal(/braided/symmetric monoidal) natural isomorphisms* $G \circ F \to \mathrm{Id}_{\mathbb{M}}$ *and* $F \circ G \to \mathrm{Id}_{\mathbb{N}}$.

As we have seen, symmetric monoidal equivalence is a fairly strong condition. All of the required coherence conditions guarantee that it preserves the whole symmetric monoidal structure.

2.6. 2-groupoids

In the large part of this thesis, our object of study is, on one hand, homotopy type theory, and on the other, permutations. To understand these categorically, we will need a notion of a groupoid.

Definition 2.6.1. *We say that a category* $\mathbb{C}$ *is a groupoid, if for any objects* A, B *and any morphism* $f : \mathrm{Hom}(A, B)$*,* f *is an isomorphism. By* $\mathbf{Core}(\mathbb{C})$ *we denote a maximal groupoid inside a category* $\mathbb{C}$.

Note that being a groupoid does not mean that any two objects A, B are isomorphic, as there is also a possibility that there are simply no morphisms between A and B.

It is now a good moment to go back a bit, to the very definition of a category. We have already seen that there can be additional structure on the Hom-set: in the case of **Cat**, the Hom-sets are actually categories themselves! This will be a motivating example for the definition of a bicategory.

Definition 2.6.2. *A bicategory (a particular example of a weak 2-category), or **Cat**-enriched category, $\mathbb{C}$ consists of:*

- *a collection of objects* $\mathrm{Ob}(\mathbb{C})$
- *for every pair of objects A, B in the category, a collection of morphisms* $\mathrm{Hom}(A, B)$*, written $A \to B$*
- *for every pair of objects A, B and morphisms $f, g : \mathrm{Hom}(A, B)$, a collection of 2-morphisms* $\mathrm{Hom}(f, g)$*, written $f \Rightarrow g$.*

together with

- *for each object A, an identity morphism* id_A *in* $\mathrm{Hom}(A, A)$
- *for each pair of morphisms $f : \mathrm{Hom}(A, B)$ and $g : \mathrm{Hom}(B, C)$, a composition $g \circ f : \mathrm{Hom}(A, C)$*
- *for each morphism $f : \mathrm{Hom}(A, A)$, an identity 2-morphism $\mathrm{id}_f : \mathrm{Hom}(f, f)$*
- *for each triple of morphisms $f, g : \mathrm{Hom}(A, B)$ and $h : \mathrm{Hom}(B, C)$ and a 2-morphism $\alpha : \mathrm{Hom}(f, g)$, a left whiskering 2-morphism $h \triangleleft \alpha : \mathrm{Hom}(h \circ f, h \circ g)$*
- *for each triple of morphisms $f : \mathrm{Hom}(A, B)$ and $g, h : \mathrm{Hom}(B, C)$ and a 2-morphism $\alpha : \mathrm{Hom}(f, g)$, a right whiskering 2-morphism $\alpha \triangleright f : \mathrm{Hom}(h \circ f, g \circ f)$*
- *for each morphism $f : \mathrm{Hom}(A, B)$, a left unitor $\lambda_f : \mathrm{Hom}(\mathrm{id}_B \circ f, f)$ and an inverse left unitor $\lambda_f^{-1} : \mathrm{Hom}(f, \mathrm{id}_B \circ f)$*
- *for each morphism $f : \mathrm{Hom}(A, B)$, a right unitor $\rho_f : \mathrm{Hom}(f \circ \mathrm{id}_B, f)$ and an inverse left unitor $\rho_f^{-1} : \mathrm{Hom}(f, f \circ \mathrm{id}_B)$*
- *for each triple of morphisms $f : \mathrm{Hom}(A, B)$, $g : \mathrm{Hom}(B, C)$ and $h : \mathrm{Hom}(C, D)$, an associator $\alpha_{f,g,h} : \mathrm{Hom}((h \circ g) \circ f, h \circ (g \circ f))$ and an inverse associator $\alpha_{f,g,h}^{-1} : \mathrm{Hom}(h \circ (g \circ f), (h \circ g) \circ f)$*

such that a suitable collection of equalities holds, such as in [Ahrens et al., 2022].

Definition 2.6.3. *A special case of a bicategory is a poset-enriched category, in which, for each pair of objects A, B and 1-morphisms $f, g : \mathrm{Hom}(A, B)$, there is at most one 2-morphism in* $\mathrm{Hom}(f, g)$ *(that is,* Hom*-sets are posetal categories).*

2.7. Weak symmetric rig groupoids

Until now, we have only concerned ourselves with one symmetric monoidal structure given on a category. However, in many cases, there are actually two such structures - not independent, but interacting with each other. The prototypical example - and, at the same time, our main object of study - are monoidal structures of sums and products. Rig[3] categories are what allows us to generalise these notions.

[3] A pun on a "ring without negation"...

Definition 2.7.1. *A category $\mathbb{M}$ is said to be a symmetric rig category, if comes with two symmetric monoidal structures denoted by $(\oplus, \mathbf{0})$ and $(\otimes, \mathbf{1})$, and with natural isomorphisms*

$$\begin{aligned} d_l &: A \otimes (B \oplus C) \leftrightarrow (A \otimes B) \oplus (A \otimes C) \\ d_r &: (B \oplus C) \otimes A \leftrightarrow (B \otimes A) \oplus (C \otimes A) \\ a_l &: A \otimes \mathbf{0} \leftrightarrow \mathbf{0} \\ a_r &: \mathbf{0} \otimes A \leftrightarrow \mathbf{0} \end{aligned}$$

satisfying a collection of appropriate coherence conditions, listed by [Laplaza, 1972] and by [Kelly, 1974].

Combining this with the bicategorical approach presented earlier, we arrive on the final definition.

Definition 2.7.2. *A weak symmetric rig category is a poset-enriched category with symmetric rig structure, in which monoidal, braided, symmetric and rig laws hold up to 2-morphisms. A symmetric rig category is a weak symmetric rig groupoid if all its morphisms and 2-morphisms are isomorphisms.*

We do not list the coherence laws for weak symmetric rig groupoids here explicitly, as the list is very long, but we do discuss them in the context of the equations in a reversible programming language Π, and give the full list (of more than 100 equations) in Appendix A. Note however, that requiring the category to be poset-enriched guarantees that there are no additional (higher-level) coherence laws required, other than those coming from the rig category definition, as there is at most one 2-morphism between any two compatible 1-morphisms.

Chapter 3

Trinitarianism

> The Christian doctrine of trinitarianism states that there is one God that is manifest in three persons, the Father, the Son, and the Holy Spirit, who together form the Holy Trinity. The doctrine of computational trinitarianism holds that computation manifests itself in three forms: proofs of propositions, programs of a type, and mappings between structures.
>
> These three aspects give rise to three sects of worship: Logic, which gives primacy to proofs and propositions; Languages, which gives primacy to programs and types; Categories, which gives primacy to mappings and structures. The central dogma of computational trinitarianism holds that Logic, Languages, and Categories are but three manifestations of one divine notion of computation. There is no preferred route to enlightenment: each aspect provides insights that comprise the experience of computation in our lives.
>
> Robert Harper

Inspired by the quote above, and having covered the basics of category theory in the last chapter, we can now dive into the two remaining pieces of Trinitarianism - programming, and logic. This chapter presents them in a brief overview, to illustrate the prototypical example of the so-called Curry-Howard-Lambek correspondence.

Being completely honest with the Reader, this particular example is not really what this work is about. However, everything later on - including the Homotopy Type Theory, discussed in the next chapter, and Reversible C-H-L correspondence, which is the proper topic of this thesis - draws on the intuition developed here.

This chapter is based mostly on Urzyczyn [2022], Pitts [2016] and Ker [2009] λ-calculus notes, with the categorical semantics based on Bucciarelli [2022] notes.

3.1. Lambda calculus

Lambda calculus is a formal system used to describe the structure of computation. The objects of study, on the most basic level, are lambda terms - formal strings in a certain language. Assuming an infinite collection of variables $x, y, z, \ldots$, we give the syntax of this language

below:

$$
\begin{aligned}
\langle var \rangle &:= x, y, z, \ldots \\
\langle term \rangle &:= \langle var \rangle && \text{variable} \\
&\quad (\langle term \rangle \langle term \rangle) && \text{application} \\
&\quad \lambda \langle var \rangle . \langle term \rangle && \text{abstraction}
\end{aligned}
$$

We will use letters $M, N, K, L, \ldots$ as meta-variables denoting terms of λ-calculus. We will also assume that the application is left-associative, and that abstraction binds more strongly than application. We will think about the abstraction $\lambda x.M$ as a function which, after applying it to a term N, produces a value $M[x/N]$ - i.e., term M with variable x replaced by (substituted with) term N.

Now, it is a bit unfortunate that we have to work with lambda terms as they are written above, as they, by being formal strings, do distinguish between "identity functions" $\lambda x.x$ and $\lambda y.y$, even though that, for all intents and purposes, these should be equivalent. One solution is to replace variables with so-called de-Bruijn indices, but this makes it difficult to actually read the lambda-terms[1]. What we will do here is to assume that lambda-terms are always considered up to *α-conversion.*

Definition 3.1.1 (Free variables)**.** *For a lambda term M, we define a set of free variables* $\mathrm{FV}(M)$ *by induction:*

$$
\begin{aligned}
\mathrm{FV}(x) &\triangleq \{x\} && \textit{variable} \\
\mathrm{FV}(MN) &\triangleq \mathrm{FV}(M) \cup \mathrm{FV}(N) && \textit{application} \\
\mathrm{FV}(\lambda x.M) &\triangleq \mathrm{FV}(M) - \{x\} && \textit{abstraction}
\end{aligned}
$$

We say that a variable that is not free is a bound variable.

Definition 3.1.2 (α-conversion)**.** *α-conversion is an equivalence relation on λ-terms, written $N =_\alpha M$, which identifies terms N and M if it is possible to derive a common term from both N and M by renaming bound variables to fresh variables, that is, variables not appearing in the terms N, M.*

From this point onward, we will identify the terms which are in the same abstraction class of α-conversion relation. Furthermore, following [Ker, 2009] exposition, we will adopt the *variable convention*, which (informally) states that in any definition, theorem or proof below, we will avoid any clashes between different bound variables and free variables by silently α-converting the terms to a suitable representation.

Now that we have the syntax of the language, we can define its computational behaviour. To do so, we introduce the notion of β-reduction - which will rely on the substitution operation. It is important to note that substitution is only defined up to $=_\alpha$ relation.

Definition 3.1.3 (Substitution)**.** *For a variable x and λ-terms M, N, substitution $M[N/x]$ is*

[1] Although this technique is widely used when it is not humans, but computers, that have to read the expressions!

defined by induction on the structure of M, in the following way:

$$\begin{aligned} x[N/x] &\triangleq N && \text{variable} \\ y[N/x] &\triangleq y && \text{variable, if } x \neq y \\ (KL)[N/x] &\triangleq (K[N/x]L[N/x]) && \text{application} \\ (\lambda y.M)[N/x] &\triangleq \lambda y.M[N/x] && \text{abstraction}, y \neq x, y \notin FV(N) \end{aligned}$$

Note that the definition of substitution is greatly simplified thanks to the variable convention we assumed earlier - for example, we do not have to worry about y appearing free in N.

Definition 3.1.4 (β-reduction)**.** *β-reduction, written $M \to^\beta N$ is a congruence closure of the relation given by reductions of the form $(\lambda x.M)N \to^\beta M[N/x]$. That is, it is defined by induction on the term structure, as follows:*

$$\frac{}{(\lambda x.M)N \to^\beta M[N/x]} \qquad \frac{M \to^\beta M'}{\lambda x.M \to^\beta \lambda x.M'}$$

$$\frac{M \to^\beta M'}{MN \to^\beta M'N} \qquad \frac{N \to^\beta N'}{MN \to^\beta MN'}$$

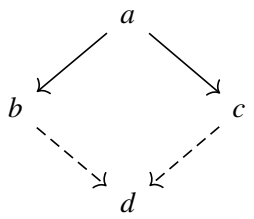
We will write $M \twoheadrightarrow^\beta N$ for the transitive-reflexive closure of $\to^\beta$ reduction relation.

The simplicity of the λ-calculus might be misleading. As defined above, it already exhibits great computational power. In particular, we have the following theorem.

Theorem 3.1.5. *Given any two λ-terms M and N, the question of whether $M \twoheadrightarrow^\beta N$ is undecidable.*

However, another theorem is true, which gives us some control over the behaviour of λ-terms, and which will be very useful (as a generalised tool) in our own proof of normalisation of Coxeter relations-based rewriting system developed in Chapter 7.

Definition 3.1.6 (Diamond property)**.** *We say that a binary relation $\to$ defined on a set A has the diamond property, if for any $a, b, c : A$, if $a \to b$ and $a \to c$, then there exists $d : A$ such that $b \to d$ and $c \to d$. It can be illustrated on a diagram as follows:*



Definition 3.1.7 (Local confluence)**.** *We say that a binary relation $\to$ defined on a set A has the local confluence property, if for any $a, b, c : A$, if $c \leftarrow a \to b$, then there exists $d : A$ such that $b \twoheadrightarrow d \twoheadleftarrow c$, where $\twoheadrightarrow$ denotes the transitive-reflexive closure of $\to$.*

Theorem 3.1.8. *$\to^\beta$ is locally confluent.*

Note that in the statement of the local confluence definition, the only admissible case is when M reduces to N and K in a single step. A stronger property would be as follows.

Definition 3.1.9 (Church-Rosser)**.** *We say that a binary relation $\to$ defined on a set A has the Church-Rosser property, if for any $a, b, c : A$, if $c \twoheadleftarrow a \twoheadrightarrow b$, then there exists $d : A$ such that $b \twoheadrightarrow d \twoheadleftarrow c$, where $\twoheadrightarrow$ denotes the transitive-reflexive closure of $\to$.*

Theorem 3.1.10. *β-reduction for λ-calculus has the Church-Rosser property.*

I omit the proof of this theorem here, as it would be quite specific to λ-calculus - as we will be concerned with a different rewriting system in the later chapters, this part does not generalise well. The important point is, however, that the proof does not follow simply from β-reduction being locally confluent: in case of arbitrary rewriting relation, local confluence does not imply Church-Rosser property, as the following counterexample shows:

$$(1) \longleftarrow (2) \underset{\longrightarrow}{\longleftarrow} (3) \longrightarrow (4)$$

However, local confluence *can* imply Church-Rosser if we assume there are no infinite chains of reductions.

Definition 3.1.11 (Strong normalisation)**.** *We say that a binary relation $\rightarrow$ defined on a set A has a strong normalisation property, if there does not exist an infinite chain of elements of A such that*

$$a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow \dots$$

Theorem 3.1.12 (Newman's lemma)**.** *Local confluence and strong normalisation imply Church-Rosser property.*

Newman's lemma provides an alternative, and possibly simpler, way of establishing Church-Rosser property for rewriting systems that, in contrast to β-reduction in λ-calculus defined above, *are* strongly normalising - an example of which follows in the next section.

3.2. Simply-typed λ-calculus

In the last section, we have said that the behaviour of λ-terms under β-reduction is undecidable. However, we can modify the system a bit, to get the guarantees of termination, by imposing more structure - i.e., requiring the terms to be *typed*.

Starting again from the syntax, we *annotate* simply-typed lambda calculus (STLC) terms with type information. Assuming a collection of primitive types $\tau, \sigma, \dots$, type syntax is given as:

$$\begin{aligned} \langle type \rangle &:= \tau, \sigma, \dots && \text{base types} \\ & \quad \langle type \rangle \rightarrow \langle type \rangle && \text{function types} \end{aligned}$$

We will use $\phi, \psi, \dots$ as meta-variables denoting type terms.

We assume that a well-formed context (denoted by a meta-variable Γ) is a multiset of variables with type annotations.

$$\frac{}{() \ \mathbf{ctx}} \qquad \frac{\Gamma \ \mathbf{ctx}}{\Gamma, x : \phi \ \mathbf{ctx}}$$

We introduce the following rules for typing the λ-terms:

$$\frac{}{\Gamma, (x : \phi) \vdash x : \phi} \qquad \frac{\Gamma, (x : \phi) \vdash M : \psi}{\Gamma \vdash \lambda x.M : \phi \rightarrow \psi} \qquad \frac{\Gamma \vdash M : \phi \rightarrow \psi \qquad \Gamma \vdash N : \phi}{\Gamma \vdash (MN) : \psi}$$

Definition 3.2.1. *We say that a λ-term M is typable if there exists a context Γ and a type ϕ such that $\Gamma \vdash M : \phi$.*

Typable λ-terms constitute a strict subset of all λ-terms. For example, the following terms are not typable:

$$\Omega \triangleq (\lambda x.xx)(\lambda x.xx) \qquad Y \triangleq \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

Thanks to this restriction, strong normalisation *does* hold for STLC.

Proposition 3.2.2. *For a λ-term M, if there exists a context Γ and a type ϕ such that $\Gamma \vdash M : \phi$, then there does not exist an infinite reduction chain*

$$M \to^{\beta} M_1 \to^{\beta} M_2 \to^{\beta} \dots$$

What is more, we can prove that β-reduction is admissible under typability.

Proposition 3.2.3. *For a λ-term M, if there exists a context Γ and a type ϕ such that $\Gamma \vdash M : \phi$, and $M \to^{\beta} N$, then $\Gamma \vdash N : \phi$.*

This means that we can restrict local confluence property to STLC. These propositions together with Newman's lemma give the final theorem.

Theorem 3.2.4. *STLC has the Church-Rosser property.*

This theorem is, of course, already a consequence of the full untyped λ-calculus having the Church-Rosser property. However, using Newman's lemma constitutes a different, and more generalizable proof, which we will mirror in Chapter 7.

3.3. Extending STLC

In the previous section, we have considered only the basic λ-calculus (variables, abstraction and application) with simple types (base types and arrow constructor). In the untyped λ-calculus, we can easily define all sorts of important mathematical objects using so-called *Church encodings*. For example, even though all we have is λ-terms, we can still operate on numbers in a following way:

$$\begin{aligned} 0 &\triangleq \lambda f.(\lambda x.x) \\ 1 &\triangleq \lambda f.(\lambda x.fx) \\ 2 &\triangleq \lambda f.(\lambda x.f(fx)) \\ &\dots \\ n+1 &\triangleq \lambda f.\lambda x.f((nf)x) \end{aligned}$$

And define addition and multiplication

$$+ \triangleq \lambda n.\lambda m.\lambda x.m(n(x)) \qquad \times \triangleq \lambda n.\lambda m.\lambda x.(mn)x$$

Even though $+$ and $\times$ (with some modifications) would still work in STLC, there are limits to this: for example, subtraction, i.e. $n \mapsto m \mapsto \max(0, n-m)$, is not definable in STLC. Thus, to be able to program, we need to add some additional constants and base types to the language. Two examples we consider here are the product type and the singleton type.

We will extend:

- Types with a constant $\top$ for the singleton type, and a binary constructor $\times$ for the product type.

- Base language with the constant (), an operator $\langle -, - \rangle$, and constants π^1, π^2
- Typing rules with

$$\frac{\Gamma \vdash M : \phi \qquad \Gamma \vdash N : \psi}{\Gamma \vdash \langle M, N \rangle : \phi \times \psi}$$

$$\frac{\Gamma \vdash M : \phi \times \psi}{\Gamma \vdash \pi^1(M) : \phi} \qquad \frac{\Gamma \vdash M : \phi \times \psi}{\Gamma \vdash \pi^2(M) : \psi}$$

$$\frac{}{\Gamma \vdash () : \top}$$

- β-reduction rules with[2]

$$\pi^1(\langle M, N \rangle) \to^\beta M$$
$$\pi^2(\langle M, N \rangle) \to^\beta N$$

- As well as, appropriately, the rules for FV(−), α-conversion and substitution.

3.4. Minimal logic

Minimal logic is the implicative fragment of the intuitionistic propositional logic - i.e, a logic with the only logical operator being implication. We can write a Gentzen-style, natural deduction proof system for this logic, as follows.

$$\frac{\Gamma, \phi \vdash_M \phi}{\Gamma \vdash_M \phi \to \phi} \text{ (Axiom)}$$

$$\frac{\Gamma, \phi \vdash_M \psi}{\Gamma \vdash_M \phi \to \psi} (\to \text{ Intro}) \qquad \frac{\Gamma \vdash_M \phi \to \psi \qquad \Gamma \vdash_M \phi}{\Gamma \vdash_M \psi} (\to \text{ Elim})$$

It is now immediately clear, that the deduction system for this logic precisely corresponds to typing rules for STLC, with terms erased:

Theorem 3.4.1. *For a formula ϕ in minimal logic, $\vdash_M \phi$ if and only if there is a STLC term M such that $() \vdash M : \phi$.*

3.5. (Negative fragment of) intuitionistic propositional logic

Negative fragment of intuitionistic propositional logic (IPL) extends the minimal logic with conjunction and Truth. This gives the following additional proof system rules.

$$\frac{\Gamma \vdash_L \phi \qquad \Gamma \vdash_L \psi}{\Gamma \vdash_L \phi \wedge \psi} (\wedge \text{ Intro})$$

$$\frac{\Gamma \vdash_L \phi \wedge \psi}{\Gamma \vdash_L \phi} (\wedge \text{ Elim}) \qquad \frac{\Gamma \vdash_L \phi \wedge \psi}{\Gamma \vdash_L \psi} (\wedge \text{ Elim})$$

$$\frac{}{\Gamma \vdash_L \top} (\top \text{ Intro})$$

As before, we can prove a theorem about this system

Theorem 3.5.1. *For a formula ϕ in the negative fragment of intuitionistic propositional logic, $\vdash_L \phi$ if and only if there is a term M in STLC extended with products and singleton type, such that $() \vdash M : \phi$.*

[2]There is, of course, no β-reduction rule for the singleton type, as there is no elimination rule to match with introduction.

3.6. STLC and CCC

As we have already described the translation between IPL and STLC, let us now show how to interpret terms in STLC extended with products and empty type, in a Cartesian-closed category.

Formally, given a Cartesian-closed category $\mathbb{C}$, and a mapping Ω from base types to $\mathrm{Ob}(\mathbb{C})$, we will define semantics for λ-calculus using $[\![-]\!]$, in the following way:

- Types are mapped to objects in $\mathbb{C}$:

$$\begin{aligned}
[\![\sigma]\!] &\triangleq \Omega(\sigma) && \text{base types}\\
[\![\phi \to \psi]\!] &\triangleq [\![\phi]\!]^{[\![\psi]\!]} && \text{function types}\\
[\![\phi \times \psi]\!] &\triangleq [\![\phi]\!] \times [\![\psi]\!] && \text{product types}\\
[\![\top]\!] &\triangleq \mathbf{T} && \text{singleton type}
\end{aligned}$$

- Contexts are also mapped to objects in $\mathbb{C}$:

$$\begin{aligned}
[\![[\,]]\!] &\triangleq \mathbf{T} && \text{empty context}\\
[\![\Gamma, (x:\phi)]\!] &\triangleq [\![\Gamma]\!] \times [\![\phi]\!] && \text{context, inductive step}
\end{aligned}$$

- A term in a context $\Gamma \vdash M : \phi$ is mapped to a morphism in $\mathrm{Hom}([\![\Gamma]\!], [\![\phi]\!])$:

$$\begin{aligned}
[\![x_1:\phi_1, .., x_n:\phi_n \vdash x_k : \phi_k]\!] &\triangleq \pi_k && \text{variable}\\
[\![\Gamma \vdash MN : \psi]\!] &\triangleq \mathrm{eval} \circ \langle [\![\Gamma \vdash M : \phi \to \psi]\!], [\![\Gamma \vdash N : \phi]\!] \rangle && \text{application}\\
[\![\Gamma \vdash \lambda x.M : \phi \to \psi]\!] &\triangleq \Lambda([\![\Gamma, (x:\phi) \vdash M : \psi]\!]) && \text{abstraction}\\
[\![\Gamma \vdash \langle M, N\rangle : \phi \times \psi]\!] &\triangleq \langle [\![\Gamma \vdash M : \phi]\!], [\![\Gamma \vdash N : \psi]\!] \rangle && \text{pairing}\\
[\![\Gamma \vdash \pi^1(M) : \phi]\!] &\triangleq \pi_1 \circ [\![\Gamma \vdash M : \phi \times \psi]\!] && \text{first projection}\\
[\![\Gamma \vdash \pi^2(M) : \psi]\!] &\triangleq \pi_2 \circ [\![\Gamma \vdash M : \phi \times \psi]\!] && \text{second projection}\\
[\![\Gamma \vdash () : \top]\!] &\triangleq \mathbf{T}_\star && \text{singleton}
\end{aligned}$$

  where the case for the first (/second) projection relies on the existence of a unique ϕ ($/\psi$) such that $\Gamma \vdash M : \phi \times \psi$.

3.7. Curry-Howard-Lambek correspondence

> [It is] sometimes grandly called the "Curry-Howard isomorphism", the small problem being that neither Curry nor Howard invented it, nor is it an isomorphism.
>
> Robert Harper

Finally, we can look at how simply-typed lambda calculus, Cartesian-closed categories, and intuitionistic propositional logic relate to each other! The comparisons are best illustrated in a table.

Lambda Calculus	Categories	Logic
type	object	proposition
λ-term of type A	morphism $\top \to A$	proof of A
product type	categorical product	conjunction
function type	exponential object	implication
singleton type	terminal object	Truth
application	eval	modus-ponens

By extending λ-calculus with positive structure (empty type, sums and negation), Cartesian-closed categories with co-Cartesian structure, and making the logic into the full Intuitionistic Propositional logic, we get additional correspondences.

Lambda Calculus	Categories	Logic
sum type	categorical coproduct	disjunction
empty type	initial object	Falsehood
function into empty type	morphism into initial object	negation

Though not developed here, we will nevertheless going to use even more connections in the next chapter - although the categorical point of view will be undeniably less prominent than the connection between logic and programming.

Type theory	Categories	Logic
universal quantification	dependent product	dependent product type
existential quantification	dependent sum	dependent sum type

For monoidal-closed categories (where the exponential is defined with respect to the tensor product), Curry-Howard-Lambek correspondence goes to linear logic and linear lambda calculus - which is, unfortunately, outside of the scope of this thesis.

But what about Symmetric Rig Groupoids that we spent so much time defining in Chapter 2? Well, defining and proving the Curry(-Howard)-Lambek correspondence for them is precisely the topic of this thesis! We will describe the outline of the proof in Chapter 6, dive into details in 7, and present the full table in Figure 10.1. But to do all that, we need one more tool.

Chapter 4

Homotopy Type Theory

Homotopy Type Theory (HoTT) is an approach to type theory that introduces two novel concepts to a preexisting (intensional) Martin-Lof type theory (MLTT) - namely, univalence axiom, and higher inductive types. Thus, we start with the description of MLTT, and then move to the full HoTT. The presentation in this section is based mainly on [Univalent Foundations Program, 2013], as well as [Escardó, 2018] and [Rijke, 2018].

4.1. Martin-Lof type theory

Martin-Lof type theory, as a deductive system, will consist of judgements and derivation rules. The usual approach to mathematics is to define a deductive system (for example, first-order logic) and work in a particular theory inside this system (such as ZFC). The first-order logic can be thought of having a single form of judgement - saying that a proposition has a proof. The rules of the particular logical calculus (such as Gentzen natural deduction) then dictate what kind of operations we can employ to produce new judgements. In a similar way, in MLTT, a judgement can be of three possible forms:

- Γ **ctx** - Γ is a well-formed context
- $\Gamma : a : A$ - a is an element of type A in the context Γ
- $\Gamma : a_1 \equiv a_2 : A$ - elements a_1 and a_2 are judgmentally equal in the context Γ

Thus, we will no longer identify a judgement with provability (or truthfulness) of a proposition. Instead, a proposition will correspond to a particular type, and a proof of such a proposition will correspond to an element in the proposition's type - similarly to what we saw at the end of Chapter 3.

Judgemental equality can be thought of as equality by definition, or algorithmic equality. Being a judgement means that it lives in the meta-theory - there is no way (and it wouldn't make sense to) "prove" or "disprove" that kind of equality. It is simply expressing the fact that two objects are, from the point of view inside the theory, indistinguishable and fully interchangeable. In addition to expanding the definition, this type of judgement can be used, for example, to express α-conversion. In particular, judgemental equality is thought to be simple, meaning that it should always be decidable to check whether it holds.

4.1.1. Type universes

To avoid Russell-style paradoxes, the theory postulates a cumulative sequence of type universes

$$\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$$

The symbols for the universes are primitive - natural numbers indexing the sequence are thought to live in the meta-theory. There are two rules governing the universes:

$$\frac{\Gamma\ \mathbf{ctx}}{\Gamma \vdash \mathcal{U}_i : \mathcal{U}_{i+1}}\ \mathcal{U}\text{-Intro} \qquad \frac{\Gamma \vdash A : \mathcal{U}_i}{\Gamma \vdash A : \mathcal{U}_{i+1}}\ \mathcal{U}\text{-Cumul}$$

As we will (mostly) use really only a single universe, we will omit the universe index and write simply $\mathcal{U}$.

4.1.2. Context rules

The context, as in the case of λ-calculus, is an ordered list of assumptions - variables and their associated types. The order is important, since the theory is concerned with dependent types, which means that types later in the list can depend on the types earlier. Rules governing the context specify it form inductively:

$$\frac{}{()\ \mathbf{ctx}} \qquad \frac{(x_1 : A_1, x_2 : A_2, \dots x_n : A_n) \vdash A_{n+1} : \mathcal{U}}{(x_1 : A_1, x_2 : A_2, \dots x_n : A_n, x_{n+1} : A_{n+1})\ \mathbf{ctx}}$$

Also, it is possible to exctract a variable from the context:

$$\frac{(x_1 : A_1, x_2 : A_2, \dots x_n : A_n)\ \mathbf{ctx}}{(x_1 : A_1, x_2 : A_2, \dots x_n : A_n) \vdash x_i : A_i}$$

4.1.3. Structural rules

Structural rules describe the meta-theoretic behaviour of the deduction system. They include weakening and substitution:

$$\frac{\Gamma \vdash A : \mathcal{U}_i \qquad \Gamma, \Delta \vdash b : B}{\Gamma, x : A, \Delta \vdash b : B}\ \text{Weak} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma, x : A, \Delta \vdash b : B}{\Gamma, \Delta[a/x] \vdash b[a/x] : B[a/x]}\ \text{Subst}$$

their interaction with judgemental equality:

$$\frac{\Gamma \vdash a : A \qquad \Gamma, x : A, \Delta \vdash b_1 \equiv b_2 : B}{\Gamma, \Delta[a/x] \vdash b_1[a/x] \equiv b_2[a/x] : B[a/x]} \qquad \frac{\Gamma \vdash A : \mathcal{U}_i \qquad \Gamma, \Delta \vdash b_1 \equiv b_2 : B}{\Gamma, x : A, \Delta \vdash b_1 \equiv b_2 : B}$$

as well as the set of rules making the judgemental equality into an equivalence relation that is compatible with itself:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash a \equiv a : A} \qquad \frac{\Gamma \vdash a_1 \equiv a_2 : A}{\Gamma \vdash a_2 \equiv a_1 : A} \qquad \frac{\Gamma \vdash a_1 \equiv a_2 : A \qquad \Gamma \vdash a_2 \equiv a_3 : A}{\Gamma \vdash a_1 \equiv a_3 : A}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash A \equiv B : U_i}{\Gamma \vdash a : B} \qquad \frac{\Gamma \vdash a_1 \equiv a_2 : A \qquad \Gamma \vdash A \equiv B : U_i}{\Gamma \vdash a_1 \equiv a_2 : B}$$

4.1.4. Primitive types

So far, we only specified the deductive structure of the MLTT. However, since we are working in type theory, all other structures are expressed in the same language as the deductive system, the first example of which was the structure of universe types. In addition to the universe types, MLTT postulates a number of primitive types, which we describe now.

Each type in the theory will be described by five main forms of rules:

- *Type formation rule* - specifying what is needed to write out the type
- *Introduction rules* - explaining how to create values of the type
- *Elimination rules* - providing a way to use the values of the type
- *Computation rules* - allowing for simplification of computations in the type - describing, for example, what happens when elimination is applied directly after introduction, similar to the β-reduction from λ-calculus
- *Uniqueness rules* - allowing for treating values of the type as they were in a particular format, similar to the η-rule from λ-calculus (which we haven't really covered in Chapter 3)

Empty type

Empty type, denoted by $\mathbf{0}$, is possibly the simplest type one can add to the theory. We have the following type formation and elimination rules:

$$\frac{}{\Gamma \vdash \mathbf{0} : \mathcal{U}} \qquad \frac{\Gamma \vdash \bot : \mathbf{0} \qquad \Gamma, x : \mathbf{0} \vdash A : \mathcal{U}}{\Gamma \vdash \mathsf{ind}_{\mathbf{0}}(x, A) : A[\bot/x]}$$

The elimination rule expresses the fact that *ex falso sequitur quodlibet*. Empty type has no introduction and computation rules. Notation-wise, we will use $\lambda_{\bot}()$ for the eliminator of $\mathbf{0}$.

Singleton type

> We might make the tongue-in-cheek claim that, while philosophers have expended much ink on the nature of truth, we have now determined that truth is the unit type of functional programming.
>
> Adam Chlipala

The singleton type, denoted $\mathbf{1}$, is defined to have only one element $\star$. The type formation, introduction, elimination, computation and uniqueness rules are given below.

$$\frac{}{\Gamma \vdash \mathbf{1} : \mathcal{U}} \qquad \frac{}{\Gamma \vdash \star : \mathbf{1}} \qquad \frac{\Gamma, x : \mathbf{1} \vdash C : \mathcal{U} \qquad \Gamma \vdash c : C[\star/x] \qquad \Gamma \vdash a : \mathbf{1}}{\Gamma \vdash \mathsf{ind}_{\mathbf{1}}(x, C, c, a) : C[a/x]}$$

$$\frac{\Gamma, x : \mathbf{1} \vdash C : \mathcal{U} \qquad \Gamma \vdash c : C[\star/x] \qquad \Gamma \vdash a : \mathbf{1}}{\Gamma \vdash \mathsf{ind}_{\mathbf{1}}(\star/x, C, c, a) \equiv c : C[\star/x]}$$

We will use $\lambda(\star \mapsto a)$ for the eliminator of $\mathbf{1}$.

Coproduct

Coproduct type, given types A and B, will be a disjoint sum $A + B$, having as elements either left injections $\mathsf{inl}(a)$ for $a : A$, or right injections $\mathsf{inr}(b)$ for $b : B$. The type formation, introduction, elimination, computation and uniqueness rules are given below.

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash B : \mathcal{U}}{\Gamma \vdash A + B : \mathcal{U}}$$

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash B : \mathcal{U} \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{inl}(a) : A + B} \qquad \frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash B : \mathcal{U} \qquad \Gamma \vdash b : A}{\Gamma \vdash \mathsf{inr}(b) : A + B}$$

$$\frac{\Gamma, z : (A + B) \vdash C : \mathcal{U} \qquad \Gamma, x : A \vdash c : C[\mathsf{inl}(x)/z] \qquad \Gamma, x : B \vdash d : C[\mathsf{inr}(x)/z] \qquad \Gamma \vdash e : A + B}{\Gamma \vdash \mathsf{ind}(z, C, c, d, e) : C[e/z]}$$

$$\frac{\Gamma, z : (A + B) \vdash C : \mathcal{U} \qquad \Gamma, x : A \vdash c : C[\mathsf{inl}(x)/z] \qquad \Gamma, x : B \vdash d : C[\mathsf{inr}(x)/z] \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{ind}(z, C, c, d, \mathsf{inl}(a)) \equiv c[a/x] : C[\mathsf{inl}(a)/z]}$$

$$\frac{\Gamma, z : (A + B) \vdash C : \mathcal{U} \qquad \Gamma, x : A \vdash c : C[\mathsf{inl}(x)/z] \qquad \Gamma, x : B \vdash d : C[\mathsf{inr}(x)/z] \qquad \Gamma \vdash b : B}{\Gamma \vdash \mathsf{ind}(z, C, c, d, \mathsf{inr}(b)) \equiv d[b/x] : C[\mathsf{inr}(b)/z]}$$

Finite types

Using the coproduct and the singleton type, we can define a collection of finite types that will be useful in the metatheory.

Definition 4.1.1 (Finite types)**.** *In metatheory, we introduce the following labels for finite types:*

$$\begin{aligned} \mathbf{2} &\triangleq \mathbf{1} + \mathbf{1} \\ \mathbf{3} &\triangleq (\mathbf{1} + \mathbf{1}) + \mathbf{1} \\ \mathbf{4} &\triangleq ((\mathbf{1} + \mathbf{1}) + \mathbf{1}) + \mathbf{1} \\ &\dots \end{aligned}$$

along with the appropriate constructors. For example, for type $\mathbf{3}$*, we will have constructors*

$$\begin{aligned} 0_{\mathbf{3}} &\triangleq \mathsf{inr}(\star) \\ 1_{\mathbf{2}} &\triangleq \mathsf{inl}(\mathsf{inr}(\star)) \\ 2_{\mathbf{3}} &\triangleq \mathsf{inl}(\mathsf{inl}(\star)) \end{aligned}$$

We will only need some finite number of those types, which will be known in advance - by convention, whenever we will use a finite type in this form, we assume it has been also defined in this context.

Dependent product

Dependent product type, for a type A and a type family B indexed by elements of A, is written as $\Pi_{(x:A)} B(x)$, or equivalently $(x : A) \to B$. Its elements are *dependent functions*, i.e. functions whose output's type depends on the input's value.

Dependent product is often read as a universal quantifier, under the types-as-propositions interpretation, as its values prescribe a way of constructing proofs of output propositions for every element in the domain.

The type formation, introduction, elimination, computation and uniqueness rules for the dependent product are given below.

$$\frac{\Gamma \vdash A : U_i \qquad \Gamma, x : A \vdash B : U_i}{\Gamma \vdash \Pi_{(x:A)} B : U_i}$$

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda(x : A).b : \Pi_{(x:A)} B} \qquad \frac{\Gamma \vdash f : \Pi_{(x:a)} B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}$$

$$\frac{\Gamma, x : A \vdash b : B \qquad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x : A).b : \Pi_{(x:A)} B)\ a \equiv b[a/x] : B[a/x]}$$

$$\frac{\Gamma \vdash f : \Pi_{(x:a)} B}{\Gamma \vdash f \equiv (\lambda(x : A).f\ x) : \Pi_{(x:a)} B}$$

Dependent sum

If the dependent product was to be read as the universal quantifier, the dependent sum, written as $\Sigma_{(x:A)} B$ for a type A and a type family B indexed by elements of A, is meant to serve as an existential quantifier in the propositions-as-types interpretation. Its inhabitants are pairs of values (a, b_a) - left element being drawn from A and right element from $B[a/x]$.

The type formation, introduction, elimination and computation rules for the dependent sum are given below. The uniqueness principle is provable in the theory, which is why it is not given as a definitional rule below.

$$\frac{\Gamma \vdash A : U_i \qquad \Gamma, x : A \vdash B : U_i}{\Gamma \vdash \Sigma_{(x:A)} B : U_i}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma, x : A \vdash B : \mathcal{U} \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma_{(x:A)} B}$$

$$\frac{\Gamma, z : \Sigma_{(x:A)} B \vdash\colon C : U_i \qquad \Gamma, x : A, y : B \vdash g : C[(x, y)/z] \qquad \Gamma \vdash c : \Sigma_{(x:a)} B}{\Gamma \vdash \mathsf{ind}_{\Sigma_{(x:a)} B}(z.C, x.y.g, c) : C[c/z]}$$

$$\frac{\Gamma, z : \Sigma_{(x:A)} B \vdash\colon C : U_i \qquad \Gamma, x : A, y : B \vdash g : C[(x, y)/z] \qquad \Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \mathsf{ind}_{\Sigma_{(x:a)} B}(z.C, x.y.g, (a, b)) \equiv g[(a, b)/(x, y)] : C[(a, b)/z]}$$

4.2. Identity type

Identity type, written $a_1 =_A a_2$ for two elements $a_1, a_2 : A$ is the fundamental piece of HoTT that distinguishes it from all other type theories. Its rules, as for other types before, are presented below.

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash a : A \qquad \Gamma \vdash b : A}{\Gamma \vdash a =_A b : \mathcal{U}}$$

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathsf{refl}_a : a =_A a}$$

$$\frac{\Gamma, x : A, y : B, p : x =_A y \vdash C : \mathcal{U} \qquad \Gamma \vdash q : a =_A b \qquad \Gamma, z : A \vdash c : C[z/x, z/y, \mathsf{refl}_z/p]}{\Gamma \vdash \mathsf{ind}_{=_A}(x.y.p.C, z.c, a, b, q) : C[a/x, b/y, q/p]}$$

$$\frac{\Gamma, x : A, y : B, p : x =_A y \vdash C : \mathcal{U} \qquad \Gamma \vdash a : A \qquad \Gamma, z : A \vdash c : C[z/x, z/y, \mathsf{refl}_z/p]}{\Gamma \vdash \mathsf{ind}_{=_A}(x.y.p.C, z.c, a, b, q) \equiv c[a/z] : C[a/x, b/y, \mathsf{refl}_a/p]}$$

Although its elimination rule may seem complicated, the moral sense is very simple: for a type defined over the family of equalities, it is enough to specify the behavior on reflexivity - everything else follows. It is thus called *path induction*, and the type can therefore be also given as an inductively defined type family, in the sense of Section 4.5. We will see it in action in a few examples below. However, one important thing to note is that we require the family C to be uniform over equality! For example, we cannot hope to prove a proposition that states

$$\Pi_{(A:\mathcal{U})}\Pi_{(a:A)}\Pi_{(p:a=_A a)} p =_{a=_A a} \mathsf{refl}_a$$

because the family $C := p =_{a=_A a} \mathsf{refl}_a$ is not uniform over terms a and b. In other words, a path p might not be equal to refl_a as elements of the type $a =_A a$, even though the pair (a, p) will be equal to the pair (a, refl_a) as elements of $\Sigma_{x:A} x =_A a$.

Note that there is no η rule for identity type. If we added it to the theory, it would imply the *uniqueness of identity proofs*, which is incompatible with Homotopy Type Theory - we come back to this in Section 4.3.

4.2.1. Identity type operations

One example property of paths we can prove with path induction is that, for any path $p : a =_A b$, there is an inverse path $p^{-1} : b =_A a$. It is simply an immediate consequence of path induction, applied to the type

$$C := \Pi_{(A:\mathcal{U})}\Pi_{(a:A)}\Pi_{(b:A)}\Pi_{(p:a=_A b)} b =_A a$$

This type - obviously - has an inhabitant $\mathsf{refl}_a : C[a, a, \mathsf{refl}_a]$ - thus, it has an inhabitant for any $p : a =_A b$, which we denote by p^{-1}. This feels a bit magical, as C does not even depend on p, but it is a consequence of equality type being inductively defined.

Definition 4.2.1. *For any type A, elements $a, b, c : A$ and paths $p : a =_A b$ and $q : b =_A c$ we define the path composition $p \cdot q$ by path induction, as an element of the type*

$$C := \Pi_{(A:\mathcal{U})}\Pi_{(a,b:A)}\Pi_{(p:a=_A b)}\Pi_{(c:A)}\Pi_{(q:b=_A c)} a =_A c$$

Then, we can prove the properties of paths:

- For any paths $p : a =_A b$, $q : b =_A c$ and $r : c =_A d$, composition is associative: $(p \cdot q) \cdot r =_{a=_A d} p \cdot (q \cdot r)$
- For any $a, b : A$ and a path $p : a =_A b$, the path refl_a is a neutral element for composition $\mathsf{refl}_a \cdot p = p$ (and similarly for $p \cdot \mathsf{refl}_b = p$)
- Any path $p : a =_A b$ is invertible, the inverse element being the path p^{-1}, giving $p \cdot p^{-1} = \mathsf{refl}_a$ and $p^{-1} \cdot p = \mathsf{refl}_b$

The first two properties are strikingly similar to the axioms of categories we have considered in Chapter 2. Taking into account the third property, it is now clear that types form groupoids – with objects being the type's elements, and morphisms being paths between them, they satisfy all the required axioms. Even more, we can define operations on higher dimensional paths, such as:

Definition 4.2.2. *For a type $A : \mathcal{U}$, elements $a, b, c : A$, (1-)paths $p, q : a =_A b$ and $r : b =_A c$ and a path $\alpha : p =_{a=_A b} q$, we can define (by path induction) left whiskering 2-path:*

$$\alpha \triangleleft r : p \cdot r =_{a=_A c} q \cdot r$$

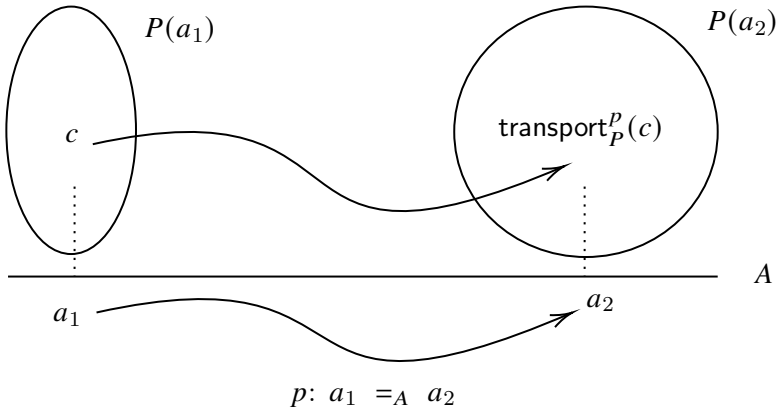


Figure 4.1: Transporting an element between fibers.

In similar manner, we can go on defining right whiskering, unitors, associator, and prove all the required properties, which guarantee that a type is not only a category, but a bicategory! And since all 2-paths are again invertible, it is a 2-groupoid. Repeating this construction ad infinitum, it can be shown that types in Homotopy Type Theory form ∞-groupoids (even though we did not discuss relevant axioms here).

Then, it is natural to ask: since types are categories, what are functions between types? And the answer is that these are functors between categories. By defining a function on elements, we are simultaneously defining it on paths - morphisms. This map is called ap, defined (as everything else up to this point) by path induction.

Definition 4.2.3 (ap)**.** *For any types $A, B : \mathcal{U}$ and a function $f : A \to B$, we can lift it to a functorial action on paths:*

$$\mathsf{ap}_f : \Pi_{(a_1,a_2:A)}\Pi_{(p:a_1=_A a_2)} f(a_1) =_B f(a_2)$$

We can then prove that ap_f is indeed a functor, i.e. that it preserves composition and identity paths, using the computation rule for identity type.

The last important function we discuss here is a straightforward application of path induction to type families.

Definition 4.2.4 (transport)**.** *Using path induction, for any type A, elements $a, b : A$, equality $p : a =_A b$ and type family $\Pi_{(x:A)}P(x)$, we define function*

$$\mathsf{transport}_P^p : P(a) \to P(b)$$

In other words, transport allows us to construct elements in fibers lying over different points in the base type - as long as we have a path between them in the base type. We can represent it in a diagrammatic on the Figure 4.1.

Transport, as ap_f, acts in a functorial way on paths.

Lemma 4.2.5 (Computing with transport)**.** *For any type A we have:*

- $\mathsf{transport}^{\mathsf{refl}_A}_{\mathcal{U}} = \mathsf{id}_A$
- *If* $p : A =_{\mathcal{U}} B$, *then* $\mathsf{transport}^{p^{-1}}_{\mathcal{U}} = (\mathsf{transport}^{p}_{\mathcal{U}})^{-1}$
- *If* $p : A =_{\mathcal{U}} B$ *and* $q : B =_{\mathcal{U}} C$, *then* $\mathsf{transport}^{p \cdot q}_{\mathcal{U}} = \mathsf{transport}^{p}_{\mathcal{U}} \circ \mathsf{transport}^{q}_{\mathcal{U}}$

4.3. Univalence axiom

As far as the equality type is concerned, MLTT makes it quite limited in scope. Path induction, transport, ap and all other similar devices are really convenient to use, but the definition of identity type introduction rule limits the equality to reflexivity. Indeed, in MLTT, the following rule of *uniqueness of identity of proofs* (UIP) is admissible, but not provable [Hofmann and Streicher, 1996]:

Axiom (Axiom UIP)**.**

$$\frac{\Gamma \vdash p : a =_A a}{\Gamma \vdash \mathrm{uip} : p =_{a=_A a} \mathsf{refl}\, a}$$

The *univalence axiom*, on the other hand, allows us to define paths different from refl_a. It is an addition to the theory, which extends equality to a much wider scope, allowing us to treat merely *equivalent* objects as equal. This is what makes the theory so powerful, and convenient. To properly define the univalence axiom, we have to first start by defining what an equivalence is.

Definition 4.3.1 (Homotopy)**.** *We define the type of homotopies between functions* $f, g : A \to B$ *as*

$$f \sim g \triangleq \prod_{a:A} f(a) =_B g(a)$$

We say that functions f, g *are homotopic if there exists a homotopy between them.*

Lemma 4.3.2. *For any types* A, B*, homotopy is an equivalence relation on the space of functions* $A \to B$*.*

Definition 4.3.3 (Homotopy equivalence)**.** *A function* $f : A \to B$ *is a homotopy equivalence if there exist two functions* $g, h : B \to A$*, such that* $f \circ g \sim \mathsf{id}_B$ *and* $h \circ f \sim \mathsf{id}_A$*, where* $\mathsf{id}_\star(x) = x$ *denotes the identity function.*

Definition 4.3.4 (Types equivalence)**.** *Types* A, B *are said to be homotopically equivalent (or just equivalent), written* $A \simeq B$*, if there exists a function* $f : A \to B$ *such that* f *is a homotopy equivalence.*

Lemma 4.3.5 (Identity is equivalence)**.** *For any type* A*, function* id_A *is an equivalence.*

Lemma 4.3.6 ($\mathsf{idtoequiv}$)**.** *For any two types* A, B*, there is a function*

$$\mathsf{idtoequiv} : (A =_{\mathcal{U}} B) \to A \simeq B$$

Proof. For a path $p : A =_{\mathcal{U}} B$, we have a function $\mathsf{T}p : A \to B$. This function is an equivalence: its inverse is $\mathsf{transport}^{p^{-1}}_{\mathcal{U}}$, by Lemma 4.2.5. Thus, the composition is identity, so by Lemma 4.3.5 it is an equivalence. □

Note that we can use $\mathsf{transport}^{=}_{_}$ to turn paths into equivalences.

Definition 4.3.7. *For a given type A, its elements $a, b : A$ and a type family $\Pi_{(x:A)} P(x)$, we define the function*

$$\mathsf{transport\text{-}equiv}(P) : \Pi_{(x,y:A)} \Pi_{(x=_A y)} P(x) \simeq P(y)$$
$$\mathsf{transport\text{-}equiv}(P)(p) \triangleq \mathsf{transport}^P_P(\mathsf{id}_A)$$

Now, we have all the necessary technology to finally define the univalence axiom.

Axiom (Univalence Axiom)**.** *For any types A, B, the function* $\mathsf{idtoequiv} : (A \simeq B) \to (A =_{\mathcal{U}} B)$ *is an equivalence. In other words, we assert the existence of a function*

$$\mathsf{ua} : (A \simeq B) \to (A =_{\mathcal{U}} B)$$

together with homotopies[1]

$$\mathsf{ua}_\eta : \mathsf{ua} \circ \mathsf{idtoequiv} \sim \mathsf{id}_{A=_{\mathcal{U}} B} \qquad \mathsf{ua}_\beta : \mathsf{idtoequiv} \circ \mathsf{ua} \sim \mathsf{id}_{A \simeq B}$$

Of course, the univalence axiom is not compatible with UIP. This is, however, a good thing. We can now prove theorems with ease that were hard to do otherwise, since for any two types that we can identify with equivalence, we can use transport to move between conveniently.

From the univalence axiom, we can extract function extensionality, which allows us to identify functions that are element-wise equal.

Lemma 4.3.8. *For any two functions $f, g : \Pi_{(a:A)} B(a)$, we have the equivalence*

$$\lambda{=} : f =_{\Pi_{(a:A)} B(a)} g \simeq \prod_{(a:A)} f(a) =_{B(a)} g(a)$$

4.4. Homotopy levels

Because UIP is not true in HoTT, identity types do not collapse on the higher levels. However, for simple types - such as the $\mathbf{1}$, for example - the structure does trivialise. Thus, it is useful to make a classification of all types, according to the level at which the identity type structure collapses to a point.

Definition 4.4.1 (Homotopy levels)**.** *We characterise homotopy levels of types in the following way:*

- *We say that a type A is contractible, or equivalently having homotopy level -1, if there is $a : A$ such that for all other $b : A$ we have $a =_A b$.*

$$\mathsf{isContr}\, A \triangleq \mathsf{hasLevel}_{-1} \triangleq \sum_{(a:A)} \prod_{(b:A)} a =_A b$$

- *We say that a type A is a proposition, or equivalently having homotopy level 0, if its equality types are contractible (so it has at most one element).*

$$\mathsf{isProp}\, A \triangleq \mathsf{hasLevel}_0 \triangleq \prod_{(a,b:A)} \mathsf{isContr}(a =_A b)$$

[1] Technically, ua_η is derivable, as shown by Hofmann and Streicher [1996]

- *We say that a type A is a set, or equivalently having homotopy level 1, if its equality types are propositions.*

$$\mathsf{isSet}\, A \triangleq \mathsf{hasLevel}_1 \triangleq \prod_{(a,b:A)} \mathsf{isProp}(a =_A b)$$

- *We say that a type A is a groupoid, or equivalently having homotopy level 2, if its equality types are sets.*

$$\mathsf{isGrp}\, A \triangleq \mathsf{hasLevel}_2 \triangleq \prod_{(a,b:A)} \mathsf{isSet}(a =_A b)$$

- *We say that a type A has homotopy level $n+1$ if its equality types have homotopy level n.*

$$\mathsf{hasLevel}_{n+1}\, A \triangleq \prod_{(a,b:A)} \mathsf{hasLevel}_n(a =_A b)$$

In the proof, we will be concentrating on groupoids, and the most interesting parts will be their loopspaces.

Definition 4.4.2 (Pointed type)**.** *We say that a pointed type is pair (A, a), such that $A : \mathcal{U}$ is a type and and $a : A$ is its basepoint.*

Definition 4.4.3 (Loopspace)**.** *Given a pointed type (A, a), its loopspace is defined as a pointed type*

$$\Omega(A, a) \triangleq ((a =_A a), \mathsf{refl}_a))$$

4.5. Inductive types

Giving the rules of type formation, introduction, elimination and computation can start to be a bit tedious - especially when we consider that there is still a large number of important types we need in the later parts of this thesis, but did not yet cover, such as products $A \times B$, natural numbers $\mathbb{N}$, lists $\mathsf{List}(A)$, orderings $n < m$, and more. The problem becomes even more obvious when one considers that, what is written in a programming language or a proof assistant, is usually something along the lines of:

Definition 4.5.1 (Natural numbers type)**.** *The natural numbers type $\mathbb{N}$ is inductively defined by the following constructors*

$$0 : \mathbb{N}$$
$$\mathsf{S} : \mathbb{N} \to \mathbb{N}$$

In other words, in the definition, we only specify the type formation rules, while all the rest of the structure is automatically inferred. To do that here, we will use so-called *W-types*.

Type T will be specified using a type $A : \mathcal{U}$ and a type family $B : A \to \mathcal{U}$ - we use notation $W_{(a:A)}B(a)$ for its definition. Intuitively, A will be a type of labels for constructors of T, while B will be a type counting the recursive arity of constructors of T[2]. Labels for

[2]Note that *arity* in this context means only the number of recursive references. For example, if we were to construct a type boxing booleans Box, its type constructor $\square : \mathbf{2} \to \mathsf{Box}$ would instead be written as two nullary constructors $\square_{\mathsf{inl}(\star)} : \mathsf{Box}$ and $\square_{\mathsf{inr}(\star)} : \mathsf{Box}$ - i.e. we would have $A \triangleq \mathbf{2}$ and $B(-) \triangleq \mathbf{0}$

constructors, as well as arities, will come from the types $\mathbf{0}, \mathbf{1}, \mathbf{2}, \ldots$ that we introduced in Definition 4.1.1.

Thus, we have the following rules (in all cases, we write $T := W_{(x:A)}B(x)$ for readability):

$$\frac{\Gamma \vdash A : \mathcal{U} \qquad \Gamma, (x : A) \vdash B}{\Gamma \vdash T : \mathcal{U}}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash f : B(a) \to T}{\Gamma \vdash \mathsf{sup}_W(f, a) : T}$$

$$\frac{\Gamma, z : T \vdash C : \mathcal{U} \qquad \Gamma, z : T \vdash d : \Pi_{(a:A)}\Pi_{(u:B(a)\to T)}(\Pi_{(y:B(a))}C(u(y)) \to C(\mathsf{sup}(a, u)))}{\Gamma \vdash \mathsf{ind}_W(C, z, d) : \Pi_{(z:T)}C(z)}$$

$$\frac{\Gamma, z : T \vdash C : \mathcal{U} \qquad \Gamma, z : T \vdash d : \Pi_{(a:A)}\Pi_{(u:B(a)\to T)}(\Pi_{(y:B(a))}C(u(y)) \to C(\mathsf{sup}(a, u)))}{\Gamma \vdash \mathsf{ind}_W(C, \mathsf{sup}(a, u)/z, d) \equiv d(a, u, f \circ u) : C[\mathsf{sup}(a, u)/z]}$$

In case of the natural numbers type discussed above, we have

$$\mathbb{N} \triangleq W_{a:\mathbf{2}}\, \mathsf{rec}_{b:\mathbf{2}}(\mathcal{U}, \mathbf{0}, \mathbf{1}, b)$$

where $\mathsf{rec}_{b:\mathbf{2}}$ is the recursion principle for the type $\mathbf{2}$, which chooses $\mathbf{0}$ if the argument $b := 0_{\mathbf{1}}$, and $\mathbf{1}$ if the argument $b := 1_{\mathbf{1}}$. Thus, we can define

$$\begin{aligned} 0^{\mathbb{N}} &\triangleq \mathsf{sup}(0_{\mathbf{2}}, \lambda x.\, \mathsf{ind}_{\mathbf{0}}(x, \mathbb{N})) \\ 1^{\mathbb{N}} &\triangleq \mathsf{sup}(1_{\mathbf{2}}, \lambda x.0^{\mathbb{N}}) \\ \mathsf{S}^{\mathbb{N}} &\triangleq \lambda n.\, \mathsf{sup}(1_{\mathbf{2}}, \lambda x.n) \end{aligned}$$

Thus, for the types that we will use in the next sections, we define them in the human-readable form here, and assume the appropriate translation to W-types. Natural numbers have already been defined in Definition 4.5.1. We will also need:

- Lists: for a type A, we define a type of lists $\mathsf{List}(A)$ with constructors:

  $\mathsf{nil} : \mathsf{List}(A)$

  $_::_ : A \to \mathsf{List}(A) \to \mathsf{List}(A)$

- Products: for types A and B, we define a product type $A \times B$ with one constructor:

  $\langle _, _ \rangle : A \to B \to A \times B$

- Strict ordering: for natural numbers $n, m : \mathbb{N}$, we define an inequality type $n < m$ with constructors:

  $(n : \mathbb{N}) \to 0 < \mathsf{S}\, n$

  $(n : \mathbb{N}) \to (m : \mathbb{N}) \to (n < m) \to \mathsf{S}\, n < \mathsf{S}\, m$

- As well as some more specialised types we introduce in later chapters, such as in Definition 7.5.2, Definition 7.5.4 and Definition 7.5.3

- And a plethora of different types we use in the proof assistant formalisation, which are omitted entirely in the text of this thesis.

4.6. Higher inductive types

Up to this point, we have only covered types where the constructors defined *points* in the type. The main strength of Homotopy Type Theory is, however, being able to operate on types with non-trivial higher structure. A Higher Inductive Type (HIT) is a type where constructors specify not only elements of the base type, but also paths (and higher paths). The prototypical example is a circle.

Definition 4.6.1 (Circle)**.** *A circle S^1 is a HIT with one point constructor* $\mathsf{base} : S^1$ *and one path constructor* $\mathsf{loop} : \mathsf{base} =_{S^1} \mathsf{base}$.

Although not known a priori, it can be proven that $\mathsf{loop} \neq \mathsf{refl}_{\mathsf{base}}$. What's more, it can be shown that the structure of the type $\mathsf{base} =_{S^1} \mathsf{base}$ is actually isomorphic to $\mathbb{Z}$ - in other words, computing the fundamental group of a circle to be integers.

Two HITs that we will use in the proof are truncation and set quotient.

Definition 4.6.2 (Truncation)**.** *For a type $A : \mathcal{U}$, its (-1)-truncation $||A||_{-1}$ (written $||A||$) is a HIT constructed according to the following rules.*

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash |a| : ||A||} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : A}{\Gamma \vdash \mathsf{trunc}_{(a,b)} : |a| =_{||A||} |b|}$$

$$\frac{\Gamma \vdash \mathsf{isProp}\, B \qquad \Gamma \vdash g : A \to B \qquad \Gamma, x : B, y : B \vdash \mathsf{trunc}_{(x,y)} : x =_B y}{\Gamma \vdash \hat{g} : ||A|| \to B}$$

$$\frac{\Gamma \vdash \mathsf{isProp}\, B \qquad \Gamma \vdash g : A \to B \qquad \Gamma, x : B, y : B \vdash t^*_{(x,y)} : x =_B y}{\Gamma \vdash \hat{g}(|a|) \equiv g(a)}$$

$$\frac{\Gamma \vdash \mathsf{isProp}\, B \qquad \Gamma \vdash a_1 : A \qquad \Gamma \vdash a_2 : A \qquad \Gamma \vdash g : A \to B \qquad \Gamma, x : B, y : B \vdash t^* : x =_B y}{\Gamma \vdash \mathsf{trunc\text{-}ap} : \mathsf{ap}_{\hat{g}}(\mathsf{trunc}_{(a,b)}) =_B t^*[g(a)/x, g(b)/y]}$$

Truncation is collapsing all elements of a given type to a single point - by introducing the path constructor that equates them. Because, as we have already shown, all functions are functors, the computation rule guarantees that functions out of truncation preserve equality.

Using the truncation we can define, for a given function, its fiber type, and the type of its image, which will be useful later.

Definition 4.6.3 (Function image)**.** *For types $A, B : \mathcal{U}$ and a function $f : A \to B$, we define its type family of fibers* $\mathsf{fib}_f : \Pi_{(b:B)} \to \mathcal{U}$*, and its image* $\mathsf{im}(f)$ *to be, respectively*

$$\mathsf{fib}_f(b) = \sum_{(x:A)} f(a) = b \qquad \textit{and} \qquad \mathsf{im}(f) = \sum_{(b:B)} ||\,\mathsf{fib}_f(b)||$$

The other higher inductive we need is the set quotient, which we will later use to define the group presentation in Chapter 7.

Definition 4.6.4 (Set quotient)**.** *For a type $A : \mathcal{U}$ which is a set, and a binary prop-valued relation $R : A \to A \to \mathcal{U}$ defined on A, set quotient A/R is a type formed according to the following rules*[3]*.*

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash q(a) : A/R} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : A \qquad \Gamma \vdash r : R(a,b)}{\Gamma \vdash \overline{r} : q(a) =_{A/R} q(b)}$$

[3]To simplify, we only give rules for non-dependent case

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : A \qquad \Gamma \vdash p : a =_{A/R} b \qquad \Gamma \vdash q : a =_{A/R} b}{\Gamma \vdash \mathsf{trunc}_{(p,q)} : p =_{a=_{A/R}b} q}$$

$$\frac{\Gamma \vdash f : A \to B \qquad \Gamma \vdash p : \Pi_{a_1 =_{A/R} a_2} f(a_1) =_B f(a_2)}{\Gamma \vdash \mathsf{quot}(f)(p) : A/R \to B}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash f : A/R \to B \qquad \Gamma \vdash p : f(a) =_B b}{\Gamma \vdash f(q(a)) \equiv b}$$

Note that we don't require R to be an equivalence relation, since this is given by the groupoid properties of HIT anyway - but it being an equivalence relation guarantees better properties of A/R, such as the characterization of the path space $R(a,b) \simeq (q(a) =_{A/R} q(b))$.

4.7. Finite types

In Definition 4.1.1, we have constructed the meta-theoretic finite types, to enable the proper treatment of W-types. However, these have had significant limitations, such as inability to talk about "all finite types", or doing any operations on them inside the theory.

We now introduce the proper finite type definition. For that, we first need a definition of a canonical n-element type. For formalisation convenience, we used two flavours of such types interchangeably. First definition follows the same spirit as Definition 4.1.1, but now as a type family over $\mathbb{N}$.

Definition 4.7.1 (Canonical finite types, variant I)**.** *We define the family of finite types* $\mathsf{Fin}_n : \mathbb{N} \to \mathcal{U}$ *inductively, by*

$$\mathsf{Fin}_0 \triangleq \bot$$
$$\mathsf{Fin}_{\mathsf{S}\,n} \triangleq \top + \mathsf{Fin}_n$$

Definition 4.7.2 (Canonical finite types, variant II)**.** *We define the family of finite types* $\mathsf{Fin}_n : \mathbb{N} \to \mathcal{U}$ *inductively, by*

$$\mathsf{Fin}_n = \sum_{k:\mathbb{N}} k < n$$

Lemma 4.7.3. *Those two definitions of canonical finite types are equivalent.*

Having the canonical n-element type, we can now say that a finite type is any type that is (merely) equivalent to some Fin_n.

Definition 4.7.4 (Finite types)**.**

$$\mathsf{isFin}(X) \triangleq \sum_{n:\mathbb{N}} \|X =_{\mathcal{U}} \mathsf{Fin}_n\|_{-1}$$

Lemma 4.7.5. *For any type* X*,* $\mathsf{isFin}(X)$ *is a proposition.*

4.8. Univalent universes

Definition 4.8.1 (Univalent universe)**.** *A universe is defined by:*

- *a type* $U : \mathcal{U}$*,*
- *type family* $\mathsf{El} : U \to \mathcal{U}$*.*

We call the pair (U, El) *a univalent universe (or, equivalently,* El *a univalent type family), if* $\mathsf{transport\text{-}equiv}(\mathsf{El})$ *is an equivalence.*

Intuitively, the type U should be understood as the *codes* for types in the universe, while the family El as providing their interpretations. An appropriately chosen universe might serve as a model for a theory inside HoTT.

A theorem by Kapulkin et al. [2012] states that being a univalent universe is a sufficient condition for the equality type to be reflected in the universe - i.e., uniquely determined by the structure of the equivalences. This is true for the big universe $\mathcal{U}$, since the canonical type family $U \mapsto U$ is univalent.

Proposition 4.8.2 (Univalent Subuniverse)**.** *A universe predicate is a type family* $P : \mathcal{U} \to \mathcal{U}$ *whose fibers are propositions, that is,* $P(X)$ *is a proposition for every* X*. Given such a predicate* P*, the fibration* $\pi_1 : \sum_{X:\mathcal{U}} P(X) \to \mathcal{U}$ *is univalent and generates a univalent subuniverse* $\mathcal{U}_P \triangleq (\sum_{X:\mathcal{U}} P(X), \pi_1)$

Starting from the universe $\mathcal{U}$, our goal is to cut out a smaller universe of finite types, in which we will be able to develop the proof in Chapter 6. To do that, we use the following definition.

Definition 4.8.3 (Univalent Subuniverse of Finite Types)**.** *The univalent subuniverse of finite types is defined as*

$$\mathcal{U}_{\mathsf{Fin}} \triangleq \sum_{X:\mathcal{U}} \mathsf{isFin}(X).$$

For a type Fin_n*, we write* $F_n \triangleq (\mathsf{Fin}_n, (n, |\,\mathsf{refl}\,|))$ *for its inclusion in* $\mathcal{U}_{\mathsf{Fin}}$*.*

Note that $\mathcal{U}_{\mathsf{Fin}}$ being univalent follows directly from Lemma 4.7.5 and Proposition 4.8.2.

During the course of the proof, we will mostly be working within one specified cardinality. Thus, it will be useful to have a definition analogous to $\mathcal{U}_{\mathsf{Fin}}$, but for n-element types.

Definition 4.8.4 ($\mathcal{U}_{\mathsf{Fin}_n}$)**.** *For any* $n : \mathbb{N}$*, we define*

$$\mathcal{U}_{\mathsf{Fin}_n} \triangleq \sum_{X:\mathcal{U}} \|X =_{\mathcal{U}} \mathsf{Fin}_n\|_{-1}.$$

to be the univalent subuniverse of n*-element sets.*

Lemma 4.8.5. $\mathcal{U}_{\mathsf{Fin}}$ *can be equivalently seen as the collection of all types of finite cardinality, that is*

$$\mathcal{U}_{\mathsf{Fin}} \simeq \sum_{n:\mathbb{N}} \mathcal{U}_{\mathsf{Fin}_n}$$

Because $\mathcal{U}_{\mathsf{Fin}_n}$ is a univalent subuniverse, we can characterise its path space. It only has one point, since we explicitly require all of them to be equal. Because of the truncation, the type is (just) a groupoid. The only non-trivial element of the path space is thus the structure of 1-loops, which are equivalent to bijections on an n-element set.

Theorem 4.8.6. $\mathcal{U}_{\mathsf{Fin}_n}$ *is a pointed (1-)groupoid for every* $n : \mathbb{N}$*, and*

$$\Omega(\mathcal{U}_{\mathsf{Fin}_n}, F_n) \simeq \mathsf{Aut}(\mathsf{Fin}_n)$$

$\mathcal{U}_{\mathsf{Fin}}$ *is a weak (1-)groupoid with connected components for every* $n : \mathbb{N}$*.*

Theorem 4.8.7. $\mathcal{U}_{\mathsf{Fin}}$ *is a weak symmetric rig groupoid, the additive monoidal structure given by* $(F_0, +)$ *and multiplicative monoidal structure given by* $(F_1, \times)$*.*

Chapter 5

Reversible programming language

Π is a programming language for writing bijections between finite types and equivalences between these bijections. The syntax consists of three parts: types, 1-combinators (expressing programs), and 2-combinators (expressing programs' equivalences).

5.1. Types

Types in *Π* are built from:

- constants $\mathbf{0}, \mathbf{1}$ and
- binary type constructors + and ×

A concrete example of a *Π* type might be

$$\mathbf{2} := \mathbf{1} + \mathbf{1} \qquad \text{or} \qquad (\mathbf{1} + (\mathbf{1} + \mathbf{1})) + (((\mathbf{0} + \mathbf{0}) \times \mathbf{1}) + \mathbf{1})$$

We will use A, B, C as meta-variables for the *Π* types.

5.2. Programs

1-combinators (or just combinators) generate equivalences between *Π* types. The binary type constructor for 1-combinators between types A and B is written as $A \leftrightarrow_1 B$. The intention is to have enough constants and constructors to be able to write any permutation between *Π* types. Available constants are shown below.

$$\begin{array}{rrcll}
\mathsf{id}\leftrightarrow_1 : & A & \leftrightarrow_1 & A & : \mathsf{id}\leftrightarrow_1 \\
\\
\mathsf{unite}_+\mathsf{l} : & \mathbf{0} + A & \leftrightarrow_1 & A & : \mathsf{uniti}_+\mathsf{l} \\
\mathsf{swap}_+ : & A + B & \leftrightarrow_1 & B + A & : \mathsf{swap}_+ \\
\mathsf{assocl}_+ : & A + (B + C) & \leftrightarrow_1 & (A + B) + C & : \mathsf{assocr}_+ \\
\\
\mathsf{unite}\star\mathsf{l} : & \mathbf{1} \times A & \leftrightarrow_1 & A & : \mathsf{uniti}\star\mathsf{l} \\
\mathsf{swap}\star : & A \times B & \leftrightarrow_1 & B \times A & : \mathsf{swap}\star \\
\mathsf{assocl}\star : & A \times (B \times C) & \leftrightarrow_1 & (A \times B) \times C & : \mathsf{assocr}\star \\
\\
\mathsf{absorbr} : & \mathbf{0} \times A & \leftrightarrow_1 & \mathbf{0} & : \mathsf{factorzl} \\
\mathsf{dist} : & (A + B) \times C & \leftrightarrow_1 & (A \times C) + (B \times C) & : \mathsf{factor}
\end{array}$$

To write programs in Π, in addition to constants for 1-combinators, we also have three kinds of composition operators.

- Sequential composition

$$\frac{\vdash c_1 : A \leftrightarrow_1 B \quad \vdash c_2 : B \leftrightarrow_1 C}{\vdash c_1 \circledcirc c_2 : A \leftrightarrow_1 C}$$

- Parallel + composition

$$\frac{\vdash c_1 : A \leftrightarrow_1 B \quad \vdash c_2 : C \leftrightarrow_1 D}{\vdash c_1 \oplus c_2 : A + C \leftrightarrow_1 B + D}$$

- Parallel $\times$ composition

$$\frac{\vdash c_1 : A \leftrightarrow_1 B \quad \vdash c_2 : C \leftrightarrow_1 D}{\vdash c_1 \otimes c_2 : A \times C \leftrightarrow_1 B \times D}$$

One example of a Π combinator might be a *controlled-not* gate known from reversible Boolean circuits [Toffoli, 1980]:

$$\begin{aligned}
&\mathsf{controlled\text{-}not} \;:\; (\mathbf{2} \times \mathbf{2}) \leftrightarrow (\mathbf{2} \times \mathbf{2}) \\
&\mathsf{controlled\text{-}not} \;=\; \mathsf{dist} \circledcirc (\mathsf{id}\!\leftrightarrow_1 \oplus (\mathsf{id}\!\leftrightarrow_1 \otimes \mathsf{swap}\star)) \circledcirc \mathsf{factorzl}
\end{aligned}$$

or, written in equational style

$$\begin{aligned}
&(\mathbf{1}+\mathbf{1}) \times \mathbf{2} && \leftrightarrow_1 \langle \mathsf{dist} \rangle \\
&(\mathbf{1} \times \mathbf{2}) + (\mathbf{1} \times \mathbf{2}) && \leftrightarrow_1 \langle \mathsf{id}\!\leftrightarrow_1 \oplus (\mathsf{id}\!\leftrightarrow_1 \otimes \mathsf{swap}\star) \rangle \\
&(\mathbf{1} \times \mathbf{2}) + (\mathbf{1} \times \mathbf{2}) && \leftrightarrow_1 \langle \mathsf{factorzl} \rangle \\
&(\mathbf{1}+\mathbf{1}) \times \mathbf{2}
\end{aligned}$$

5.3. Program equivalences

Finally, 2-combinators generate equivalences between Π 1-combinators. For readability, we present only the ones dealing with associativity here - the rest can be found in the Appendix A. For arbitrary Π types A, B, C, D, E, F and 1-combinators

$$c_1 : A \leftrightarrow_1 B \qquad c_2 : B \leftrightarrow_1 C \qquad c_3 : C \leftrightarrow_1 D \qquad c_4 : E \leftrightarrow_1 F$$

we have the following 2-combinators

$$\begin{aligned}
&\mathsf{assoc}\circledcirc\mathsf{l} \;:\; (c_1 \circledcirc (c_2 \circledcirc c_3)) \leftrightarrow_2 ((c_1 \circledcirc c_2) \circledcirc c_3) \\
&\mathsf{assoc}\circledcirc\mathsf{r} \;:\; ((c_1 \circledcirc c_2) \circledcirc c_3) \leftrightarrow_2 (c_1 \circledcirc (c_2 \circledcirc c_3)) \\
&\mathsf{assocl}_+\mathsf{l} \;:\; ((c_1 \oplus (c_2 \oplus c_4)) \circledcirc \mathsf{assocl}_+) \leftrightarrow_2 (\mathsf{assocl}_+ \circledcirc ((c_1 \oplus c_2) \oplus c_4)) \\
&\mathsf{assocl}_+\mathsf{r} \;:\; (\mathsf{assocl}_+ \circledcirc ((c_1 \oplus c_2) \oplus c_4)) \leftrightarrow_2 ((c_1 \oplus (c_2 \oplus c_4)) \circledcirc \mathsf{assocl}_+) \\
&\mathsf{assocr}_+\mathsf{r} \;:\; (((c_1 \oplus c_2) \oplus c_4) \circledcirc \mathsf{assocr}_+) \leftrightarrow_2 (\mathsf{assocr}_+ \circledcirc (c_1 \oplus (c_2 \oplus c_4))) \\
&\mathsf{assocr}_+\mathsf{l} \;:\; (\mathsf{assocr}_+ \circledcirc (c_1 \oplus (c_2 \oplus c_4))) \leftrightarrow_2 (((c_1 \oplus c_2) \oplus c_4) \circledcirc \mathsf{assocr}_+)
\end{aligned}$$

5.4. Equivalences of equivalences

One might ask whether there is still some next level, above 2. This is, however, not the case. Since we treat types as 2-groupoids (in the homotopical sense), the equivalences between them are sets, so equivalences between them form a prop, which trivializes next levels: for any types A, B, 1-combinators $c_1, c_2 : A \leftrightarrow_1 B$ and 2-combinators $\alpha_1, \alpha_2 : c_1 \leftrightarrow_2 c_2$, we have

$$\star : \alpha_1 \leftrightarrow_3 \alpha_2$$

Phrasing it differently, even though there are two bijections from **2** to itself, providing two distinct proofs ($\mathsf{id}\leftrightarrow_1$ and $\mathsf{swap}\star$) of the equality between the type **2** with itself, there is only one way to show that a bijection is equal to another bijection (which is, for example, to compare them extensionally).

Why do we need the second layer of combinators then, if they are all equal? One answer is that we still want to have some concrete, explicit witness for the equality, which we can then evaluate to the path in the semantics. Another is that there is still more (syntactic) structure on the 2-combinators, which allows us to use this structure in a potential search for equivalence (described in more detail in Chapter 8).

Full specification of Π syntax, written in Agda, is included in Appendix A.

5.5. Intended semantics

One simple way to build semantics for Π is to do it in the **Core(Set)**, using a $[\![-]\!]$ function. A type A would be mapped to a finite set $[\![A]\!]$, and a combinator $c : A \leftrightarrow_1 B$ would be mapped to a bijection between sets $[\![c]\!] : [\![A]\!] \to [\![B]\!]$.

However, this does not take into account the second level of combinators. To use those, the semantics has to be in a 2-groupoid. Moreover, preserving the structure of the language requires there to be additive and multiplicative structures in the category.

Thus, a sound semantics for Π was constructed in weak rig groupoids by [Carette and Sabry, 2016]. They have also conjectured it to be complete - i.e., that the *coherence conditions* for symmetric rig groupoids exactly match the level-2 combinators of Π. This is the theme of the next Chapter 6.

Chapter 6

Equivalence of $\mathbf{\Pi}_{\text{cat}}$ and $\mathcal{U}_{\mathsf{Fin}}$

This chapter outlines the main result - i.e., the equivalence of the weak symmetric rig groupoid (formalised as the syntactic category $\mathbf{\Pi}_{\text{cat}}$ in Section 6.4) and the groupoid of finite sets and bijections $\mathcal{U}_{\mathsf{Fin}}$ (constructed in Chapter 4). We state it in the Theorem 6.5.5 at the end of the chapter, and go through the key lemmas (without proofs) in Section 6.5. The proof is fully described in [Choudhury et al., 2022] and in [Choudhury and Karwowski, 2021], as well as in the code formalisation attached to this thesis.

Π is, as we have seen in Chapter 5, a relatively complex language - especially if we compare it to λ-calculus developed in Chapter 3. Thus, building the semantics is going to be much more involved than the $[\![-]\!]$ mapping described in Section 3.6. It is, in fact, going to be so complex, that the most accessible route is to go through two smaller, more accessible languages, simplifying away the coherence conditions in the process.

6.1. Π^+ language

The first intermediate language we are going to look at is Π^+, which is the additive fragment of Π. Thus, the types are built from constants $0, 1$ and the binary constructor $+$.

The 1-combinators for Π^+ are the 1-combinators for Π, restricted to the additive fragment. The full list of 2-combinators, denoted by $c_1 \Leftrightarrow^+ c_2$ for compatible 1-combinators c_1, c_2 is rel-

$$
\begin{array}{rrcll}
\mathsf{id}{\leftrightarrow_1}: & A & \leftrightarrow^+ & A & : \mathsf{id}{\leftrightarrow_1} \\
\mathsf{unite_+l}: & \mathbf{0}+A & \leftrightarrow^+ & A & : \mathsf{uniti_+l} \\
\mathsf{swap_+}: & A+B & \leftrightarrow^+ & B+A & : \mathsf{swap_+} \\
\mathsf{assocl_+}: & A+(B+C) & \leftrightarrow^+ & (A+B)+C & : \mathsf{assocr_+}
\end{array}
$$

$$
\frac{\vdash c_1 : A \leftrightarrow^+ B \quad \vdash c_2 : B \leftrightarrow^+ C}{\vdash c_1 \circledcirc c_2 : A \leftrightarrow^+ C}
\qquad
\frac{\vdash c_1 : A \leftrightarrow^+ B \quad \vdash c_2 : C \leftrightarrow^+ D}{\vdash c_1 \oplus c_2 : A + C \leftrightarrow^+ B + D}
$$

egated to the Appendix B. Note that even though Π^+ does not have a built-in multiplication, it is possible to build one as repeated addition - thus making it possible to compile programs written in Π to Π^+.

6.2. $\Pi^\wedge$ language

The second intermediate language we are going to look at is $\Pi^\wedge$. This time, the types are simplified down to the information about their cardinality - in other words, types of $\Pi^\wedge$ are just natural numbers. We can say that this "strictifies" the associativity of + constructor in Π^+.

Since there is so little structure present in the types, 1-combinators for Π^+ are just an identity and head-swap (notice how the parallel composition has changed from a binary to a unary operator).

$$\begin{array}{rrcll} \mathsf{id}{\leftrightarrow_1}: & n & \leftrightarrow^\wedge & n & : \mathsf{id}{\leftrightarrow_1} \\ \mathsf{swap}: & \mathsf{S}\,\mathsf{S}\,n & \leftrightarrow^\wedge & \mathsf{S}\,\mathsf{S}\,n & : \mathsf{swap} \end{array}$$

$$\frac{\vdash c_1 : n \leftrightarrow^\wedge m \quad \vdash c_2 : m \leftrightarrow^\wedge o}{\vdash c_1 \circledcirc c_2 : n \leftrightarrow^\wedge o} \qquad \frac{\vdash c : n \leftrightarrow^\wedge m}{\vdash \oplus(c) : \mathsf{S}\,n \leftrightarrow^\wedge \mathsf{S}\,m}$$

In $\Pi^\wedge$, 2-combinators are denoted by $c_1 \Leftrightarrow^\wedge c_2$ for compatible 1-combinators c_1, c_2. Full specification is given in Appendix C.

6.3. Symmetric group

Finally, we can see that the bare-bones language of $\Pi^\wedge$ is extremely close to being just a presentation of the symmetric group S_n. Formally, it is not the case, since, in a way, it describes a coproduct of all $\Sigma_n \mathcal{S}_n$, and, instead of having the generators explicitly listed, we have the combinator $\oplus(-)$ which can be used to write appropriate adjacent transpositions.

This will be, then, our last step in the simplification of Π - a proper, and convenient, presentation of $\mathcal{S}_n$. Chapter 7 describes this construction in more detail.

6.4. Syntactic categories

To properly formalise the proof in the categorical language, we first have to define syntactic categories for Π, Π^+ and $\Pi^\wedge$ languages. As the whole point of Π is to be able to explicitly talk about the higher structure, the formalism will be in bicategories.

Proposition 6.4.1. *We can form a bicategory* $\mathbf{\Pi}_{\mathrm{cat}}$ *with*

- *Π types (U) for objects,*
- *for $X, Y : U$, a collection of 1-morphisms $X \leftrightarrow Y$,*
- *for $p, q : X \leftrightarrow Y$, a collection of 2-morphisms $p \Leftrightarrow q$.*

which has a symmetric rig groupoid structure, with tensor products + an $\times$.

Proposition 6.4.2. $\mathbf{\Pi}_{\mathrm{cat}}$ *is a free weak symmetric rig groupoid on zero generators.*

We repeat the same construction for the Π^+ language.

Proposition 6.4.3. *We can form a bicategory* $\mathbf{\Pi}^+_{\mathrm{cat}}$ *with*

- *Π^+ types (U^+) for objects,*
- *for $X, Y : U^+$, a collection of 1-morphisms $X \leftrightarrow^+ Y$,*
- *for $p, q : X \leftrightarrow^+ Y$, a collection of 2-morphisms $p \Leftrightarrow^+ q$.*

As we have already noted, Π^+ does not have a built-in product, but it is possible to define it as repeated addition, using the $+$ monoidal structure:

Definition 6.4.4 ($\times : U^+ \to U^+ \to U^+$)**.**

$$\begin{aligned} \mathbf{0} \times Y &\triangleq \mathbf{0} \\ \mathbf{1} \times Y &\triangleq Y \\ (X_1 + X_2) \times Y &\triangleq X_1 \times Y + X_2 \times Y \end{aligned}$$

Lemma 6.4.5. *There are two symmetric monoidal structures on $\mathbf{\Pi}^+_{\text{cat}}$, given by $(\mathbf{0}, +)$ and $(\mathbf{1}, \times)$, with $\times$ distributing over $+$, giving it a symmetric rig groupoid structure.*

Proposition 6.4.6. $\mathbf{\Pi}^+_{\text{cat}}$ *is a free symmetric monoidal groupoid on one generator.*

Finally, we take the same approach to $\Pi^\wedge$.

Proposition 6.4.7. *We can form a bicategory $\mathbf{\Pi}^\wedge_{\text{cat}}$ with*

- *$\Pi^\wedge$ types ($U^\wedge$) for objects,*
- *for $X, Y : U^\wedge$, a collection of 1-morphisms $X \leftrightarrow^\wedge Y$,*
- *for $p, q : X \leftrightarrow^\wedge Y$, a collection of 2-morphisms $p \Leftrightarrow^\wedge q$.*

$\Pi^\wedge$ has, as objects, simply natural numbers, which makes it easy to define the required monoidal structure.

Lemma 6.4.8. $\mathbf{\Pi}^\wedge_{\text{cat}}$ *has a monoidal structure, with the unit given by 0 and the tensor product given by natural number addition.*[1]

6.5. Proof outline

The full proof of the main Theorem 6.5.5 is the composition of three main lemmas, which carry the structure of $\mathbf{\Pi}_{\text{cat}}$ along the progressively simplified syntactic categories of $\mathbf{\Pi}^+_{\text{cat}}$ and $\mathbf{\Pi}^\wedge_{\text{cat}}$. We state them, without proofs, below.

Lemma 6.5.1. *There is a rig equivalence between $\mathbf{\Pi}_{\text{cat}}$ and $\mathbf{\Pi}^+_{\text{cat}}$.*

Lemma 6.5.2. *There is a monoidal equivalence between $\mathbf{\Pi}^+_{\text{cat}}$ and $\mathbf{\Pi}^\wedge_{\text{cat}}$.*

$\mathcal{U}_{\mathsf{Fin}}$ as a 1-groupoid, is a (locally strict) weak $(2,0)$-category, and we use this to state our final equivalence.

Lemma 6.5.3. *There is a symmetric monoidal equivalence between $\mathbf{\Pi}^\wedge_{\text{cat}}$ and $\mathcal{U}_{\mathsf{Fin}}$.*

[1] We lift the full symmetric monoidal structure form $\mathbf{\Pi}^+_{\text{cat}}$ to $\mathbf{\Pi}^\wedge_{\text{cat}}$ using the monoidal equivalence.

Proof. By Theorem 4.8.6, paths in $\mathcal{U}_{\mathsf{Fin}}$ are equivalent to families of loops on Fin_n for every $n : \mathbb{N}$, that is, automorphisms of finite sets of size n, with the loopspace encoding the automorphism group. Thus, objects in $\mathcal{U}_{\mathsf{Fin}}$ are mapped to their cardinalities in $\Pi^\wedge$, 1-morphisms are decoded to words in $\mathcal{S}_n$ to generate a sequence of adjacent transpositions, producing a 1-combinator in $\Pi^\wedge$. Finally, 2-paths are quoted back to 2-combinators in $\Pi^\wedge$. □

Thus, the key lemma that is needed in the proof above is the following statement (depending, of course, on the precise definition of $\mathcal{S}_n$, which we are yet to give, in Definition 7.5.8).

Lemma 6.5.4. *For all $n : \mathbb{N}$, we have $\mathcal{S}_n \simeq \mathsf{Aut}(\mathsf{Fin}_n)$*

Although quite obvious from the classical perspective, it is quite involved when we work in a formal setting in HoTT. Chapter 7 describes this part of the proof in depth. It is, by size, also the largest part of the formalization.

The intuition is that, on the side of Π (and, correspondingly, $\mathbf{\Pi}_{\mathrm{cat}}$), we have an infinite number of programs between $\mathbf{1} \leftrightarrow \mathbf{1}$, such as, for example:

$$\begin{aligned} \mathsf{id}{\leftrightarrow_1} &: \mathbf{1} \leftrightarrow \mathbf{1} \\ \mathsf{id}{\leftrightarrow_1} \circledcirc \mathsf{id}{\leftrightarrow_1} &: \mathbf{1} \leftrightarrow \mathbf{1} \\ \mathsf{id}{\leftrightarrow_1} \circledcirc \mathsf{id}{\leftrightarrow_1} \circledcirc \mathsf{id}{\leftrightarrow_1} &: \mathbf{1} \leftrightarrow \mathbf{1} \\ &\dots \end{aligned}$$

while on the side of $\mathcal{U}_{\mathsf{Fin}}$, there is only one path (one bijection) on the one-element set F_1. Thus, somewhere along the way, this infinite structure has to be collapsed, i.e. evaluated to the denotational semantics. The part of the proof where this happens is precisely Lemma 6.5.4. All of the earlier parts simplify the combinators and prepare the ground for this, but still operate on the *syntactic* programs, and not their semantic realizations.

By composing Lemma 6.5.1, Lemma 6.5.2 and Lemma 6.5.3 above, we get our desired result.

Theorem 6.5.5. *Free weak symmetric rig groupoid on zero generators is equivalent to the weak groupoid of sets and bijections.*

Chapter 7

The Group of Permutations

The goal of this section is to reconcile two different approaches to defining the symmetric group – as an automorphism group, and as a group syntactically presented using *generators* and *relations*. The generators of the group are similar to the primitive combinators in a (reversible) programming language – the group structure gives the composition and inverse operations, and the relations describe how these primitive combinators interact with each other.

First, we will define the required notions of free groups and group presentations, and state some of their most important properties. Then, we introduce our chosen *Coxeter presentation* for $\mathcal{S}_n$. To solve the word problem for $\mathcal{S}_n$, we will use a rewriting system, with a suitable, well behaved collection of reduction rules corresponding to the Coxeter presentation equations. Finally, we describe the normal forms in this rewriting system, using Lehmer codes, and prove the correspondence between them and the type $\mathsf{Aut}(\mathsf{Fin}_\mathsf{n})$ of automorphisms on a finite set.

7.1. Presenting the permutation group

One way of thinking about presentations of $\mathcal{S}_n$ is via sorting algorithms, which use different primitive operations. A sorting algorithm has to calculate a permutation of a list or a finite set, which satisfies the invariant of being a sorted sequence, which means, the primitive operations of a sorting algorithm are able to generate all the permutations on a given list. So, a chosen set of reversible operations in a sorting algorithm can be a good candidate for the generators of a permutation group. For example, we could generate the permutation group on Fin_n by using generators (primitive operations) that:

- swap the i-th element with the $(i+1)$-th element, that is, adjacent swaps, or
- swap the i-th element with the j-th element, for arbitrary i-s and j-s, or
- swap the i-th element with an element at a fixed position, or
- reverse a prefix Fin_k of Fin_n for $k \leq n$, or
- cyclically shift any subset of Fin_n.

Bubble sort uses the primitive operation of adjacent swaps, insertion sort and selection sort use the primitive operation of swapping the i-th element with the j-th element, cycle sort uses cyclical shifts of sub-sequences, pancake sort uses reversals of the prefixes of the list, et cetera. The choice of generators for our presentation is important for the following reasons.

- It affects the difficulty of solving the word problem in $\mathcal{S}_n$ and formalising the proof of its correctness.
- The choice of generators dictates which words become normal forms in this presentation of $\mathcal{S}_n$. These normal forms dictate the shape of the synthesised and normalised Boolean circuits (which is the application we have in mind – see Chapter 8)
- Finally, the generators have to closely match the Π combinators so that we can quote back a permutation to a Π program, for the proof of completeness.

We will show that it is possible to encode all Π combinators using adjacent transpositions.

7.2. Groups

From universal algebra, a group is simply a set with a 0-ary constant e (the neutral element), a binary operation $- \cdot -$ for group multiplication, and a unary inverse operation $-^{-1}$. The neutral element has to satisfy unit and inverse laws, and the multiplication has to be associative (see Definition 7.2.1).

A very simple example of a group is $\mathbb{Z}$, where the neutral element is 0, the inverse of k is $-k$, and the group multiplication is given by integer addition.

Definition 7.2.1 (Group)**.** *In type theory, a group G can be defined as a set S with the following pieces of data:*

1. *a unit or neutral element $e : S$*
2. *a multiplication function $m : S \times S \to S$ written as $(g_1, g_2) \mapsto g_1 \cdot g_2$, that satisfies*
   (a) *the unit laws, for all $g : S$, that $g \cdot e = g$ and $e \cdot g = g$*
   (b) *the associativity law, for all $g_1, g_2, g_3 : S$, that $g_1 \cdot (g_2 \cdot g_3) = (g_1 \cdot g_2) \cdot g_3$*
3. *an inversion function $i : S \to S$ written as $g \mapsto g^{-1}$, that satisfies*
   (a) *the inverse laws, for all $g : S$, that $g \cdot g^{-1} = e$ and $g^{-1} \cdot g = e$*

7.3. Free groups

Usually, there are are many equations, besides the group axioms, that hold for the elements of a group. For example, in the group $\mathsf{Aut}(\mathbf{2})$, or $\mathbb{Z}_2$, we have an equation $1 + 1 = 0$, which is not a consequence of the group axioms, but is specific to this particular group. A free group has the property that no other equations hold except the ones directly implied by the group axioms. For example, the additive group of integers $\mathbb{Z}$ is the free group on the singleton set.

Definition 7.3.1 (Free group)**.** *Given a set A, the free group $F(A)$ on it is given by a higher inductive type with the following point and path constructors. Notice the similarity with the definition of a group structure (Definition 7.2.1), but note that each operation here is a generator for $F(A)$.*

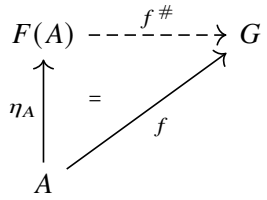
- *An inclusion function $\eta_A : A \to F(A)$*
- *A multiplication function $m : F(A) \times F(A) \to F(A)$*

- *An element* $e : F(A)$
- *An inverse function* $i : F(A) \to F(A)$

- *For every* $x, y, z : F(A)$*, a path* $\mathsf{assoc} : m(x, m(y, z)) = m(m(x, y), z)$
- *For every* $x : F(A)$*, paths* $\mathsf{unitr} : m(x, e) = x$ *and* $\mathsf{unitl} : m(e, x) = x$
- *For every* $x : F(A)$*, paths* $\mathsf{invr} : m(x, i(x)) = e$ *and* $\mathsf{invl} : m(i(x), x) = e$
- *A 0-truncation, for every* $x, y : F(A)$ *and* $p, q : x = y$*, a 2-path* $\mathsf{trunc} : p = q$

A group homomorphism between groups G and H is a function $f : G \to H$ between the underlying sets that preserves the group structure. Giving a group homomorphism out of the free group is equivalent to giving a function out of the generating set. This is the universal property of free groups, stemming from the free-forgetful adjunction between the category of groups and sets.

Proposition 7.3.2 (Universal Property of $F(A)$)**.** *Given a group* G *and a map* $f : A \to G$*, there is a unique group homomorphism* $f^{\#} : \mathrm{Hom}(F(A), G)$ *such that* $f^{\#} \circ \eta_A \sim f$*. Equivalently, composition with* η_A *gives an equivalence* $\mathrm{Hom}(F(A), G) \simeq A \to G$*.*

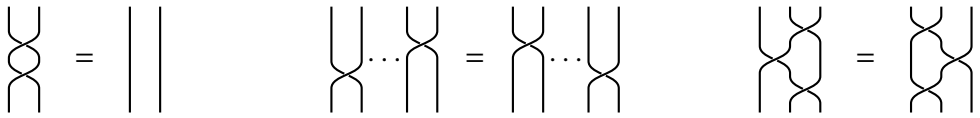


Following the universal-algebraic definition, in HoTT, we could use a naive higher inductive type to define the free group, which enforces the group axioms by adding path constructors (see Definition 7.3.1). Using the induction principle, we can easily verify the universal property. However, since this definition of $F(A)$ has lots of path constructors corresponding to each group axiom, characterising its path space is difficult.

Instead, we will think about elements of the free group as words over an alphabet of letters drawn from the generating set *and* the set of their formal inverses. If we take the disjoint union of A with itself, that is, $A + A$ as the group's underlying set, we can use $\mathsf{inl}/\mathsf{inr}$ to mark the elements – $\mathsf{inl}\, a$ means a and $\mathsf{inr}\, a$ means a^{-1}. Then, we can encode the free group using the free monoid, that is, lists of $A + A$. Additionally, we need to ensure that the inverse laws hold, so we have to coalesce adjacent occurrences of a and a^{-1}.

Definition 7.3.3 (Free group)**.** *Let* A *be a set, and* $\mathsf{List}(\text{–})$ *the free monoid. The free group* $F(A)$ *on* A *is the set-quotient of* $\mathsf{List}(A + A)$ *by the congruence closure of the relation* $a :: a^{-1} :: \mathsf{nil} \sim \mathsf{nil}$ *and* $a^{-1} :: a :: \mathsf{nil} \sim \mathsf{nil}$*.*

Proposition 7.3.4. $F(A) \triangleq \mathsf{List}(A + A)/{\sim^*}$ *has a group structure, with the empty list* nil *for the neutral element, multiplication given by list append* $+\!\!+$*, and inverse given by flipping* inl *and* inr*, followed by reversing the list. Further,* $F(A)$ *with* $\eta_A(a) \triangleq \mathsf{inl}(a) :: \mathsf{nil}$ *satisfies the universal property of free groups, as stated in Proposition 7.3.2.*



(a) cancel (b) swap (c) braid

Figure 7.1: Braiding diarams for Coxeter relations.

7.4. Group presentations

A presentation of a group builds it by starting from the free group $F(A)$ and introducing additional equations that are satisfied in the resulting group. For example, if we take $F(\mathbf{1}) \triangleq \mathbb{Z}$ and add an equation $1 + 1 = 0$, the resulting group would be $\mathbb{Z}_2 \simeq \mathsf{Aut}(\mathbf{2})$. Note that not all groups have finite (or computable) presentations, and a group can have a number of different presentations.

Definition 7.4.1 (Group presentation)**.** *Let A be a set and $R : \mathsf{List}(A + A) \to \mathsf{List}(A + A) \to \mathcal{U}$ a binary relation on $\mathsf{List}(A + A)$. The group $F(\langle A; R\rangle)$ presented by A and R, is given by the set-quotient of the free group $F(A)$ by the congruence closure of R.*

The universal property of the above definition is similar to Proposition 7.3.2 except the relation has to be preserved by the function mapping out of the generating set.

Proposition 7.4.2 (Universal property of $F(\langle A; R\rangle)$)**.** *Given a group G and a map $f : A \to G$, such that f extended to $F(A)$ respects R, there is a unique group homomorphism $f^{\#} : F(\langle A; R\rangle) \to^{G} G$ such that $f^{\#} \circ \eta_A \sim f$.*

Before, the only way to decide the equality of two elements in a group was to evaluate and check them on the nose, but in a group presentation, this is reduced to deciding whether one word – a representative of the equivalence class of the group's elements, can be reduced to another word, using the group's relations. However, these equations are not directed, so it is not always possible to construct a well-behaved rewriting system. In general, the word problem for groups is undecidable.

Coxeter Presentation To present the group $\mathcal{S}_n$, the primitive operations we use will be adjacent swaps. When dealing with permutations on an $n+1$-element set, there are n adjacent transpositions – transposition number k swaps elements at indices k and $k + 1$. Thus, the generating set is $\mathsf{Fin}_{\mathsf{n}}$. There are three relations that we're going to specify for this presentation – we visualise them as braid diagrams in Figure 7.1.

7.1a Swapping the same two elements twice in a row should be the same as doing nothing.

7.1b When swapping two distinct pairs of elements, the order in which swapping happens should not matter, that is, we can slide the wires freely.

7.1c There are two equivalent ways of swapping the first and last elements in a sequence of three elements.

This construction is called a Coxeter presentation of $\mathcal{S}_n$. Writing it formally, we encode the rules discussed above using a relation $\leftrightsquigarrow$ on $\mathsf{List}(\mathsf{Fin}_{\mathsf{n}})$ (Definition 7.4.3), and then take its congruence closure $\overset{*}{\leftrightsquigarrow}$ (Definition 7.4.4).

Definition 7.4.3 ($\leftrightsquigarrow$: $\mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n) \to \mathcal{U}$).

$$\begin{aligned}
\mathsf{cancel} &: \forall n \to (n :: n :: \mathsf{nil}) \leftrightsquigarrow \mathsf{nil} \\
\mathsf{swap} &: \forall k, n \to (\mathsf{S}\, k < n) \to (n :: k :: \mathsf{nil}) \leftrightsquigarrow (k :: n :: \mathsf{nil}) \\
\mathsf{braid} &: \forall n \to (\mathsf{S}\, n :: n :: \mathsf{S}\, n :: \mathsf{nil}) \leftrightsquigarrow (n :: \mathsf{S}\, n :: n :: \mathsf{nil})
\end{aligned}$$

Definition 7.4.4 ($\overset{*}{\leftrightsquigarrow}$: $\mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n) \to \mathcal{U}$).

$$\begin{aligned}
\leftrightsquigarrow\text{-}\mathsf{refl} &: \forall w \to w \overset{*}{\leftrightsquigarrow} w \\
\leftrightsquigarrow\text{-}\mathsf{sym} &: \forall w_1, w_2 \to w_1 \overset{*}{\leftrightsquigarrow} w_2 \to w_2 \overset{*}{\leftrightsquigarrow} w_1 \\
\leftrightsquigarrow\text{-}\mathsf{trans} &: \forall w_1, w_2, w_3 \to w_1 \overset{*}{\leftrightsquigarrow} w_2 \to w_2 \overset{*}{\leftrightsquigarrow} w_3 \to w_1 \overset{*}{\leftrightsquigarrow} w_3 \\
\leftrightsquigarrow\text{-}\mathsf{cong}\text{-}+ &: \forall w_1, w_2, w_3, w_4 \to w_1 \overset{*}{\leftrightsquigarrow} w_2 \to w_3 \overset{*}{\leftrightsquigarrow} w_4 \to w_1 + w_3 \overset{*}{\leftrightsquigarrow} w_2 + w_4 \\
\leftrightsquigarrow\text{-}\mathsf{rel} &: \forall w_1, w_2 \to w_1 \leftrightsquigarrow w_2 \to w_1 \overset{*}{\leftrightsquigarrow} w_2
\end{aligned}$$

The idea for solving the word problem for $\mathcal{S}_n$ is to turn these undirected relations into a rewriting system $(\mathsf{List}(\mathsf{Fin}_n), \overset{*}{\leftrightsquigarrow})$, so that, by repeatedly applying the reduction rules as long as possible, any two $\overset{*}{\leftrightsquigarrow}$-equal terms would eventually converge to the same normal form.

For this to work, we first need the system to have the strong normalization property (as defined in Definition 3.1.11). We define the following ordering on words:

Definition 7.4.5. *For any two words $w, v : \mathsf{List}(\mathsf{Fin}_n)$, we have $w < v$ if either:*

- *w is shorter than v, or*
- *w and v have the same length, and w is lexicographically smaller than v.*

Lemma 7.4.6. *The $<$ order is well-founded.*

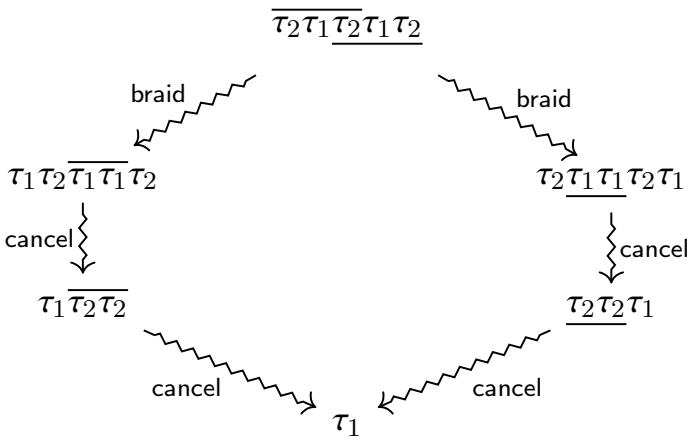
Now, we can observe that after throwing out reflexivity and symmetry, the right hand sides of the relations $\overset{*}{\leftrightsquigarrow}$ are strictly smaller than the left hand sides, in terms of $<$. Thus, by directing the relation from left to right, we would get the normalization property (c.f. Definition 3.1.11) out of the box. Second, we need the normal forms to be unique, so that we can get a normalisation function. This will be true if the rewriting system has local confluence property (c.f. Definition 3.1.7). For example, in our system, the pair in Figure 7.2a converges. Unfortunately, this is not true for all critical pairs – an example is in Figure 7.2b, where left and right endpoints are normal with respect to the $\overset{*}{\leftrightsquigarrow}$ relation.

7.5. Rewriting via Coxeter

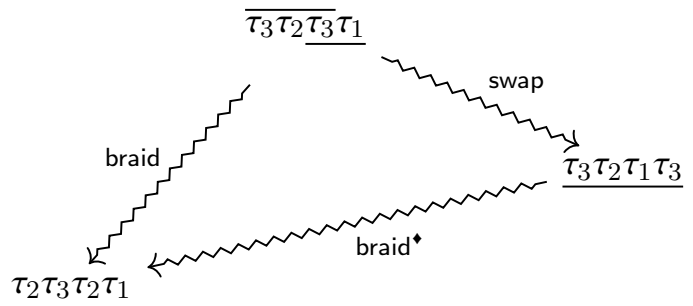
Because of this counter-example, the relations have to be changed. In this section, we formally define a rewriting system $(\mathsf{List}(\mathsf{Fin}_n), \rightsquigarrow)$, partially based on the Coxeter relations, and prove that it has the desired properties of confluence and termination. We prove that the new relation defined by this system is equivalent, in a technical sense, to the standard Coxeter relation $\overset{*}{\leftrightsquigarrow}$. First, we need to define a function $n \swarrow k$.

Definition 7.5.1 ($\swarrow$: $(n : \mathbb{N}) \to (k : \mathbb{N}) \to \mathsf{List}(\mathsf{Fin}_{k+n})$).

$$\begin{aligned}
n \swarrow 0 &\triangleq \mathsf{nil} \\
n \swarrow \mathsf{S}\, k &\triangleq (k + n) :: (n \swarrow k)
\end{aligned}$$

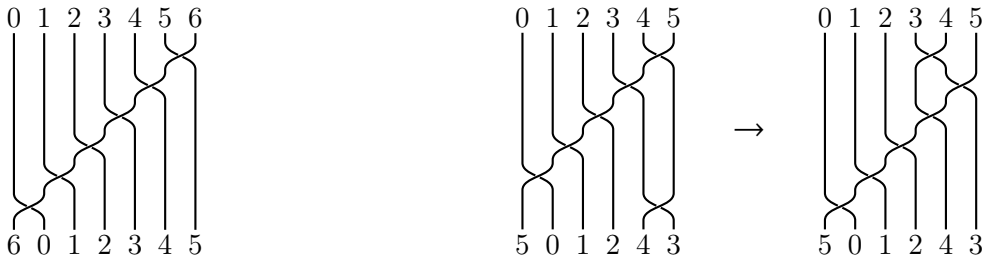


(a) Converging critical pair



(b) Non-converging critical pair that converges with braid♦

Figure 7.2: Critical Pairs in $\overset{*}{\rightsquigarrow}$



(a) $0 \swarrow 6$ (b) $\mathsf{braid}^\blacklozenge$ with $n = 0$ and $k = 3$

Figure 7.3: Braiding diagrams for modified Coxeter relations.

The result of this function is the sequence $[k + n - 1, k + n - 2, k + n - 3, \ldots, n]$, which is a sequence of transpositions that moves the element at index $k + n$ left by k places, shifting all the elements in between one place right (see Figure 7.3a). Then, the directed relation $\rightsquigarrow$ is defined with the following generators (inlining the congruence closure in $\rightsquigarrow$, allowing arbitrary reductions inside the list).

Definition 7.5.2 ($\rightsquigarrow$: $\mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n) \to \mathcal{U}$)**.**

$$\mathsf{cancel}^\blacklozenge : \forall n, l, r \to (l +\!\!+ n :: n :: r) \rightsquigarrow (l +\!\!+ r)$$
$$\mathsf{swap}^\blacklozenge : \forall n, k, l, r \to (\mathsf{S}\, k < n) \to (l +\!\!+ n :: k :: r) \rightsquigarrow (l +\!\!+ k :: n +\!\!+ r)$$
$$\mathsf{braid}^\blacklozenge : \forall n, k, l, r \to (l +\!\!+ (n \swarrow 2 + k) +\!\!+ (1 + k + n) :: r) \rightsquigarrow (l +\!\!+ (k + n) :: (n \swarrow 2 + k) +\!\!+ r)$$

Constructors $\mathsf{cancel}^\blacklozenge$ and $\mathsf{swap}^\blacklozenge$ correspond directly to the appropriate constructors of $\leftrightsquigarrow$ and can be visualised in the same way as before. The remaining constructor $\mathsf{braid}^\blacklozenge$ uses the $\swarrow$ function to exchange the order of a long sequence of transpositions and a single transposition afterwards. For example, for $n = 0$ and $k = 3$, it allows for the reduction $[4, 3, 2, 1, 0, 4] \rightsquigarrow [3, 4, 3, 2, 1, 0]$ (see Figure 7.3b – note the distinction between wires, where numbers represent the values, and transpositions, where numbers represent which wires are crossing).

Note that the previous braid rule is a special case of $\mathsf{braid}^\blacklozenge$, with $k = 0$. As before, the left-hand sides of the relation are (lexicographically) strictly larger than the right-hand sides. We define the transitive closure of $\rightsquigarrow$ to be $\overset{+}{\rightsquigarrow}$ (Definition 7.5.4) and its reflexive-transitive closure to be $\overset{*}{\rightsquigarrow}$ (Definition 7.5.3).

Definition 7.5.3 ($\overset{*}{\rightsquigarrow}$: $\mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n) \to \mathcal{U}$)**.**

$$\rightsquigarrow\text{-}\mathsf{refl} : \forall w \to w \overset{*}{\rightsquigarrow} w$$
$$\rightsquigarrow\text{-}\mathsf{trans} : \forall w_1, w_2, w_3 \to w_1 \rightsquigarrow w_2 \to w_2 \overset{*}{\rightsquigarrow} w_3 \to w_1 \overset{*}{\rightsquigarrow} w_3$$

Definition 7.5.4 ($\overset{+}{\rightsquigarrow}$: $\mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n) \to \mathcal{U}$)**.**

$$\overset{+}{\rightsquigarrow}\text{-}\mathsf{rel} : \forall w_1, w_2 \to w_1 \rightsquigarrow w_2 \to w_1 \overset{+}{\rightsquigarrow} w_2$$
$$\overset{+}{\rightsquigarrow}\text{-}\mathsf{trans} : \forall w_1, w_2, w_3 \to w_1 \rightsquigarrow w_2 \to w_2 \overset{+}{\rightsquigarrow} w_3 \to w_1 \overset{+}{\rightsquigarrow} w_3$$

Despite the increased complexity of the generators, the rewriting system $(\mathsf{List}(\mathsf{Fin}_n), \rightsquigarrow)$ has the properties we desire. It satisfies local confluence – for example, using $\mathsf{braid}^\blacklozenge$, we can now show that the problematic critical pair Figure 7.2b converges - and it is strongly normalizing, so by Newman's lemma (Theorem 3.1.12), it produces a unique normal form.

Theorem 7.5.5.

1. $\rightsquigarrow$ *is locally confluent.*
2. $\overset{+}{\rightsquigarrow}$ *is strongly normalizing (since for every* $w \overset{+}{\rightsquigarrow} v$*, it holds that* $v < w$*).*
3. $\overset{*}{\rightsquigarrow}$ *has the Church-Rosser property.*
4. *For every* w*, there exists a unique normal form* v *such that* $w \overset{*}{\rightsquigarrow} v$*.*

The modified form of the Coxeter relations are unwieldy and difficult to prove properties about by induction. However, we can make the following observation relating it to $\overset{*}{\leftrightsquigarrow}$.

Proposition 7.5.6. *The relations* $\overset{*}{\leftrightsquigarrow}$ *and* $\overset{*}{\rightsquigarrow}$ *are equivalent in the following sense: for every* w *and* v*,* $w \overset{*}{\leftrightsquigarrow} v$ *iff there is a* u *such that* $w \overset{*}{\rightsquigarrow} u$ *and* $v \overset{*}{\rightsquigarrow} u$*.*

By Theorem 7.5.5(4), we get a unique choice function $\mathfrak{nf} : \mathsf{List}(\mathsf{Fin}_n) \to \mathsf{List}(\mathsf{Fin}_n)$ that produces a normal form for terms of $\mathsf{List}(\mathsf{Fin}_n)$. We state two important properties enjoyed by $\mathfrak{nf}$.

Proposition 7.5.7.

1. *For all* $l : \mathsf{List}(\mathsf{Fin}_n)$*, we have that* $l \overset{*}{\leftrightsquigarrow} \mathfrak{nf}(l)$*.*
2. $\mathfrak{nf}$ *is idempotent, that is,* $\mathfrak{nf} \circ \mathfrak{nf} \sim \mathfrak{nf}$*.*

Finally, we define the type $\mathcal{S}_n$ as the set-quotient of $\mathsf{List}(\mathsf{Fin}_n)$ by $\overset{*}{\leftrightsquigarrow}$,

Definition 7.5.8 ($\mathcal{S}_n$)**.** $\mathcal{S}_n \triangleq \mathsf{List}(\mathsf{Fin}_n)/\overset{*}{\leftrightsquigarrow}$

Note that reductions need not be unique, hence $w \overset{*}{\leftrightsquigarrow} v$ is not necessarily a proposition. So, the quotient $\mathcal{S}_n$ is not effective, that is, $\mathsf{q\text{-}rel} : w \overset{*}{\leftrightsquigarrow} v \to q(w) = q(v)$ is not an equivalence. Using the $\mathfrak{nf}$ function, we could instead define a new relation $\approx$ to equate those terms that have the same normal form, $(w \approx v) \triangleq (\mathfrak{nf}(w) = \mathfrak{nf}(v))$. This relation is prop-valued, and we could quotient $\mathsf{List}(\mathsf{Fin}_n)$ by $\approx$, obtaining an equivalent definition for $\mathcal{S}_n$.

Proposition 7.5.9.

1. $\mathfrak{nf}$ *splits into a section-retraction pair, that is, we have* $\mathsf{List}(\mathsf{Fin}_n) \overset{s}{\to} \mathcal{S}_n \overset{r}{\to} \mathsf{List}(\mathsf{Fin}_n)$ *such that* $s \circ r \sim \mathfrak{nf}$ *and* $r \circ s \sim \mathsf{id}_{\mathcal{S}_n}$*.*
2. $\mathsf{im}(\mathsf{q}) \simeq \mathcal{S}_n \simeq \mathsf{im}(\mathfrak{nf})$

Notice however, that a group presentation, as defined in Definition 7.3.3, requires the relation to be on the set of words $A + A$, where the right copy corresponds to the set of formal inverses of the generators. The constructor cancel specifies that the inverse of each element is again the same element, using which we can show that our definition of $\mathcal{S}_n$ is equivalent to the definition of a presented group, by lifting the $\overset{*}{\leftrightsquigarrow}$ relation.

Theorem 7.5.10.

1. *There is a group structure on* $\mathcal{S}_n$*, where the identity element is* nil*, multiplication is given by list append, and inverse is given by list reversal.*
2. $\mathcal{S}_n$ *is equivalent to the group presented by generators* Fin_n *with the relations given by* $\overset{*}{\leftrightsquigarrow}$ *extended to* $\mathsf{List}(\mathsf{Fin}_n + \mathsf{Fin}_n)$ *along the co-diagonal map* $\nabla_A : A + A \to A$*.*

To decide if two words in $\mathsf{List}(\mathsf{Fin}_n)$ are $\overset{*}{\leftrightsquigarrow}$-equal, we simply have to compute their normal forms using $\mathfrak{nf}$. They correspond to the same permutation if and only if these normal forms are equal, which is decidable for $\mathsf{List}(\mathsf{Fin}_n)$.

7.6. Lehmer Codes

To prove the equivalence between $\mathsf{Aut}(\mathsf{Fin}_n)$ and $\mathcal{S}_n$, we will need to define functions back and forth between the two types. The terms in $\mathcal{S}_n$ can be identified with equivalence classes of terms in $\mathsf{List}(\mathsf{Fin}_n)$ with respect to the Coxeter relation $\overset{*}{\leftrightsquigarrow}$. The easiest way to define a function out of this presentation is to define it on the representatives. We know that these are the unique normal forms in the set-quotient given by $\mathfrak{q} \circ \mathfrak{nf}$, but now we will explicitly describe what these representatives look like, using an encoding called Lehmer codes [Lehmer, 1960]. Formally, we define $\mathsf{Lehmer}(n)$ to be an $n+1$-element tuple, where the position $k \leq n$ stores an element of Fin_k. Since the 0-th position is trivial, in practice it is ignored [Dubois and Giorgetti, 2018, Vajnovszki, 2011].

Definition 7.6.1 ($\mathsf{Lehmer} : \mathbb{N} \to \mathcal{U}$)**.**

$$\begin{aligned} \mathsf{Lehmer}(0) &\triangleq \mathsf{Fin}_{\mathsf{s}\,0} \\ \mathsf{Lehmer}(\mathsf{S}\,n) &\triangleq \mathsf{Lehmer}(n) \times \mathsf{Fin}_{\mathsf{s}\,\mathsf{s}\,n} \end{aligned}$$

Given a permutation $\sigma : \mathsf{Aut}(\mathsf{Fin}_n)$, for any element $i : \mathsf{Fin}_n$, we can define the inversion count of i as the number of smaller elements appearing after it in the permutation.

Definition 7.6.2 (Inversion count)**.** *Given a permutation $\sigma : \mathsf{Aut}(\mathsf{Fin}_n)$, the inversion count of $k : \mathsf{Fin}_n$ is given by*

$$\mathsf{inv}_\sigma(i) = \#\{j < i | \sigma(j) > \sigma(i)\}$$

Just knowing the inversion counts for all the elements, one can reconstruct the starting permutation. Also, observe that $\mathsf{inv}_\sigma(i) < i$, thus fitting in the i-th place of a Lehmer code tuple. As an example, consider the following tabulated presentation of a permutation of Fin_5.

$$\sigma = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 \\ \mathbf{2} & \mathbf{1} & \mathbf{4} & \mathbf{0} & \mathbf{3} \end{pmatrix}$$

The Lehmer code for the permutation σ is then the 5-tuple

$$l = (\mathsf{inv}_\sigma(0), \mathsf{inv}_\sigma(1), \mathsf{inv}_\sigma(2), \mathsf{inv}_\sigma(3), \mathsf{inv}_\sigma(4)) = (0, 1, 2, 0, 2)$$

To decode the permutation back from this Lehmer code, we perform an algorithm similar to *insertion sort*. The element of the Lehmer code being currently processed is highlighted in the left column of the table below. Starting from a sorted list, the element at index k has to be given $l[k]$ inversions. Because of the the invariant that all the elements before newly processed one are smaller than it, the proper number of inversions is created by simply shifting the element $l[k]$ places left.

$(0, 1, 2, 0, 2)$	$[\mathbf{0}, \mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}]$
$(0, 1, 2, 0, 2)$	$[\mathbf{1}, \mathbf{0}, \mathbf{2}, \mathbf{3}, \mathbf{4}]$
$(0, 1, 2, 0, 2)$	$[\mathbf{2}, \mathbf{1}, \mathbf{0}, \mathbf{3}, \mathbf{4}]$
$(0, 1, 2, 0, 2)$	$[\mathbf{2}, \mathbf{1}, \mathbf{0}, \mathbf{3}, \mathbf{4}]$
$(0, 1, 2, 0, 2)$	$[\mathbf{2}, \mathbf{1}, \mathbf{4}, \mathbf{0}, \mathbf{3}]$

Writing formally, to turn a Lehmer code into a word in $\mathcal{S}_n$, we define a function $\mathfrak{em}$. As described above, the number r at position k in the tuple describes how many inversions the element $\mathbf{k}$ has. Thus, we need to perform r many adjacent transpositions to get to the desired position, which is given by $(\mathsf{S}\,n - r) \swarrow r$.

Definition 7.6.3 ($\mathfrak{em} : (n : \mathbb{N}) \to \mathsf{Lehmer}(n) \to \mathsf{List}(\mathsf{Fin}_{\mathsf{S}\,n})$)**.**

$$\mathfrak{em}_0(0) \triangleq \mathsf{nil}$$
$$\mathfrak{em}_{\mathsf{S}\,n}((r, l)) \triangleq \mathfrak{em}_n(l) +\!\!+ ((\mathsf{S}\,n - r) \swarrow r)$$

We can show that the function $\mathfrak{em}_n$ gives an equivalence between $\mathsf{Lehmer}(n)$ and $\mathsf{im}(\mathfrak{nf})$.

Theorem 7.6.4.

1. *For any Lehmer code c, $\mathfrak{em}_n(c)$ is a normal form with respect to $\overset{*}{\rightsquigarrow}$, that is, $\mathfrak{em}_n(c)$ is in $\mathsf{im}(\mathfrak{nf})$.*
2. *Any element of $\mathsf{im}(\mathfrak{nf})$ can be constructed from a unique Lehmer code by $\mathfrak{em}$, that is, the fibers of $\mathfrak{em}_n : \mathsf{Lehmer}(n) \to \mathsf{im}(\mathfrak{nf})$ are contractible.*

Therefore, there is an equivalence between $\mathsf{Lehmer}(n)$ and $\mathsf{im}(\mathfrak{nf})$.

Proof. For any code $c : \mathsf{Lehmer}(n)$, we have that

$$\mathfrak{em}_n(c) = (r_0 \swarrow k_0) +\!\!+ (r_1 \swarrow k_1) +\!\!+ \cdots +\!\!+ (r_{m-1} \swarrow k_{m-1})$$

is a concatenation of $n \geq m \geq 0$ non-empty strictly decreasing lists. Reductions in $\rightsquigarrow$ cannot happen inside any of the strictly decreasing $(r \swarrow k)$: $\mathsf{cancel}^\blacklozenge$ requires repeating elements, $\mathsf{swap}^\blacklozenge$ acts when consecutive numbers differ by at least 2, and $\mathsf{braid}^\blacklozenge$ acts on a non-monotone sequence. This leaves the case of reduction happening on a fragment that borders two (or more) subsequences. Again, $\mathsf{cancel}^\blacklozenge$ requires two equal consecutive numbers, which would then have to be the last one in some $(r_i \swarrow k_i)$ sequence, and the first one in the next $(r_{i+1} \swarrow k_{i+1})$. But the first number in a sequence $(r_{i+1} \swarrow k_{i+1})$ is larger than every number in $(r_i \swarrow k_i)$ – which also shows why $\mathsf{swap}^\blacklozenge$ cannot happen. The remaining case of $\mathsf{braid}^\blacklozenge$ follows similarly, since its argument is a decreasing sequence followed immediately by a number equal to the first element of this sequence. □

Corollary 7.6.5. *For all $n : \mathbb{N}$, $\mathcal{S}_n \simeq \mathsf{im}(\mathfrak{nf}) \simeq \mathsf{Lehmer}(n)$.*

7.7. Running Lehmer Codes

Finally, it is time to complete our goal of characterising the symmetric groups. Having produced a Lehmer code by normalising words in $\mathcal{S}_n$, we need to run it to produce a concrete bijection of finite sets, and, given a bijection between finite sets, we need to encode it as a Lehmer code. We will prove that these maps construct an equivalence between the types $\mathsf{Lehmer}(n)$ and $\mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}\,n})$. The idea for this proof is borrowed from Molzer [2021]. [1]

Definition 7.7.1 ($\mathsf{Fin}^-_{\mathsf{n}} : \mathsf{Fin}_{\mathsf{n}} \to \mathcal{U}$)**.** *For $n : \mathbb{N}$, the type family $\mathsf{Fin}^-_{\mathsf{n}}$ picks out all elements in $\mathsf{Fin}_{\mathsf{n}}$ except the one in the argument.*

$$\mathsf{Fin}^-_{\mathsf{n}}(i) \triangleq \sum_{j:\mathsf{Fin}_{\mathsf{n}}} i \neq j$$

Note that $\mathsf{Fin}^-_{\mathsf{n}}(i)$ for $i : \mathsf{Fin}_{\mathsf{n}}$ is a subtype of $\mathsf{Fin}_{\mathsf{n}}$ and is hence a set. We state and prove a few auxiliary lemmas about how $\mathsf{Fin}^-_{\mathsf{n}}$ interacts with Fin – these are obtained by counting arguments using the fact that $\mathsf{Fin}_{\mathsf{n}}$ and $\mathsf{Fin}^-_{\mathsf{n}}$ both have decidable equality.

[1] Note that the indices for the type of permutations are off-by-one, because we chose $\mathsf{Fin}_{\mathsf{n}}$ to represent generators for permutations on $\mathsf{Fin}_{\mathsf{S}\,\mathsf{n}}$.

Lemma 7.7.2.

1. *For any* $k : \mathsf{Fin}_{\mathsf{S}n}$, *we have* $\mathsf{Fin}^{-}_{\mathsf{S}n}(k) \simeq \mathsf{Fin}_n$.
2. *For any* $n : \mathbb{N}$, *we have*

$$\mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}n}) \simeq \sum_{k:\mathsf{Fin}_{\mathsf{S}n}} \left(\mathsf{Fin}^{-}_{\mathsf{S}n}(n) \simeq \mathsf{Fin}^{-}_{\mathsf{S}n}(n-k)\right)$$

.

Proof. The first proposition follows from simply constructing the bijections using the decidable equality of Fin_n, and making sure to punch-in and punch-out the element k at the right place.

The second proposition involves some combinatorial tricks. On the left, we have the type of automorphisms of $\mathsf{Fin}_{\mathsf{S}n}$. Assume a particular $\sigma : \mathsf{Fin}_{\mathsf{S}n} \xrightarrow{\sim} \mathsf{Fin}_{\mathsf{S}n}$. Pick k to be the inversion count of n, the largest element in $\mathsf{Fin}_{\mathsf{S}n}$. Then, the image of n under ϕ has to be $n-k$, since all other elements in the set are smaller. Removing those two from the domain and co-domain of ϕ, the rest of the elements are fixed by σ, so we compute the bijection between the rest of the elements.

For the other direction, if we are given a k and a bijection π between $\mathsf{Fin}^{-}_{\mathsf{S}n}(n)$ and $\mathsf{Fin}^{-}_{\mathsf{S}n}(n-k)$, we can extend π to $\sigma : \mathsf{Fin}_{\mathsf{S}n} \simeq \mathsf{Fin}_{\mathsf{S}n}$ by inserting the element n at the position $n-k$, resulting in the element n having inversion count k. □

Using these facts, we can now prove the main result of this section.

Theorem 7.7.3. *For all* $n : \mathbb{N}$, $\mathsf{Lehmer}(n) \simeq \mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}n})$.

Proof. For $n = 0$, note that $\mathsf{Lehmer}(0)$ is contractible, and so is $\mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}0})$. For $n = \mathsf{S}\,m$, we compute a chain of equivalences.

			$\mathsf{Lehmer}(\mathsf{S}\,m)$	
		$\simeq$	$\mathsf{Fin}_{\mathsf{S}\mathsf{S}m} \times \mathsf{Lehmer}(m)$	by definition
	$\mathsf{Lehmer}(0)$	$\simeq$	$\mathsf{Fin}_{\mathsf{S}\mathsf{S}m} \times \mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}m})$	induction hypothesis
$\simeq$	$\mathbf{1}$	$\simeq$	$\sum_{k:\mathsf{Fin}_{\mathsf{S}\mathsf{S}m}} \mathsf{Fin}_{\mathsf{S}m} \simeq \mathsf{Fin}_{\mathsf{S}m}$	Σ over a constant family
$\simeq$	$\mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}0})$	$\simeq$	$\sum_{k:\mathsf{Fin}_{\mathsf{S}\mathsf{S}m}} \mathsf{Fin}^{-}_{\mathsf{S}\mathsf{S}m}(m) \simeq \mathsf{Fin}_{\mathsf{S}m}$	by Lemma 7.7.2 Item 1
		$\simeq$	$\sum_{k:\mathsf{Fin}_{\mathsf{S}\mathsf{S}m}} \mathsf{Fin}^{-}_{\mathsf{S}\mathsf{S}m}(m) \simeq \mathsf{Fin}^{-}_{\mathsf{S}\mathsf{S}m}(m-k)$	by Lemma 7.7.2 Item 1
		$\simeq$	$\mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}\mathsf{S}m})$	by Lemma 7.7.2 Item 2

□

By composing Theorem 7.7.3 and Corollary 7.6.5, we obtain the final equivalence.

Corollary 7.7.4. *For all* $n : \mathbb{N}$, $\mathcal{S}_n \simeq \mathsf{Lehmer}(n) \simeq \mathsf{Aut}(\mathsf{Fin}_{\mathsf{S}n})$.

Chapter 8

Complexity questions

8.1. Π program equivalence problem

Based on what we did up to this point, one way to decide the Π program equivalence is to use Normalization-by-Evaluation and compare the denotations of Π programs in $\mathcal{U}_{\mathsf{Fin}}$. However, our proof suffers from the exponential explosion of types and 1-combinators during the translation between Π and Π^+, when we implement multiplication as repeated addition. This makes the construction of the denotations require possibly exponential time and memory, in the size of the input program. A natural question to ask is thus: is it possible to do better?

The answer is: no, we cannot hope for a more efficient solution, unless some hard hypotheses in computer science were resolved to be true. To prove that, we have to start from the notion of a Boolean circuit equivalence problem.

Definition 8.1.1. *For a finite set of Boolean functions B, a Boolean circuit is a tuple*

$$C = (V, E, \alpha, \beta, o)$$

where (V, E) is a finite directed acyclic graph, $o \in V$ is an output gate, $\alpha : E \to \mathbb{N}$ is an injective function and $\beta : V \to B \cup \{x_i : i \in \mathbb{N}\}$, such that the following conditions hold:

- *If $v \in V$ has in-degree 0, then $\beta(v) \in \{x_i : i \in \mathbb{N}\}$, and we say that v is an input node*
- *If $v \in V$ has in-degree $k > 0$, then $\beta(v)$ is a k-ary function.*
- *Every $v \in V$ o has out-degree equal to 1, while o has out-degree equal to 0.*

We say that the input of a Boolean circuit is a mapping $\phi : (x_1, x_2, \ldots x_n)$ to $\mathbf{2} \triangleq \{0, 1\}$.

Definition 8.1.2. *For a Boolean circuit $C = (V, E, \alpha, \beta, o)$, for each $v \in V$ we define its value $\mathsf{val}(v)$ on input ϕ as follows:*

- *If v in an input node with $b(v) = x_i$, then $\mathsf{val}(v) = \phi(x_i)$*
- *Otherwise, $\mathsf{val}(v)$ is the value of its Boolean function $\beta(v)$ computed on $(\mathsf{val}(s_1), \mathsf{val}(s_1), \ldots \mathsf{val}(s_k))$, where s_i are predecessors to v and $\alpha((s_i, v)) < \alpha((s_j, v))$ for $i < j$.*

We define the value of the Boolean circuit C as $\mathsf{val}(o)$.

The problem EQ of circuit equivalence is to decide whether for two circuits C_1, C_2, their value is the same for all inputs. The problem is in co-NP, since computing a value of a circuit on an input can be done in $PTIME$, and the witness of circuit non-equivalence is the input on which they differ. Using EQ, we can characterise the co-NP class [Blum, 2007].

Theorem 8.1.3. *EQ is co-NP complete.*

A reversible circuit is a circuit composed from reversible gates, that is, the set of Boolean functions in Definition 8.1.1 is replaced by a set of invertible Boolean functions - bijections $\mathbf{2}^n \to \mathbf{2}^n$, and the condition for out-degrees is changed such that every node (except input nodes and the output node) has an in-degree equal to its out-degree.

Lemma 8.1.4. *Reversible Boolean circuit equivalence problem is in co-NP.*

Note that it is not immediately obvious whether the reversible version of the circuit equivalence is still co-NP hard! This is because there is additional structure that one might use to derive the equivalence - namely, the behaviour on all additional outputs. However, it is still possible to prove the following theorem [Jordan, 2013].

Theorem 8.1.5. *Reversible Boolean circuit equivalence is co-NP complete.*

Applying this result in the context of Π requires the following lemma:

Lemma 8.1.6. *A reversible Boolean circuit of size n can be encoded to (and decoded from) a Π program of size $O(n)$.*

Proof. Toffoli gate is universal for reversible Boolean circuits, meaning that any reversible circuit can be written using Toffoli gates, and a constant number of ancilla bits[Toffoli, 1980]. Toffoli gate can be implemented in Π, as follows. First, we define controlled operator as

$$\begin{aligned}&\mathsf{controlled} : \forall a.(a \leftrightarrow_1 a) \to (2 \times a) \leftrightarrow_1 (2 \times a)\\&\mathsf{controlled} = \mathsf{dist} \circledcirc (\mathsf{id}\leftrightarrow_1 \oplus (\mathsf{id}\leftrightarrow_1 \otimes f)) \circledcirc \mathsf{factorzl}\end{aligned}$$

In other words, controlled checks whether the first value is false (left) or true (right). If it is false - leaves the input as it was, if it was true - applies f to the input.

Applying the controlled to swap⋆ we get the controlled-not gate we used as the example when introducing Π in Chapter 5.

$$\begin{aligned}&\mathsf{cnot} : (2 \times 2) \leftrightarrow_1 (2 \times 2)\\&\mathsf{cnot} = \mathsf{controlled}\ \ \mathsf{swap}\star\end{aligned}$$

Applying the controlled operator again, we get the Toffoli gate.

$$\begin{aligned}&\mathsf{toffoli} : (2 \times 2) \leftrightarrow_1 (2 \times 2)\\&\mathsf{toffoli} = \mathsf{controlled}\ \ \mathsf{cnot}\end{aligned}$$

Because of its universality, all other reversible gates (i.e. all Π 1-combinators) can be built using a finite number of toffoli gates. □

Corollary 8.1.7. *Equivalence of Π programs, and thus deciding equality of morphism in weak symmetric rig groupoids, is co-NP hard.*

Corollary 8.1.8. *If, for any two* Π *programs* P_1, P_2 *of size* n *for which there exits a 2-combinator between them, there also exists a 2-combinator of size* $poly(n)$*, then* $co-NP \subseteq NP$*.*

Thus, efficiently solving the equivalence problem in general is difficult. However, it is still possible that the search over Π 2-combinators can yield more efficient (in practice, or even on average) heuristic than solving the reversible circuit equivalence problem by brute-force. In particular, we can observe that a proof of equivalence can be very short, relative to the size of the programs, so such heuristics might prove complementary to the "extensional" comparison.

8.2. Synthesising short Π programs

As we already pointed out, our proof suffers from the exponential explosion of types and 1-combinators during the translation between Π and Π^+. This can make the synthesised reversible circuits very large and inefficient. Thus, another complexity-related question we might want to ask is: is it possible, for an arbitrary bijection, to write a short program in Π that implements it?

The answer to this is, again, negative. The argument follows from Claude Shannon's counting-based proof for the existence of Boolean functions that cannot be encoded in a small circuit [Aaronson, 2017].

Theorem 8.2.1. *There exists a Boolean function* $f : \mathbf{2}^n \to \mathbf{2}$ *such that any circuit to compute* f *requires at least* $\Omega(\frac{2^n}{n})$ *logic gates.*

From this theorem, we get the following corollary.

Corollary 8.2.2. *There exists a bijection* $f : \mathbf{2}^n \to \mathbf{2}^n$ *such that any* Π *program expressing* f *requires at least* $\Omega(\frac{2^n}{n})$ *combinators.*

Proof. From any reversible function $g : \mathbf{2}^n \to \mathbf{2}^n$, we can get a Boolean function $\hat{g} : \mathbf{2}^n \to \mathbf{2}$ by only looking at the first output, and ignoring the rest of $n-1$ outputs.

We say that a Boolean function $f : \mathbf{2}^n \to \mathbf{2}^n$ is embedded in a reversible function $g : \mathbf{2}^m \to \mathbf{2}^m$, if we have that $\hat{g}(\vec{x}, \vec{0}) = f(\vec{x})$, with $n \leq m$. The task of finding, for a given f, the minimal value of m such that there is a reversible function g with m inputs is the *embedding problem.* It is easy to show that we always have $n \leq m \leq n+1$ - since there is at most 2^n duplicated f outputs, having additional n bits is always enough to make it reversible[Maslov and Dueck, 2004].

Let us take f to be such as given by the Theorem 8.2.1. Then, there exists a reversible function g of size $\leq n+1$ in which f is embedded. Thus, the size of any reversible circuit implementing g has to be at least $\Omega(\frac{2^n}{n})$. The conclusion follows from Lemma 8.1.6. □

Chapter 9

Proof assistant

All of the theorems statements and proofs in Chapter 4 and Chapter 7, as well as a large part of Chapter 6 had been formalised in the proof assistant Agda, using the HoTT-Agda library. Full formalisation took about 7500 lines of code. The human-readable versions presented in this thesis are a de-formalisation of the proofs that were first done mechanically.

Curry-Howard correspondence, applied to the proof system that we use, allows us to extract the theorems as programs. In addition, even though taking the long route of proving a correspondence between Π and $\mathcal{U}_{\mathsf{Fin}}$ required more work, it is rewarded by a more comprehensive *software library* for dealing with permutations in general.

9.1. Normalisation-by-evaluation

Having computable semantics for Π language allows us to do normalisation-by-evaluation (in one direction), and program synthesis (in the other)[1]. Thus, if we start from the Toffoli gate on three bits:

```
toffoli : 2 × (2 × 2) Pi.↔₁ 2 × (2 × 2)
toffoli = controlled cnot
- dist ⊙ ((id↔₁ ⊗ (dist ⊙ ((id↔₁ ⊗ swap₊) ⊕ id↔₁) ⊙ factor)) ⊕ id↔₁) ⊙ factor
```

we can evaluate it to a permutation

```
toffoli-eval : Aut (Fin 8)
toffoli-eval = eval* toffoli
```

It is also possible to write out the permutation directly, extensionally:

```
toffoli-perm : Aut (Fin 8)
toffoli-perm = equiv f f f-f f-f
  where f : Fin 8 → Fin 8
        f (0 , ϕ) = 1
        f (1 , ϕ) = 0
        f (2 , ϕ) = 2
        f (3 , ϕ) = 3
        f (4 , ϕ) = 4
        f (5 , ϕ) = 5
        f (6 , ϕ) = 6
        f (7 , ϕ) = 7
  - elided
```

[1] The typography in this section is different from the rest of the paper, as the code is *directly* exported from Agda.

Using a lemma equiv = transforming paths in a function space to paths in the space of equivalences, we can verify that these permutations are the same

```
proof : toffoli-perm == toffoli-eval
proof = equiv= proof~
  where proof~ : (-> toffoli-perm)  (-> toffoli-eval)
        proof~ (0 , φ) = fin= idp
        proof~ (1 , φ) = fin= idp
        proof~ (2 , φ) = fin= idp
        proof~ (3 , φ) = fin= idp
        proof~ (4 , φ) = fin= idp
        proof~ (5 , φ) = fin= idp
        proof~ (6 , φ) = fin= idp
        proof~ (7 , φ) = fin= idp
        proof~ (_ , N.ltSR (N.ltSR (N.ltSR (N.ltSR (N.ltSR (N.ltSR (N.ltSR ()))))))))
```

Now, we can synthesise a normal form back to Π (technically, its additive fragment Π^+):

```
toffoli-norm : 8 Pi+.↔₁ 8
toffoli-norm = quote* toffoli-perm
- (assocl₊ ⊚ (swap₊ ⊕ id↔₁) ⊚ assocr₊) ⊚ id↔₁
```

And finally, ensure that the synthesised combinator is 2-equivalent to the one we started from!

```
toffoli-eq : _ Pi+.↔₂ _
toffoli-eq = Pi+.quote-eval₁ toffoli-norm
```

9.2. Transfer of theorems

In our development, we described a number of different representations of permutations: Π, Π^+ and $\Pi^\wedge$ programs, lists of transpositions, Lehmer codes, and automorphisms of finite sets. The equivalences between these representations allow us to transfer theorems about permutations – we prove them on the most suitable representation, and then transport them to a different representation "for free".

As an example, consider the following problem. The Toffoli gate on 3 bits can be extended to a reversible gate on 4 bits, by inserting a wire in four possible positions. This gate performs identity on the newly inserted bit, and controlled-controlled not on the three remaining ones.

$$\mathsf{toffoli}_3^1, \mathsf{toffoli}_3^2, \mathsf{toffoli}_3^3, \mathsf{toffoli}_3^4 : \mathbf{2}^4 \leftrightarrow_1 \mathbf{2}^4$$
$$\mathsf{toffoli}_3^1 = \mathsf{controlled}(\mathsf{controlled}(\mathsf{swap}\star \otimes \mathsf{id}\leftrightarrow_1)\mathsf{id}\leftrightarrow_1)\mathsf{id}\leftrightarrow_1$$
$$\mathsf{toffoli}_3^2 = \mathsf{controlled}(\mathsf{controlled}(\mathsf{id}\leftrightarrow_1 \otimes \mathsf{swap}\star)\mathsf{id}\leftrightarrow_1)\mathsf{id}\leftrightarrow_1$$
$$\mathsf{toffoli}_3^3 = \mathsf{controlled}(\mathsf{controlled}\,\mathsf{id}\leftrightarrow_1(\mathsf{swap}\star \otimes \mathsf{id}\leftrightarrow_1))\mathsf{id}\leftrightarrow_1$$
$$\mathsf{toffoli}_3^4 = \mathsf{controlled}(\mathsf{controlled}\,\mathsf{id}\leftrightarrow_1(\mathsf{id}\leftrightarrow_1 \otimes \mathsf{swap}\star))\mathsf{id}\leftrightarrow_1$$

On the other hand, it is possible to construct a proper 4-bit Toffoli gate.

$$\mathsf{toffoli}_4 : \mathbf{2}^4 \leftrightarrow_1 \mathbf{2}^4$$
$$\mathsf{toffoli}_4 = \mathsf{controlled}(\mathsf{controlled}(\mathsf{controlled}\,\mathsf{swap}\star))$$

Now, we can ask: is it possible to implement $\mathsf{toffoli}_4$ gate using only $\mathsf{toffoli}_3$ gates?

Solving this problem in the Π language representation is difficult. However, we can observe that all $\mathsf{toffoli}_3$ gates implement even permutations, that is, permutations that contain

an even number of transpositions. On the other hand, $\mathsf{toffoli}_4$ implements an odd permutation. It can be easily proved that any composition of even permutations always produces an even permutation, which answers the question to the negative. Thus, we formulate the theorem using the representation of lists of transpositions, and transfer it to circuits using the equivalences proven in the previous sections.

We define a function $\mathsf{parity} : (X \leftrightarrow Y) \to \mathbf{2}$ to compute the parity of a permutation defined by a combinator c, by first transforming a Π program into a list of transpositions, and then computing the parity of this representation, which amounts to a simple checking of the parity of the length of the list.

Proposition 9.2.1. *For two Π circuits c_1, c_2, if $c_1 \Leftrightarrow c_2$, then* $\mathsf{parity}(c_1) = \mathsf{parity}(c_2)$.

Proof. First, it is easy to check that Coxeter relations preserve parity of the list. Second, if c_1 and c_2 are related by a 2-combinator, then their Lehmer codes are the same, so their list-of-transpositions representations have to be related by the Coxeter relation. Composing these two facts completes the proof. □

Using Proposition 9.2.1, we complete the proof by computing the parity of $\mathsf{toffoli}_4$, and all combinations of the $\mathsf{toffoli}_3$ gates.

9.3. Word of caution

One observation I have, after writing a large part of the code base for this proof, is that working with a proof assistant *feels* quite different from either writing a proof by hand, on paper, and from normal programming. Agda is based on the interactive exploration paradigm - users write parts of the code, refine it, typecheck it, inspect definitions, and try out different routes, all with the help of typechecker. In this regard, it is different even from other proof assistants - Coq feels, for example, much more like a formal logic system, with the proofs derivations using tactics and goals in the natural-deduction style. Writing in Agda, is, in a way, directly experiencing Curry-Howard correspondence!

On the other hand, even having a formal proof does not guarantee its correctness. One issue is potential problems with the soundness of the system (meaning that, sometimes, bugs are discovered in Agda kernel allowing unsound reasoning to pass through the typechecker). This, however, happens much less frequently from another, which is *incorrect specification*. In particular, having already developed a large part of the proof, and experimenting on real logical circuits, we have discovered that it sometimes outputs wrong answers! As it happened, we missed one important condition in the interpreter. The main theorem statements were correct, but the "outer interface" was not. In this sense, I feel that the bigger conversation of replacing tests with formal proofs of programs' correctness is not really feasible - note, that although still quite large, our proof is both much more well-specified, and insignificant in size, compared to systems such as a C compiler [Leroy et al., 2016] or an OS kernel [Klein et al., 2009].

Chapter 10

Discussion

In this thesis, we have introduced a reversible programming language Π, described the technical background of Category Theory, Homotopy Type Theory, outlined how a formal construction of Π semantics can be built in HoTT, and explained (by de-formalization of Agda code) a fragment of the proof. The proof proceeded by modifying a known Coxeter relation used to classically define $\mathcal{S}_n$ to a form that allowed it to be used as a confluent rewriting system. Then, the normal forms of this system were described using an encoding known as Lehmer codes, which are easily translated to an extensional form of permutations.

Π	$\mathcal{S}_n$	$\mathcal{U}_{\mathsf{Fin}}$
types	Natural numbers	Finite sets
1-combinators	Generators of $\mathcal{S}_n$	1-paths
2-combinators	Relations of $\mathcal{S}_n$	2-paths

10.1. Curry-Howard-Lambek correspondence revisited

In the original formulation of Curry-Howard-Lambek correspondence we presented in Chapter 3, logic had taken a somewhat secondary role. It was, in a sense, reduced to a type part of λ-calculus - types only telling us what *is* true, putting aside the *reason* it was true. As it was subsumed by the programming, the interesting part of the correspondence really lied in the interplay between category theory (algebra) and lambda calculus (syntactic theory).

The same thing is true in the reversible setting. we can develop a theory of *reversible logic*, which reduces to the type-level programming in Π. This was done in Sparks and Sabry [2014], but fallen out of the scope of this thesis.

Still, we can summarise our proof of the correspondence between Π and the groupoid of finite types by the following table, hoping to convey the inspiration by, and the connection to, the original Curry-Howard-Lambek correspondence, in the Figure 10.1.

10.2. $\mathcal{U}_{\mathsf{Fin}}$ and FinBij

The universe of finite sets $\mathcal{U}_{\mathsf{Fin}}$ is not, in its essence, the sub-category of finite sets

$$\mathbf{FinBij} = \mathbf{Core}(\mathbf{Set}_{\mathsf{Fin}})$$

but rather some mysterious object defined by a particular model of HoTT. We did not spend much time on its relationship to **FinBij**, being satisfied with its behaviour inside the theory.

Reversible Logic	Reversible Programming	Algebra
Free commutative rig	Free symmetric rig groupoid	$\mathcal{U}_{\mathsf{Fin}}$
Propositions	Types	Objects
Entailment	Reversible programs	Isomorphisms
-	2-combinators	Homotopies
Conjunction	$\times$	$\otimes$
Disjunction	$+$	$\oplus$
k $A \wedge B \vdash B \wedge A$	swap⋆	$\mathcal{B}_{A,B}$
True proposition	$\mathbf{1}$	F_1
Decidability	Program Synthesis	Reachability in a graph
Connectives	Type constructors	Bifunctors
Propositions	Types	Objects
Proofs	Terms	Morphisms
Soundness	Soundness of 2-combinators	-
Completeness	Completeness of 2-combinators	Coherence

Figure 10.1: Reversible trinitarianism

However, even though, in the standard model of HoTT in simplicial sets [Kapulkin et al., 2012] the nature of $\mathcal{U}_{\mathsf{Fin}}$ is difficult to assert, this is not the case in the simpler models. By considering the 1-groupoid model of MLTT given by [Hofmann and Streicher, 1996], we can see that the finite sets are *actually* finite, discrete groupoids, and thus functors between them correspond exactly to hom-sets in **FinBij**.

10.3. FSMG

We have described the *free symmetric monoidal groupoid* using the syntax of Π - or rather its corresponding syntactic category. One objection might be that it is 'free' only as much as it is 'freely generated', and not as a 'free algebraic object'. It is true that the proof does not really show this. With some effort, it could be extended, and we would be able to properly derive it - the biggest change in formalisation would be in the definition of Π, where instead of specifying its three levels of combinators separately, we would use one HIT for all the levels. We did not make this decision for technical reasons - we wanted the theorem to be proven in Homotopy Type Theory proper, and not any of its variants (like Cubical Type Theory [Cohen et al., 2018]) for it to be as general as possible, and the proof assistants did not have a proper support for the HITs at the time of developing it (which they do have now). However, the theorem about the $\mathbf{\Pi}_{\mathrm{cat}}$ being 'algebraically free' *is* true, and was proven independently (in a slightly different setting) by Elgueta [2021].

10.4. Future work

One interesting direction of future work would be to study empirically whether a search over 2-combinators provides an efficient tautology solver, or one that is complementary to the currently used ones. This would require setting up an appropriate benchmark for the comparison - in particular, identifying a common representation of the input, and implementing the algorithm in an optimised way.

In the same vein, it would be interesting to see whether the Corollary 8.1.8 can be extended to a if-and-only-if statement. This would make it possible to empirically check co-NP $\subset$ NP for small circuit sizes, and potentially be a source of interesting counterexamples.

Another direction would be to extend base Π to a language with richer types - the most interesting (but possibly also quite difficult) would be to introduce infinite types and some form of fixed-point operator, using trace categories.

Yet another question is whether the connection between univalence axiom (studied here in the context of univalent subuniverse) and denotational semantics extends to other (non-reversible, richer) programming languages. This could possibly also affect understanding of compiler optimization techniques based on extensional equality of functions.

Finally, moving the formalisation to the setting of Cubical Type Theory and making the Π a into a proper HIT would formally establish the algebraic freeness of $\mathbf{\Pi}_{\mathrm{cat}}$.

10.5. Note on tools

This thesis was typeset with LaTeX, using `overleaf.com` online editor. I used bussproofs package to typeset derivation trees, braids package to draw braiding diagrams, TikZ package and `https://q.uiver.app/` editor to draw categorical diagrams and `https://www.mathcha.io/` editor to draw Figure 4.1.

The code repository has been managed with git, and kept on `github.com`. Code had been developed in Agda proof assistant, using Agda-HoTT library `https://github.com/HoTT/HoTT-Agda`. It has been edited with vscode (using agda-mode plugin `https://github.com/banacorn/agda-mode-vscode`) and emacs (using agda-mode `https://agda.readthedocs.io/en/latest/tools/emacs-mode.html`).

Bibliography

S. Aaronson. P?= NP, 2017. URL https://www.scottaaronson.com/papers/pnp.pdf.

B. Ahrens, D. Frumin, M. Maggesi, N. Veltri, and N. van der Weide. Bicategories in Univalent Foundations. *Mathematical Structures in Computer Science*, pages 1–38, Mar. 2022. ISSN 0960-1295, 1469-8072. doi: 10.1017/S0960129522000032. URL http://arxiv.org/abs/1903.01152. arXiv:1903.01152 [cs, math].

J. C. Baez. Some Definitions Everyone Should Know, July 2004. URL https://math.ucr.edu/home/baez/qg-fall2004/definitions.pdf.

A. Blum. CMU 15-451 (Algorithms). *Lecture notes*, June 2007. URL https://www.cs.cmu.edu/~avrim/451f07/lectures/lect1106.pdf.

A. Bucciarelli. Proofs, programs: classic tools, Nov. 2022. URL https://www.irif.fr/~buccia/COURS/PPOC/SEANCES/s01.pdf.

J. Carette and A. Sabry. Computing with Semirings and Weak Rig Groupoids. In P. Thiemann, editor, *Programming Languages and Systems*, volume 9632, pages 123–148. Springer Berlin Heidelberg, Berlin, Heidelberg, 2016. ISBN 978-3-662-49497-4 978-3-662-49498-1. doi: 10.1007/978-3-662-49498-1_6. URL http://link.springer.com/10.1007/978-3-662-49498-1_6. ZSCC: NoCitationData[s0] Series Title: Lecture Notes in Computer Science.

V. Choudhury and J. Karwowski. Artifact for Symmetries in Reversible Programming, Oct. 2021. URL https://zenodo.org/record/5671746.

V. Choudhury, J. Karwowski, and A. Sabry. Symmetries in reversible programming: from symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–32, Jan. 2022. ISSN 2475-1421. doi: 10.1145/3498667. URL https://dl.acm.org/doi/10.1145/3498667.

C. Cohen, T. Coquand, S. Huber, and A. Mörtberg. Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom. page 34 pages, 2018. doi: 10.4230/LIPICS.TYPES.2015.5. URL http://drops.dagstuhl.de/opus/volltexte/2018/8475/. ZSCC: NoCitationData[s1] Artwork Size: 34 pages Medium: application/pdf Publisher: Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik GmbH, Wadern/Saarbruecken, Germany.

C. Dubois and A. Giorgetti. Tests and proofs for custom data generators. *Formal Aspects of Computing*, 30(6):659–684, Nov. 2018. ISSN 0934-5043, 1433-299X. doi: 10.1007/s00165-018-0459-1. URL http://link.springer.com/10.1007/s00165-018-0459-1. ZSCC: 0000004.

J. Elgueta. The groupoid of finite sets is biinitial in the 2-category of rig categories. *Journal of Pure and Applied Algebra*, 225(11):106738, Nov. 2021. ISSN 0022-4049. doi: 10.1016/j.jpaa.2021.106738. URL https://www.sciencedirect.com/science/article/pii/S0022404921000785. ZSCC: 0000001.

M. H. Escardó. A self-contained, brief and complete formulation of Voevodsky's Univalence Axiom. Technical Report arXiv:1803.02294, arXiv, Oct. 2018. URL http://arxiv.org/abs/1803.02294. arXiv:1803.02294 [math] type: article.

M. Hofmann and T. Streicher. The Groupoid Interpretation of Type Theory. In *In Venice Festschrift*, pages 83–111. Oxford University Press, 1996.

S. P. Jordan. Strong equivalence of reversible circuits is coNP-complete. July 2013. URL https://www.nist.gov/publications/strong-equivalence-reversible-circuits-conp-complete. Last Modified: 2017-02-19T20:02-05:00.

C. Kapulkin, P. L. Lumsdaine, and V. Voevodsky. Univalence in Simplicial Sets. Mar. 2012. URL https://arxiv.org/abs/1203.2553v1.

G. M. Kelly. Coherence theorems for lax algebras and for distributive laws. In G. M. Kelly, editor, *Category Seminar*, Lecture Notes in Mathematics, pages 281–375, Berlin, Heidelberg, 1974. Springer. ISBN 978-3-540-37270-7. doi: 10.1007/BFb0063106.

A. D. Ker. Lambda Calculus and Types. *Lecture notes*, 2009. URL https://www.cs.ox.ac.uk/andrew.ker/docs/lambdacalculus-lecture-notes-ht2009.pdf.

G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, SOSP '09, pages 207–220, New York, NY, USA, Oct. 2009. Association for Computing Machinery. ISBN 978-1-60558-752-3. doi: 10.1145/1629575.1629596. URL https://doi.org/10.1145/1629575.1629596.

M. L. Laplaza. Coherence for distributivity. In G. M. Kelly, M. Laplaza, G. Lewis, and S. Mac Lane, editors, *Coherence in Categories*, Lecture Notes in Mathematics, pages 29–65, Berlin, Heidelberg, 1972. Springer. ISBN 978-3-540-37958-4. doi: 10.1007/BFb0059555.

D. H. Lehmer. Teaching combinatorial tricks to a computer. In R. Bellman and M. Hall, editors, *Proceedings of Symposia in Applied Mathematics*, volume 10, pages 179–193. American Mathematical Society, Providence, Rhode Island, 1960. ISBN 978-0-8218-1310-2 978-0-8218-9225-1. doi: 10.1090/psapm/010/0113289. URL https://doi.org/10.1090/psapm/010. ZSCC: NoCitationData[s2].

X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. CompCert - A Formally Verified Optimizing Compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, Toulouse, France, Jan. 2016. SEE. URL https://hal.inria.fr/hal-01238879.

S. MacLane. Natural Associativity and Commutativity. *Rice Institute Pamphlet - Rice University Studies*, 49(4), Oct. 1963. URL https://scholarship.rice.edu/handle/1911/62865. ZSCC: 0000723 Accepted: 2011-11-08T19:13:47Z Publisher: Rice University.

D. Maslov and G. Dueck. Reversible cascades with minimal garbage. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 23(11):1497–1509, Nov. 2004. ISSN 1937-4151. doi: 10.1109/TCAD.2004.836735. Conference Name: IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems.

M. Molzer. Cubical Agda: Simple application of Fin: Lehmer codes, Nov. 2021. URL `https://github.com/agda/cubical/blob/a1d2bb38c0794f3cb00610cd6061cf9b5410518d/Cubical/Data/Fin/LehmerCode.agda`. original-date: 2018-10-15T09:28:28Z.

A. Pitts. Brief Notes on the Category Theoretic Semantics of Simply Typed Lambda Calculus, 2016. URL `https://www.semanticscholar.org/paper/Brief-Notes-on-the-Category-Theoretic-Semantics-of-Pitts/344665f3e0ade6e0d039fb435bbc3e61feb8938f`.

E. Rijke. Introduction to homotopy type theory. *Lecture notes*, 2018.

A. Sabry, J. Carette, zsparks, C.-H. Chen, V. Choudhury, R. James, and movieverse. JacquesCarette/pi-dual: Second alpha release, Oct. 2021. URL `https://zenodo.org/record/5620828`.

Z. Sparks and A. Sabry. Superstructural Reversible Logic. page 12, 2014. URL `https://legacy.cs.indiana.edu/~sabry/papers/reversible-logic.pdf`. ZSCC: 0000007.

T. Toffoli. Reversible computing. In J. de Bakker and J. van Leeuwen, editors, *Automata, Languages and Programming*, Lecture Notes in Computer Science, pages 632–644, Berlin, Heidelberg, 1980. Springer. ISBN 978-3-540-39346-7. doi: 10.1007/3-540-10003-2_104.

T. Univalent Foundations Program. *Homotopy type theory: Univalent foundations of mathematics*. Univalent Foundations Program, Institute for Advanced Study, 2013. URL `https://homotopytypetheory.org/book`. ZSCC: 0000005.

P. Urzyczyn. Rachunek lambda - skrypt do wykładu, Apr. 2022. URL `https://www.mimuw.edu.pl/~urzy/Lambda/erlambda.pdf`.

V. Vajnovszki. A new Euler–Mahonian constructive bijection. *Discrete Applied Mathematics*, 159(14):1453–1459, Aug. 2011. ISSN 0166-218X. doi: 10.1016/j.dam.2011.05.012. URL `http://www.sciencedirect.com/science/article/pii/S0166218X11001946`. ZSCC: 0000011.

Appendix A

Full syntax of Π

We formalize the syntax of Π in Agda. This part of the formalization uses HoTT only as much as to define the type universe U_0 (written in the code as Type_0).

Types

```
- Types
data U : Type₀ where
  O : U
  I : U
  _+_ : U → U → U
  _×_ : U → U → U
```

1-combinators

Note that not all 1-combinators are primitives - some of them are defined using others. Another thing to note is that we define the negation operator ! $\leftrightarrow_1$, which we then use for convenience in the 2-combinators definitions. We omit its full definition here (it is available in the code attachment), but the important thing to note is that it is definitionally expanded, and does not cause problems with 2-combinators specification.

```
- 1-combinators
data _↔₁_ : U → U → Type₀ where
  unite₊l : O + t ↔₁ t
  uniti₊l : t ↔₁ O + t
  unite⋆l : I × t ↔₁ t
  uniti⋆l : t ↔₁ I × t
  swap₊ : r + s ↔₁ s + r
  swap⋆ : r × s ↔₁ s × r
  assocl₊ : r + (s + p) ↔₁ (r + s) + p
  assocr₊ : (r + s) + p ↔₁ r + (s + p)
  assocl⋆ : r × (s × p) ↔₁ (r × s) × p
  assocr⋆ : (r × s) × p ↔₁ r × (s × p)
  absorbr : O × t ↔₁ O
  absorbl : t × O ↔₁ O
  factorzr : O ↔₁ t × O
  factorzl : O ↔₁ O × t
  dist : (r + s) × p ↔₁ (r × p) + (s × p)
  distl : p × (r + s) ↔₁ (p × r) + (p × s)
  factor : {r s p : U} → (r × p) + (s × p) ↔₁ (r + s) × p
```

```
  factorl : {r s p : U} → (p × r) + (p × s) ↔₁ p × (r + s)
  id↔₁ : t ↔₁ t
  _⊚_  : (r ↔₁  s) → (s ↔₁ p) → (r ↔₁ p)
  _⊕_  : (r ↔₁  p) → (s ↔₁ q) → (r + s ↔₁ p + q)
  _⊗_  : (r ↔₁  p) → (s ↔₁ q) → (r × s ↔₁ p × q)

- Coherence
unite₊r : {t : U} → t + O ↔₁ t
unite₊r = swap₊ ⊚ unite₊l

uniti₊r : t ↔₁  t + O
uniti₊r = uniti₊l ⊚ swap₊

unite⋆r : {t : U} → t × I ↔₁  t
unite⋆r = swap⋆ ⊚ unite⋆l

uniti⋆r : t ↔₁  t × I
uniti⋆r = uniti⋆l ⊚ swap⋆

- Negation
!↔₁ : r ↔₁ s → s ↔₁ r
```

2-combinators

Full list of 2-combinators is quite long and involved. We split it into different sections, corresponding to various coherence conditions.

```
- 2-combinators
data _↔₂_ : {r s : U} → (r ↔₁ s) → (r ↔₁ s) → Set where
  assoc⊚l : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : p ↔₁ q} →
            (c₁ ⊚ (c₂ ⊚ c₃)) ↔₂ ((c₁ ⊚ c₂) ⊚ c₃)
  assoc⊚r : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : p ↔₁ q} →
            ((c₁ ⊚ c₂) ⊚ c₃) ↔₂ (c₁ ⊚ (c₂ ⊚ c₃))
  assocl⊕l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             ((c₁ ⊕ (c₂ ⊕ c₃)) ⊚ assocl₊) ↔₂ (assocl₊ ⊚ ((c₁ ⊕ c₂) ⊕ c₃))
  assocl⊕r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (assocl₊ ⊚ ((c₁ ⊕ c₂) ⊕ c₃)) ↔₂ ((c₁ ⊕ (c₂ ⊕ c₃)) ⊚ assocl₊)
  assocl⊗l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             ((c₁ ⊗ (c₂ ⊗ c₃)) ⊚ assocl⋆) ↔₂ (assocl⋆ ⊚ ((c₁ ⊗ c₂) ⊗ c₃))
  assocl⊗r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (assocl⋆ ⊚ ((c₁ ⊗ c₂) ⊗ c₃)) ↔₂ ((c₁ ⊗ (c₂ ⊗ c₃)) ⊚ assocl⋆)
  assocr⊕r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (((c₁ ⊕ c₂) ⊕ c₃) ⊚ assocr₊) ↔₂ (assocr₊ ⊚ (c₁ ⊕ (c₂ ⊕ c₃)))
  assocr⊗l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
               (assocr⋆ ⊚ (c₁ ⊗ (c₂ ⊗ c₃))) ↔₂ (((c₁ ⊗ c₂) ⊗ c₃) ⊚ assocr⋆)
  assocr⊗r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (((c₁ ⊗ c₂) ⊗ c₃) ⊚ assocr⋆) ↔₂ (assocr⋆ ⊚ (c₁ ⊗ (c₂ ⊗ c₃)))
  assocr⊕l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
               (assocr₊ ⊚ (c₁ ⊕ (c₂ ⊕ c₃))) ↔₂ (((c₁ ⊕ c₂) ⊕ c₃) ⊚ assocr₊)
  dist↔₂l : {r s p q v w : U}
```

$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$((a \oplus b) \otimes c) \circledcirc \mathsf{dist} \leftrightarrow_2 \mathsf{dist} \circledcirc ((a \otimes c) \oplus (b \otimes c))$

$\mathsf{dist}{\leftrightarrow_2}\mathsf{r} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$\mathsf{dist} \circledcirc ((a \otimes c) \oplus (b \otimes c)) \leftrightarrow_2 ((a \oplus b) \otimes c) \circledcirc \mathsf{dist}$

$\mathsf{distl}{\leftrightarrow_2}\mathsf{l} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$(a \otimes (b \oplus c)) \circledcirc \mathsf{distl} \leftrightarrow_2 \mathsf{distl} \circledcirc ((a \otimes b) \oplus (a \otimes c))$

$\mathsf{distl}{\leftrightarrow_2}\mathsf{r} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$\mathsf{distl} \circledcirc ((a \otimes b) \oplus (a \otimes c)) \leftrightarrow_2 (a \otimes (b \oplus c)) \circledcirc \mathsf{distl}$

$\mathsf{factor}{\leftrightarrow_2}\mathsf{l} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$((a \otimes c) \oplus (b \otimes c)) \circledcirc \mathsf{factor} \leftrightarrow_2 \mathsf{factor} \circledcirc ((a \oplus b) \otimes c)$

$\mathsf{factor}{\leftrightarrow_2}\mathsf{r} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$\mathsf{factor} \circledcirc ((a \oplus b) \otimes c) \leftrightarrow_2 ((a \otimes c) \oplus (b \otimes c)) \circledcirc \mathsf{factor}$

$\mathsf{factorl}{\leftrightarrow_2}\mathsf{l} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$((a \otimes b) \oplus (a \otimes c)) \circledcirc \mathsf{factorl} \leftrightarrow_2 \mathsf{factorl} \circledcirc (a \otimes (b \oplus c))$

$\mathsf{factorl}{\leftrightarrow_2}\mathsf{r} : \{r\ s\ p\ q\ v\ w : \mathsf{U}\}$
$\{a : r \leftrightarrow_1 s\}\ \{b : p \leftrightarrow_1 q\}\ \{c : v \leftrightarrow_1 w\} \rightarrow$
$\mathsf{factorl} \circledcirc (a \otimes (b \oplus c)) \leftrightarrow_2 ((a \otimes b) \oplus (a \otimes c)) \circledcirc \mathsf{factorl}$

$\mathsf{idl}\circledcirc\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow (\mathsf{id}{\leftrightarrow_1} \circledcirc c) \leftrightarrow_2 c$

$\mathsf{idl}\circledcirc\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow c \leftrightarrow_2 \mathsf{id}{\leftrightarrow_1} \circledcirc c$

$\mathsf{idr}\circledcirc\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow (c \circledcirc \mathsf{id}{\leftrightarrow_1}) \leftrightarrow_2 c$

$\mathsf{idr}\circledcirc\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow c \leftrightarrow_2 (c \circledcirc \mathsf{id}{\leftrightarrow_1})$

$\mathsf{linv}\circledcirc\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow (c \circledcirc !{\leftrightarrow_1}\ c) \leftrightarrow_2 \mathsf{id}{\leftrightarrow_1}$

$\mathsf{linv}\circledcirc\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow \mathsf{id}{\leftrightarrow_1} \leftrightarrow_2 (c \circledcirc !{\leftrightarrow_1}\ c)$

$\mathsf{rinv}\circledcirc\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow (!{\leftrightarrow_1}\ c \circledcirc c) \leftrightarrow_2 \mathsf{id}{\leftrightarrow_1}$

$\mathsf{rinv}\circledcirc\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c : r \leftrightarrow_1 s\} \rightarrow \mathsf{id}{\leftrightarrow_1} \leftrightarrow_2 (!{\leftrightarrow_1}\ c \circledcirc c)$

$\mathsf{unite}_+\mathsf{l}{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{unite}_+\mathsf{l} \circledcirc c_2) \leftrightarrow_2 ((c_1 \oplus c_2) \circledcirc \mathsf{unite}_+\mathsf{l})$

$\mathsf{unite}_+\mathsf{l}{\leftrightarrow_2}\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$((c_1 \oplus c_2) \circledcirc \mathsf{unite}_+\mathsf{l}) \leftrightarrow_2 (\mathsf{unite}_+\mathsf{l} \circledcirc c_2)$

$\mathsf{uniti}_+\mathsf{l}{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{uniti}_+\mathsf{l} \circledcirc (c_1 \oplus c_2)) \leftrightarrow_2 (c_2 \circledcirc \mathsf{uniti}_+\mathsf{l})$

$\mathsf{uniti}_+\mathsf{l}{\leftrightarrow_2}\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(c_2 \circledcirc \mathsf{uniti}_+\mathsf{l}) \leftrightarrow_2 (\mathsf{uniti}_+\mathsf{l} \circledcirc (c_1 \oplus c_2))$

$\mathsf{unite}_+\mathsf{r}{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{unite}_+\mathsf{r} \circledcirc c_2) \leftrightarrow_2 ((c_2 \oplus c_1) \circledcirc \mathsf{unite}_+\mathsf{r})$

$\mathsf{unite}_+\mathsf{r}{\leftrightarrow_2}\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$((c_2 \oplus c_1) \circledcirc \mathsf{unite}_+\mathsf{r}) \leftrightarrow_2 (\mathsf{unite}_+\mathsf{r} \circledcirc c_2)$

$\mathsf{uniti}_+\mathsf{r}{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{uniti}_+\mathsf{r} \circledcirc (c_2 \oplus c_1)) \leftrightarrow_2 (c_2 \circledcirc \mathsf{uniti}_+\mathsf{r})$

$\mathsf{uniti}_+\mathsf{r}{\leftrightarrow_2}\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{O} \leftrightarrow_1 \mathsf{O}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(c_2 \circledcirc \mathsf{uniti}_+\mathsf{r}) \leftrightarrow_2 (\mathsf{uniti}_+\mathsf{r} \circledcirc (c_2 \oplus c_1))$

$\mathsf{swapl}_+{\leftrightarrow_2} : \{r\ s\ p\ q : \mathsf{U}\}\ \{c_1 : r \leftrightarrow_1 s\}\ \{c_2 : p \leftrightarrow_1 q\} \rightarrow$
$(\mathsf{swap}_+ \circledcirc (c_1 \oplus c_2)) \leftrightarrow_2 ((c_2 \oplus c_1) \circledcirc \mathsf{swap}_+)$

$\mathsf{swapr}_+{\leftrightarrow_2} : \{r\ s\ p\ q : \mathsf{U}\}\ \{c_1 : r \leftrightarrow_1 s\}\ \{c_2 : p \leftrightarrow_1 q\} \rightarrow$
$((c_2 \oplus c_1) \circledcirc \mathsf{swap}_+) \leftrightarrow_2 (\mathsf{swap}_+ \circledcirc (c_1 \oplus c_2))$

$\mathsf{unitel}\star{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{I} \leftrightarrow_1 \mathsf{I}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{unite}\star\mathsf{l} \circledcirc c_2) \leftrightarrow_2 ((c_1 \otimes c_2) \circledcirc \mathsf{unite}\star\mathsf{l})$

$\mathsf{uniter}\star{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{I} \leftrightarrow_1 \mathsf{I}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$((c_1 \otimes c_2) \circledcirc \mathsf{unite}\star\mathsf{l}) \leftrightarrow_2 (\mathsf{unite}\star\mathsf{l} \circledcirc c_2)$

$\mathsf{unitil}\star{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{I} \leftrightarrow_1 \mathsf{I}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(\mathsf{uniti}\star\mathsf{l} \circledcirc (c_1 \otimes c_2)) \leftrightarrow_2 (c_2 \circledcirc \mathsf{uniti}\star\mathsf{l})$

$\mathsf{unitir}\star{\leftrightarrow_2}\mathsf{l} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{I} \leftrightarrow_1 \mathsf{I}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$
$(c_2 \circledcirc \mathsf{uniti}\star\mathsf{l}) \leftrightarrow_2 (\mathsf{uniti}\star\mathsf{l} \circledcirc (c_1 \otimes c_2))$

$\mathsf{unitel}\star{\leftrightarrow_2}\mathsf{r} : \{r\ s : \mathsf{U}\}\ \{c_1 : \mathsf{I} \leftrightarrow_1 \mathsf{I}\}\ \{c_2 : r \leftrightarrow_1 s\} \rightarrow$

```
        (unite⋆r ◎ c₂) ↔₂ ((c₂ ⊗ c₁) ◎ unite⋆r)
  uniter⋆↔₂r : {r s : U} {c₁ : I ↔₁ I} {c₂ : r ↔₁ s} →
        ((c₂ ⊗ c₁) ◎ unite⋆r) ↔₂ (unite⋆r ◎ c₂)
  unitil⋆↔₂r : {r s : U} {c₁ : I ↔₁ I} {c₂ : r ↔₁ s} →
        (uniti⋆r ◎ (c₂ ⊗ c₁)) ↔₂ (c₂ ◎ uniti⋆r)
  unitir⋆↔₂r : {r s : U} {c₁ : I ↔₁ I} {c₂ : r ↔₁ s} →
        (c₂ ◎ uniti⋆r) ↔₂ (uniti⋆r ◎ (c₂ ⊗ c₁))
  swapl⋆↔₂ : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} →
        (swap⋆ ◎ (c₁ ⊗ c₂)) ↔₂ ((c₂ ⊗ c₁) ◎ swap⋆)
  swapr⋆↔₂ : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} →
        ((c₂ ⊗ c₁) ◎ swap⋆) ↔₂ (swap⋆ ◎ (c₁ ⊗ c₂))
  id↔₂      : {r s : U} {c : r ↔₁ s} → c ↔₂ c
  trans↔₂   : {r s : U} {c₁ c₂ c₃ : r ↔₁ s} →
        (c₁ ↔₂ c₂) → (c₂ ↔₂ c₃) → (c₁ ↔₂ c₃)
  _⊡_ : {r s p : U}
        {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : r ↔₁ s} {c₄ : s ↔₁ p} →
        (c₁ ↔₂ c₃) → (c₂ ↔₂ c₄) → (c₁ ◎ c₂) ↔₂ (c₃ ◎ c₄)
  resp⊕↔₂  : {r s p q : U}
        {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : r ↔₁ s} {c₄ : p ↔₁ q} →
        (c₁ ↔₂ c₃) → (c₂ ↔₂ c₄) → (c₁ ⊕ c₂) ↔₂ (c₃ ⊕ c₄)
  resp⊗↔₂  : {r s p q : U}
        {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : r ↔₁ s} {c₄ : p ↔₁ q} →
        (c₁ ↔₂ c₃) → (c₂ ↔₂ c₄) → (c₁ ⊗ c₂) ↔₂ (c₃ ⊗ c₄)
- - below are the combinators added for the RigCategory structure
  id↔⊕id↔↔₂ : {r s : U} → (id↔₁ {r} ⊕ id↔₁ {s}) ↔₂ id↔₁
  split⊕-id↔ : {r s : U} → (id↔₁ {_+_ r s}) ↔₂ (id↔₁ ⊕ id↔₁)
  hom⊕◎↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
    ((c₁ ◎ c₃) ⊕ (c₂ ◎ c₄)) ↔₂ ((c₁ ⊕ c₂) ◎ (c₃ ⊕ c₄))
  hom◎⊕↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
        ((c₁ ⊕ c₂) ◎ (c₃ ⊕ c₄)) ↔₂ ((c₁ ◎ c₃) ⊕ (c₂ ◎ c₄))
  id↔⊗id↔↔₂ : {r s : U} → (id↔₁ {r} ⊗ id↔₁ {s}) ↔₂ id↔₁
  split⊗-id↔ : {r s : U} → (id↔₁ {_×_ r s}) ↔₂ (id↔₁ ⊗ id↔₁)
  hom⊗◎↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
    ((c₁ ◎ c₃) ⊗ (c₂ ◎ c₄)) ↔₂ ((c₁ ⊗ c₂) ◎ (c₃ ⊗ c₄))
  hom◎⊗↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
        ((c₁ ⊗ c₂) ◎ (c₃ ⊗ c₄)) ↔₂ ((c₁ ◎ c₃) ⊗ (c₂ ◎ c₄))
- - associativity triangle
  triangle⊕l : {r s : U} →
    (unite₊r {r} ⊕ id↔₁ {s}) ↔₂ assocr₊ ◎ (id↔₁ ⊕ unite₊l)
  triangle⊕r : {r s : U} →
    assocr₊ ◎ (id↔₁ {r} ⊕ unite₊l {s}) ↔₂ (unite₊r ⊕ id↔₁)
  triangle⊗l : {r s : U} →
    ((unite⋆r {r}) ⊗ id↔₁ {s}) ↔₂ assocr⋆ ◎ (id↔₁ ⊗ unite⋆l)
  triangle⊗r : {r s : U} →
    (assocr⋆ ◎ (id↔₁ {r} ⊗ unite⋆l {s})) ↔₂ (unite⋆r ⊗ id↔₁)
  pentagon⊕l : {r s p q : U} →
    assocr₊ ◎ (assocr₊ {r} {s} {_+_ p q}) ↔₂
    ((assocr₊ ⊕ id↔₁) ◎ assocr₊) ◎ (id↔₁ ⊕ assocr₊)
  pentagon⊕r : {r s p q : U} →
    ((assocr₊ {r} {s} {p} ⊕ id↔₁ {q}) ◎ assocr₊) ◎ (id↔₁ ⊕ assocr₊) ↔₂
    assocr₊ ◎ assocr₊
  pentagon⊗l : {r s p q : U} →
    assocr⋆ ◎ (assocr⋆ {r} {s} {_×_ p q}) ↔₂
    ((assocr⋆ ⊗ id↔₁) ◎ assocr⋆) ◎ (id↔₁ ⊗ assocr⋆)
  pentagon⊗r : {r s p q : U} →
    ((assocr⋆ {r} {s} {p} ⊗ id↔₁ {q}) ◎ assocr⋆) ◎ (id↔₁ ⊗ assocr⋆) ↔₂
```

```
    assocr⋆ ◎ assocr⋆
- - from the braiding
- - unit coherence
  unite₊l-coh-l : {r : U} → unite₊l {r} ↔₂ swap₊ ◎ unite₊r
  unite₊l-coh-r : {r : U} → swap₊ ◎ unite₊r ↔₂ unite₊l {r}
  unite⋆l-coh-l : {r : U} → unite⋆l {r} ↔₂ swap⋆ ◎ unite⋆r
  unite⋆l-coh-r : {r : U} → swap⋆ ◎ unite⋆r ↔₂ unite⋆l {r}
  hexagonr⊕l : {r s p : U} →
    (assocr₊ ◎ swap₊) ◎ assocr₊ {r} {s} {p} ↔₂
    ((swap₊ ⊕ id↔₁) ◎ assocr₊) ◎ (id↔₁ ⊕ swap₊)
  hexagonr⊕r : {r s p : U} →
    ((swap₊ ⊕ id↔₁) ◎ assocr₊) ◎ (id↔₁ ⊕ swap₊) ↔₂
    (assocr₊ ◎ swap₊) ◎ assocr₊ {r} {s} {p}
  hexagonl⊕l : {r s p : U} →
    (assocl₊ ◎ swap₊) ◎ assocl₊ {r} {s} {p} ↔₂
    ((id↔₁ ⊕ swap₊) ◎ assocl₊) ◎ (swap₊ ⊕ id↔₁)
  hexagonl⊕r : {r s p : U} →
    ((id↔₁ ⊕ swap₊) ◎ assocl₊) ◎ (swap₊ ⊕ id↔₁) ↔₂
    (assocl₊ ◎ swap₊) ◎ assocl₊ {r} {s} {p}
  hexagonr⊗l : {r s p : U} →
    (assocr⋆ ◎ swap⋆) ◎ assocr⋆ {r} {s} {p} ↔₂
    ((swap⋆ ⊗ id↔₁) ◎ assocr⋆) ◎ (id↔₁ ⊗ swap⋆)
  hexagonr⊗r : {r s p : U} →
    ((swap⋆ ⊗ id↔₁) ◎ assocr⋆) ◎ (id↔₁ ⊗ swap⋆) ↔₂
    (assocr⋆ ◎ swap⋆) ◎ assocr⋆ {r} {s} {p}
  hexagonl⊗l : {r s p : U} →
    (assocl⋆ ◎ swap⋆) ◎ assocl⋆ {r} {s} {p} ↔₂
    ((id↔₁ ⊗ swap⋆) ◎ assocl⋆) ◎ (swap⋆ ⊗ id↔₁)
  hexagonl⊗r : {r s p : U} →
    ((id↔₁ ⊗ swap⋆) ◎ assocl⋆) ◎ (swap⋆ ⊗ id↔₁) ↔₂
    (assocl⋆ ◎ swap⋆) ◎ assocl⋆ {r} {s} {p}
  absorbl↔₂l : {r s : U} {c₁ : r ↔₁ s} →
    (c₁ ⊗ id↔₁ {0}) ◎ absorbl ↔₂ absorbl ◎ id↔₁ {0}
  absorbl↔₂r : {r s : U} {c₁ : r ↔₁ s} →
    absorbl ◎ id↔₁ {0} ↔₂ (c₁ ⊗ id↔₁ {0}) ◎ absorbl
  absorbr↔₂l : {r s : U} {c₁ : r ↔₁ s} →
    (id↔₁ {0} ⊗ c₁) ◎ absorbr ↔₂ absorbr ◎ id↔₁ {0}
  absorbr↔₂r : {r s : U} {c₁ : r ↔₁ s} →
    absorbr ◎ id↔₁ {0} ↔₂ (id↔₁ {0} ⊗ c₁) ◎ absorbr
  factorzl↔₂l : {r s : U} {c₁ : r ↔₁ s} →
    id↔₁ ◎ factorzl ↔₂ factorzl ◎ (id↔₁ ⊗ c₁)
  factorzl↔₂r : {r s : U} {c₁ : r ↔₁ s} →
    factorzl ◎ (id↔₁ {0} ⊗ c₁) ↔₂ id↔₁ {0} ◎ factorzl
  factorzr↔₂l : {r s : U} {c₁ : r ↔₁ s} →
    id↔₁ ◎ factorzr ↔₂ factorzr ◎ (c₁ ⊗ id↔₁)
  factorzr↔₂r : {r s : U} {c₁ : r ↔₁ s} →
    factorzr ◎ (c₁ ⊗ id↔₁) ↔₂ id↔₁ ◎ factorzr
  - from the coherence conditions of RigCategory
  swap₊distl↔₂l : {r s p : U} →
    (id↔₁ {r} ⊗ swap₊ {s} {p}) ◎ distl ↔₂ distl ◎ swap₊
  swap₊distl↔₂r : {r s p : U} →
    distl ◎ swap₊ ↔₂ (id↔₁ {r} ⊗ swap₊ {s} {p}) ◎ distl
  dist-swap⋆↔₂l : {r s p : U} →
    dist {r} {s} {p} ◎ (swap⋆ ⊕ swap⋆) ↔₂ swap⋆ ◎ distl
  dist-swap⋆↔₂r : {r s p : U} →
    swap⋆ ◎ distl {r} {s} {p} ↔₂ dist ◎ (swap⋆ ⊕ swap⋆)
  assocl₊-dist-dist↔₂l : {r s p q : U} →
    ((assocl₊ {r} {s} {p} ⊗ id↔₁ {q}) ◎ dist) ◎ (dist ⊕ id↔₁) ↔₂
    (dist ◎ (id↔₁ ⊕ dist)) ◎ assocl₊
  assocl₊-dist-dist↔₂r : {r s p q : U} →
```

```
    (dist {r} ⊚ (id↔₁ ⊕ dist {s} {p} {q})) ⊚ assocl₊ ↔₂
    ((assocl₊ ⊗ id↔₁) ⊚ dist) ⊚ (dist ⊕ id↔₁)
  assocl⋆-distl↔₂l : {r s p q : U} →
    assocl⋆ {r} {s} ⊚ distl {_×_ r s} {p} {q} ↔₂
    ((id↔₁ ⊗ distl) ⊚ distl) ⊚ (assocl⋆ ⊕ assocl⋆)
  assocl⋆-distl↔₂r : {r s p q : U} →
    ((id↔₁ ⊗ distl) ⊚ distl) ⊚ (assocl⋆ ⊕ assocl⋆) ↔₂
    assocl⋆ {r} {s} ⊚ distl {_×_ r s} {p} {q}
  absorbr0-absorbl0↔₂ : absorbr {O} ↔₂ absorbl {O}
  absorbl0-absorbr0↔₂ : absorbl {O} ↔₂ absorbr {O}
  absorbr↔₂distl-absorb-unite : {r s : U} →
    absorbr ↔₂ (distl {r = r} {s} ⊚ (absorbr ⊕ absorbr)) ⊚ unite₊l
  distl-absorb-unite↔₂absorbr : {r s : U} →
    (distl {r = r} {s} ⊚ (absorbr ⊕ absorbr)) ⊚ unite₊l ↔₂ absorbr
  unite⋆r0-absorbr1↔₂ : unite⋆r ↔₂ absorbr
  absorbr1-unite⋆r-↔₂ : absorbr ↔₂ unite⋆r
  absorbl=swap⋆⊚absorbr : {r : U} → absorbl {r} ↔₂ swap⋆ ⊚ absorbr
  swap⋆⊚absorbr=absorbl : {r : U} → swap⋆ ⊚ absorbr ↔₂ absorbl {r}
  absorbr↔₂[assocl⋆⊚[absorbr⊗id↔]]⊚absorbr : {r s : U} →
    absorbr ↔₂ (assocl⋆ {O} {r} {s} ⊚ (absorbr ⊗ id↔₁)) ⊚ absorbr
  [assocl⋆⊚[absorbr⊗id↔]]⊚absorbr↔₂absorbr : {r s : U} →
    (assocl⋆ {O} {r} {s} ⊚ (absorbr ⊗ id↔₁)) ⊚ absorbr ↔₂ absorbr
  [id↔⊗absorbr]⊚absorbl↔₂assocl⋆⊚[absorbl⊗id↔]⊚absorbr : {r s : U} →
    (id↔₁ ⊗ absorbr {s}) ⊚ absorbl {r} ↔₂
    (assocl⋆ ⊚ (absorbl ⊗ id↔₁)) ⊚ absorbr
  assocl⋆⊚[absorbl⊗id↔]⊚absorbr↔₂[id↔⊗absorbr]⊚absorbl : {r s : U} →
    (assocl⋆ ⊚ (absorbl ⊗ id↔₁)) ⊚ absorbr ↔₂
    (id↔₁ ⊗ absorbr {s}) ⊚ absorbl {r}
  elimbot-A[0⊕B]↔₂l : {r s : U} →
    (id↔₁ {r} ⊗ unite₊l {s}) ↔₂
    (distl ⊚ (absorbl ⊕ id↔₁)) ⊚ unite₊l
  elimbot-A[0⊕B]↔₂r : {r s : U} →
    (distl ⊚ (absorbl ⊕ id↔₁)) ⊚ unite₊l ↔₂ (id↔₁ {r} ⊗ unite₊l {s})
  elimbot-1[A⊕B]↔₂l : {r s : U} →
    unite⋆l ↔₂
    distl ⊚ (unite⋆l {r} ⊕ unite⋆l {s})
  elimbot-1[A⊕B]↔₂r : {r s : U} →
    distl ⊚ (unite⋆l {r} ⊕ unite⋆l {s}) ↔₂ unite⋆l
  fully-distribute↔₂l : {r s p q : U} →
    (distl ⊚ (dist {r} {s} {p} ⊕ dist {r} {s} {q})) ⊚ assocl₊ ↔₂
      ((((dist ⊚ (distl ⊕ distl)) ⊚ assocl₊) ⊚ (assocr₊ ⊕ id↔₁)) ⊚
          ((id↔₁ ⊕ swap₊) ⊕ id↔₁)) ⊚ (assocl₊ ⊕ id↔₁)
  fully-distribute↔₂r : {r s p q : U} →
    ((((dist ⊚ (distl ⊕ distl)) ⊚ assocl₊) ⊚ (assocr₊ ⊕ id↔₁)) ⊚
        ((id↔₁ ⊕ swap₊) ⊕ id↔₁)) ⊚ (assocl₊ ⊕ id↔₁) ↔₂
    (distl ⊚ (dist {r} {s} {p} ⊕ dist {r} {s} {q})) ⊚ assocl₊
```

3-combinators

There is only one 3-combinator, which equates all compatible 2-combinators.

```
- 3-combinators
data _↔₃_ : {X Y : U} {p q : X ↔₁ Y} → (p ↔₂ q) → (p ↔₂ q) → Set where
  trunc : {X Y : U} {p q : X ↔₁ Y} (A B : p ↔₂ q) → A ↔₃ B
```

Appendix B

Full syntax of Π^+

As before, we formalize the syntax of Π^+ in Agda.

Types

```
-- Types
data U : Set where
  O I : U
  _+_ : U → U → U
```

1-combinators

We omit the definition of the negation here as well.

```
-- 1-combinators
data _↔₁_ : U → U → Set where
  unite₊l : {t : U} → O + t ↔₁ t
  uniti₊l : {t : U} → t ↔₁ O + t
  swap₊ : {r s : U} → r + s ↔₁ s + r
  assocl₊ : {r s p : U} → r + (s + p) ↔₁ (r + s) + p
  assocr₊ : {r s p : U} → (r + s) + p ↔₁ r + (s + p)
  id↔₁ : {t : U} → t ↔₁ t
  _◎_ : {r s p : U} → (r ↔₁ s) → (s ↔₁ p) → (r ↔₁ p)
  _⊕_ : {r s p q : U} → (r ↔₁ p) → (s ↔₁ q) → (r + s ↔₁ p + q)

-- Coherence
unite₊r : {t : U} → t + O ↔₁ t
unite₊r = swap₊ ◎ unite₊l

uniti₊r : {t : U} → t ↔₁ t + O
uniti₊r = uniti₊l ◎ swap₊

-- Negation
!↔₁ : {r s : U} → r ↔₁ s → s ↔₁ r
```

2-combinators

```
-- 2-combinators
```

```
data _↔₂_ : {X Y : U} → X ↔₁ Y → X ↔₁ Y → Set where
  assoc◎l : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : p ↔₁ q} →
            (c₁ ◎ (c₂ ◎ c₃)) ↔₂ ((c₁ ◎ c₂) ◎ c₃)
  assoc◎r : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : p ↔₁ q} →
            ((c₁ ◎ c₂) ◎ c₃) ↔₂ (c₁ ◎ (c₂ ◎ c₃))
  assocl₊l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             ((c₁ ⊕ (c₂ ⊕ c₃)) ◎ assocl₊) ↔₂ (assocl₊ ◎ ((c₁ ⊕ c₂) ⊕ c₃))
  assocl₊r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (assocl₊ ◎ ((c₁ ⊕ c₂) ⊕ c₃)) ↔₂ ((c₁ ⊕ (c₂ ⊕ c₃)) ◎ assocl₊)
  assocr₊r : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
             (((c₁ ⊕ c₂) ⊕ c₃) ◎ assocr₊) ↔₂ (assocr₊ ◎ (c₁ ⊕ (c₂ ⊕ c₃)))
  assocr₊l : {r s p q v w : U}
             {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : v ↔₁ w} →
               (assocr₊ ◎ (c₁ ⊕ (c₂ ⊕ c₃))) ↔₂ (((c₁ ⊕ c₂) ⊕ c₃) ◎ assocr₊)
  idl◎l     : {r s : U} {c : r ↔₁ s} → (id↔₁ ◎ c) ↔₂ c
  idl◎r     : {r s : U} {c : r ↔₁ s} → c ↔₂ id↔₁ ◎ c
  idr◎l     : {r s : U} {c : r ↔₁ s} → (c ◎ id↔₁) ↔₂ c
  idr◎r     : {r s : U} {c : r ↔₁ s} → c ↔₂ (c ◎ id↔₁)
  linv◎l    : {r s : U} {c : r ↔₁ s} → (c ◎ !↔₁ c) ↔₂ id↔₁
  linv◎r    : {r s : U} {c : r ↔₁ s} → id↔₁ ↔₂ (c ◎ !↔₁ c)
  rinv◎l    : {r s : U} {c : r ↔₁ s} → (!↔₁ c ◎ c) ↔₂ id↔₁
  rinv◎r    : {r s : U} {c : r ↔₁ s} → id↔₁ ↔₂ (!↔₁ c ◎ c)
  unite₊l↔₂l : {r s : U} {c₁ : O ↔₁ O} {c₂ : r ↔₁ s} →
               (unite₊l ◎ c₂) ↔₂ ((c₁ ⊕ c₂) ◎ unite₊l)
  unite₊l↔₂r : {r s : U} {c₁ : O ↔₁ O} {c₂ : r ↔₁ s} →
               ((c₁ ⊕ c₂) ◎ unite₊l) ↔₂ (unite₊l ◎ c₂)
  uniti₊l↔₂l : {r s : U} {c₁ : O ↔₁ O} {c₂ : r ↔₁ s} →
               (uniti₊l ◎ (c₁ ⊕ c₂)) ↔₂ (c₂ ◎ uniti₊l)
  uniti₊l↔₂r : {r s : U} {c₁ : O ↔₁ O} {c₂ : r ↔₁ s} →
               (c₂ ◎ uniti₊l) ↔₂ (uniti₊l ◎ (c₁ ⊕ c₂))
  swapl₊↔₂ : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} →
             (swap₊ ◎ (c₁ ⊕ c₂)) ↔₂ ((c₂ ⊕ c₁) ◎ swap₊)
  swapr₊↔₂ : {r s p q : U} {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} →
             ((c₂ ⊕ c₁) ◎ swap₊) ↔₂ (swap₊ ◎ (c₁ ⊕ c₂))
  id↔₂      : {r s : U} {c : r ↔₁ s} → c ↔₂ c
  trans↔₂   : {r s : U} {c₁ c₂ c₃ : r ↔₁ s} →
              (c₁ ↔₂ c₂) → (c₂ ↔₂ c₃) → (c₁ ↔₂ c₃)
  _⊡_ : {r s p : U}
        {c₁ : r ↔₁ s} {c₂ : s ↔₁ p} {c₃ : r ↔₁ s} {c₄ : s ↔₁ p} →
        (c₁ ↔₂ c₃) → (c₂ ↔₂ c₄) → (c₁ ◎ c₂) ↔₂ (c₃ ◎ c₄)
  resp⊕↔₂ : {r s p q : U}
            {c₁ : r ↔₁ s} {c₂ : p ↔₁ q} {c₃ : r ↔₁ s} {c₄ : p ↔₁ q} →
            (c₁ ↔₂ c₃) → (c₂ ↔₂ c₄) → (c₁ ⊕ c₂) ↔₂ (c₃ ⊕ c₄)
  id↔₁⊕id↔₁↔₂ : {r s : U} → (id↔₁ {r} ⊕ id↔₁ {s}) ↔₂ id↔₁
  split⊕-id↔₁ : {r s : U} → (id↔₁ {r + s}) ↔₂ (id↔₁ ⊕ id↔₁)
  hom⊕◎↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
    ((c₁ ◎ c₃) ⊕ (c₂ ◎ c₄)) ↔₂ ((c₁ ⊕ c₂) ◎ (c₃ ⊕ c₄))
  hom◎⊕↔₂ : {r s p q v w : U} {c₁ : v ↔₁ r} {c₂ : w ↔₁ s}
    {c₃ : r ↔₁ p} {c₄ : s ↔₁ q} →
          ((c₁ ⊕ c₂) ◎ (c₃ ⊕ c₄)) ↔₂ ((c₁ ◎ c₃) ⊕ (c₂ ◎ c₄))
  - associativity triangle
  triangle₊l : {r s : U} →
    (unite₊r {r} ⊕ id↔₁ {s}) ↔₂ assocr₊ ◎ (id↔₁ ⊕ unite₊l)
  triangle₊r : {r s : U} →
    assocr₊ ◎ (id↔₁ {r} ⊕ unite₊l {s}) ↔₂ unite₊r ⊕ id↔₁
  pentagon₊l : {r s p q : U} →
```

```
    assocr₊ ⊚ (assocr₊ {r} {s} {p + q}) ↔₂
    ((assocr₊ ⊕ id↔₁) ⊚ assocr₊) ⊚ (id↔₁ ⊕ assocr₊)
  pentagon₊r : {r s p q : U} →
    ((assocr₊ {r} {s} {p} ⊕ id↔₁ {q}) ⊚ assocr₊) ⊚ (id↔₁ ⊕ assocr₊) ↔₂
    assocr₊ ⊚ assocr₊
  - unit coherence
  unite₊l-coh-l : {r : U} → unite₊l {r} ↔₂ swap₊ ⊚ unite₊r
  unite₊l-coh-r : {r : U} → swap₊ ⊚ unite₊r ↔₂ unite₊l {r}
  hexagonr₊l : {r s p : U} →
    (assocr₊ ⊚ swap₊) ⊚ assocr₊ {r} {s} {p} ↔₂
    ((swap₊ ⊕ id↔₁) ⊚ assocr₊) ⊚ (id↔₁ ⊕ swap₊)
  hexagonr₊r : {r s p : U} →
    ((swap₊ ⊕ id↔₁) ⊚ assocr₊) ⊚ (id↔₁ ⊕ swap₊) ↔₂
    (assocr₊ ⊚ swap₊) ⊚ assocr₊ {r} {s} {p}
  hexagonl₊l : {r s p : U} →
    (assocl₊ ⊚ swap₊) ⊚ assocl₊ {r} {s} {p} ↔₂
    ((id↔₁ ⊕ swap₊) ⊚ assocl₊) ⊚ (swap₊ ⊕ id↔₁)
  hexagonl₊r : {r s p : U} →
    ((id↔₁ ⊕ swap₊) ⊚ assocl₊) ⊚ (swap₊ ⊕ id↔₁) ↔₂
    (assocl₊ ⊚ swap₊) ⊚ assocl₊ {r} {s} {p}

- Braiding compatible with unitors (redundant; provable from above)
unit-braid : unite₊l {0} ↔₂ swap₊ ⊚ unite₊l
braid-unit : swap₊ ⊚ unite₊l ↔₂ unite₊l {0}
```

3-combinators

Again, there is only one 3-combinator, which equates all compatible 2-combinators.

```
- 3-combinators
data _↔₃_ : {X Y : U} {p q : X ↔₁ Y} → (p ↔₂ q) → (p ↔₂ q) → Set where
  trunc : {X Y : U} {p q : X ↔₁ Y} (A B : p ↔₂ q) → A ↔₃ B
```

Appendix C

Full syntax of $\Pi^\wedge$

As before, we formalize the syntax of $\Pi^\wedge$ in Agda.

Types

This time, types are just natural numbers. We present the definition here, although in the actual code, we use the HoTT-library provided natural numbers (the definition being the same, but with a pragma mapping it to Agda's optimized natural number type).

```
- Types
data N : Type₀ where
  O : N
  S : (n : N) → N
```

1-combinators

Full definition of the negation is, again, omitted.

```
- 1-combinators
data _↔₁^_ : N → N → Type₀ where
  swap₊^ : (S (S n)) ↔₁^ (S (S n))
  id↔₁^  : n ↔₁^ n
  _⊚^_   : (n ↔₁^ m) → (m ↔₁^ o) → (n ↔₁^ o)
  ⊕^_    : (n ↔₁^ m) → ((S n) ↔₁^ (S m))

- Negation
!↔₁^ : t₁ ↔₁^ t₂ → t₂ ↔₁^ t₁
```

2-combinators

```
- 2-combinators
data _↔₂^_ : n ↔₁^ m → n ↔₁^ m → Set where
  assoc⊚l^ : {c₁ : t₁ ↔₁^ t₂} {c₂ : t₂ ↔₁^ t₃} {c₃ : t₃ ↔₁^ t₄} →
    (c₁ ⊚^ (c₂ ⊚^ c₃)) ↔₂^ ((c₁ ⊚^ c₂) ⊚^ c₃)
  assoc⊚r^ : {c₁ : t₁ ↔₁^ t₂} {c₂ : t₂ ↔₁^ t₃} {c₃ : t₃ ↔₁^ t₄} →
    ((c₁ ⊚^ c₂) ⊚^ c₃) ↔₂^ (c₁ ⊚^ (c₂ ⊚^ c₃))
  idl⊚l^   : {c : t₁ ↔₁^ t₂} → (id↔₁^ ⊚^ c) ↔₂^ c
  idl⊚r^   : {c : t₁ ↔₁^ t₂} → c ↔₂^ id↔₁^ ⊚^ c
```

```
idr⊚l^    : {c : t₁ ↔₁^ t₂} → (c ⊚^ id↔₁^) ↔₂^ c
idr⊚r^    : {c : t₁ ↔₁^ t₂} → c ↔₂^ (c ⊚^ id↔₁^)
linv⊚l^   : {c : t₁ ↔₁^ t₂} → (c ⊚^ !↔₁^ c) ↔₂^ id↔₁^
linv⊚r^   : {c : t₁ ↔₁^ t₂} → id↔₁^ ↔₂^ (c ⊚^ !↔₁^ c)
rinv⊚l^   : {c : t₁ ↔₁^ t₂} → (!↔₁^ c ⊚^ c) ↔₂^ id↔₁^
rinv⊚r^   : {c : t₁ ↔₁^ t₂} → id↔₁^ ↔₂^ (!↔₁^ c ⊚^ c)
id↔₂^ : {c : t₁ ↔₁^ t₂} → c ↔₂^ c
_■^_ : {c₁ c₂ c₃ : t₁ ↔₁^ t₂} →
       (c₁ ↔₂^ c₂) → (c₂ ↔₂^ c₃) → (c₁ ↔₂^ c₃)
_⊡^_ : {c₁ : t₁ ↔₁^ t₂} {c₂ : t₂ ↔₁^ t₃} {c₃ : t₁ ↔₁^ t₂} {c₄ : t₂ ↔₁^ t₃} →
       (c₁ ↔₂^ c₃) → (c₂ ↔₂^ c₄) → (c₁ ⊚^ c₂) ↔₂^ (c₃ ⊚^ c₄)
- New ones
⊕id↔₁↔₂^ : ⊕^ id↔₁^ {n = n} ↔₂^ id↔₁^ {n = (S n)}
!⊕id↔₁↔₂^ : id↔₁^ {n = (S n)} ↔₂^ ⊕^ id↔₁^ {n = n}
hom⊚⊕↔₂^ : {c₁ : (n) ↔₁^ (m)} {c₂ : (m) ↔₁^ (o)} →
        ((⊕^ c₁) ⊚^ (⊕^ c₂)) ↔₂^ ⊕^ (c₁ ⊚^ c₂)
resp⊕↔₂ :
        {c₁ : (n) ↔₁^ (m)} {c₂ : (n) ↔₁^ (m)} → (c₁ ↔₂^ c₂) → (⊕^ c₁) ↔₂^ (⊕^ c₂)
hom⊕⊚↔₂^ : {c₁ : (n) ↔₁^ (m)} {c₂ : (m) ↔₁^ (o)} →
        ⊕^ (c₁ ⊚^ c₂) ↔₂^ ((⊕^ c₁) ⊚^ (⊕^ c₂))

swapr₊↔₂^ : {c : (n) ↔₁^ (m)}
  → (⊕^ (⊕^ c)) ⊚^ swap₊^ ↔₂^ swap₊^ ⊚^ (⊕^ (⊕^ c))
swapl₊↔₂^ : {c : (n) ↔₁^ (m)}
  → swap₊^ ⊚^ (⊕^ (⊕^ c)) ↔₂^ (⊕^ (⊕^ c)) ⊚^ swap₊^

hexagonl₊l : (swap₊^ {S n}) ⊚^ ((⊕^ (swap₊^ {n})) ⊚^ swap₊^)
  ↔₂^ (⊕^ swap₊^) ⊚^ (swap₊^ ⊚^ ⊕^ swap₊^)

hexagonl₊r : (⊕^ swap₊^) ⊚^ (swap₊^ ⊚^ ⊕^ swap₊^)
  ↔₂^ (swap₊^ {S n}) ⊚^ ((⊕^ (swap₊^ {n})) ⊚^ swap₊^)
```

3-combinators

Again, there is only one 3-combinator, which equates all compatible 2-combinators.

```
- 3-combinators
data _↔₃^_ :{p q : n ↔₁^ m} → (p ↔₂^ q) → (p ↔₂^ q) → Set where
  trunc : {p q : n ↔₁^ m} (A B : p ↔₂^ q) → A ↔₃^ B
```